Basi di dati

Alessandro Carella

Indice

1 Introduzione **5**
- 1.1 Sistema organizzativo, informativo e informatico . . . 5
- 1.2 Gestione delle informazioni . . . 6
- 1.3 Dati e informazioni . . . 7

2 Sistemi di gestione di basi di dati **8**
- 2.1 Modelli di dati . . . 10
  - 2.1.1 Schemi e istanze . . . 11
  - 2.1.2 Livelli di astrazione nei DBMS . . . 11
  - 2.1.3 Indipendenza dei dati . . . 12
- 2.2 Linguaggi e utenti delle basi di dati . . . 12
  - 2.2.1 Linguaggi per basi di dati . . . 13
  - 2.2.2 Utenti e progettisti . . . 13
- 2.3 Vantaggi e svantaggi dei DBMS . . . 14

3 Metodologie e modelli per il progetto **15**
- 3.1 Il ciclo di vita dei sistemi informativi . . . 15
- 3.2 Metodologie di progettazione e basi di dati . . . 17

4 Il modello relazionale **20**
- 4.1 Struttura del modello relazionale . . . 20
  - 4.1.1 Modelli logici nei sistemi di basi di dati . . . 20
  - 4.1.2 Relazioni e tabelle . . . 21
  - 4.1.3 Relazioni con attributi . . . 22
  - 4.1.4 Relazioni e basi di dati . . . 23
  - 4.1.5 Informazione incompleta e valori nulli . . . 27
- 4.2 Vincoli di integrità . . . 29
  - 4.2.1 Vincoli di tupla . . . 31

4.2.2 Chiavi . . . 31
4.2.3 Chiavi e valori nulli . . . 32
4.2.4 Vincoli di integrità referenziale . . . 33

5 Algebra relazionale 35
5.1 Algebra relazionale . . . 36
5.1.1 Unione, intersezione e differenza . . . 36
5.1.2 Ridenominazione . . . 37
5.1.3 Selezione . . . 39
5.1.4 Proiezione . . . 41
5.1.5 Join . . . 42
5.1.6 Divisione . . . 50
5.1.7 Interrogazioni in algebra relazionale . . . 51
5.1.8 Equivalenza di espressioni algebriche . . . 53
5.1.9 Algebra con valori nulli . . . 57
5.1.10 Viste . . . 59
5.2 Calcolo relazionale . . . 61
5.2.1 Calcolo relazione su domini . . . 62
5.2.2 Pregi e difetti del calcolo su domini . . . 66
5.2.3 Calcolo su tuple con dichiarazioni di range . . . 67

6 Il modello Entità-Relazione 71
6.1 I costrutti principali del modello . . . 71
6.1.1 Entità . . . 71
6.1.2 Relazioni o associazioni . . . 72
6.1.3 Attributi . . . 74
6.1.4 Costruzione di schemi con i costrutti di base . . . 75
6.2 Altri costrutti del modello . . . 76
6.2.1 Cardinalità delle relazioni . . . 76
6.2.2 Cardinalità degli attributi . . . 78
6.2.3 Identificatori delle entità . . . 78
6.2.4 Generalizzazioni . . . 80

7 Progettazione concettuale 82
7.1 La raccolta e l'analisi dei requisiti . . . 82
7.2 Rappresentazione concettuale di dati . . . 88
7.2.1 Criteri generali di rappresentazione . . . 88
7.3 Strategie di progetto . . . 89

7.3.1 Strategia top-down . . . 89
7.3.2 Strategia bottom-up . . . 91
7.3.3 Strategia inside-out . . . 93
7.3.4 Strategia mista . . . 94
7.4 Qualità di uno schema concettuale . . . 94
7.5 Una metodologia generale . . . 96

8 Progettazione logica 98
8.1 Fasi della progettazione logica . . . 98
8.2 Analisi della prestazioni su schemi E-R . . . 99
8.3 Ristrutturazione di schemi E-R . . . 103
8.3.1 Analisi delle ridondanze . . . 104
8.3.2 Eliminazione delle generalizzazioni . . . 106
8.3.3 Partizionamento/accorpamento di concetti . . . 107
8.3.4 Scelta degli identificatori principali . . . 111
8.4 Traduzione verso il modello relazionale . . . 112
8.4.1 Entità e associazioni molti a molti . . . 112
8.4.2 Associazioni uno a molti . . . 113
8.4.3 Entità con identificatore esterno . . . 114
8.4.4 Associazioni uno a uno . . . 114
8.4.5 Traduzione di schemi complessi . . . 116
8.4.6 Tabelle riassuntive . . . 118

9 Normalizzazione 121
9.1 Ridondanza e anomalie . . . 121
9.2 Dipendenze funzionali . . . 122
9.3 Forma normale di Boyce e Codd . . . 124
9.3.1 Definizione di forma normale di Boyce e Codd . . . 124
9.3.2 Decomposizione in forma normale di Boyce e Codd . . . 125
9.4 Proprietà delle decomposizioni . . . 126
9.4.1 Decomposizione senza perdita . . . 126
9.4.2 Conservazione delle dipendenze . . . 128
9.4.3 Qualità delle decomposizioni . . . 129
9.5 Terza forma normale . . . 129
9.5.1 Limitazioni della forma normale di Boyce e Codd . . . 129
9.5.2 Definizione di terza forma normale . . . 130
9.5.3 Decomposizione in terza forma normale . . . 130
9.5.4 Altre forme normali . . . 131

9.5.5 Normalizzazione e scelta degli attributi 133
9.6 Progettazione di basi di dati e normalizzazione 135
9.6.1 Verifiche di normalizzazione su entità 135
9.6.2 Verifiche di normalizzazione su associazioni 137
9.6.3 Ulteriori decomposizioni di associazioni 138
9.6.4 Ulteriori decomposizioni di schemi concettuali 139

Capitolo 1

Introduzione

1.1 Sistema organizzativo, informativo e informatico

Definizione 1.1: Sistema organizzativo
Insieme di **risorse** e **regole** per lo svolgimento **coordinato** delle **attività** al fine di perseguire degli **scopi**

Definizione 1.2: Sistema informativo
Un sistema informativo è:

- **Componente** di una **organizzazione** che gestisce **informazioni** (di interesse)
  - è parte del sistema organizzativo
  - esegue/gestisce processi informativi ovvero processi che coinvolgono informazioni
  - ogni organizzazione ha un sistema informativo, eventualmente non esplicitato nella struttura
  - è spesso di supporto ad altri sottosistemi ed è da studiare nel contesto in cui è inserito
  - è spesso suddiviso in sottosistemi piu' o meno fortemente integrati

Esiste una dipendenza tra un sistema informativo e un **sistema informatico** per via dell'automatizzazione ed esistono organizzazioni la cui ragion d'essere è la gestione di inforazioni e che operano da secoli.

Un sistema informatico è la porzione automatizzata del sistema informativo.
Nel tempo c'è stata un'evoluzione di sistemi informativi verso **razionalizzazione** e **standardizzazione** e delle procedure e dell'organizzazione delle informazioni.
Una risorsa è una qualsiasi fonte o mezzo che valga a fornire aiuto.

1.2 Gestione delle informazioni

Ci sono diverse modalità di gestione delle informazioni:

- idee informali;
- linguaggio naturale;
- disegni, grafici, schemi;
- numeri e codici.

Inoltre, diversi sono i supporti sui quali memorizzare le informazioni:

- memoria umana;
- carta;
- dispositivi elettronici.

Nelle attività standardizzate di sistemi informativi complessi si è giunti ad una organizzazione e codifica, ad esempio nei servizi anagrafici dalle registrazioni discorsive si è giunti a:

- nome e cognome;
- estremi anagrafici;
- codice fiscale.

1.3 Dati e informazioni

Nei sistemi informatici, per ragioni tecnlogiche e per la semplicità dei meccanismi di gestione, il concetto di rappresentazione e codifica delle informazioni viene portato all'estremo: le informazioni vengono rappresentate per mezzo di dati che hanno bisogno di essere interpretati per fornire informazioni. è difficile fare una distinzione tra dato e informazione ma per rappresentare in modo approssimativo il concetto possiamo dire che i dati da soli non hanno un significato ben definito ma una volta interpretati correttamente essi forniscono delle informazioni. Quindi l'informazione deriva da un processo di interpretazione e corellazione di dati, tutto cio' che produce variazione nel patrimonio conoscitivo di un soggetto è informazione.
Per fare un esempio di distinzione tra dato e informazione consideriamo la stringa 'BRINDISI 20 26 18', questi sono 4 dati e se correllati alla domanda 'Quali sono le temperature attuali, minime e massime di oggi?' allora i dati possono essere interpretati per fornire informazione e arricchire la conoscenza.
I sistemi informatici si sono limitati a gestire solo i dati e le informazioni da esse estratte.

Capitolo 2

Sistemi di gestione di basi di dati

L'attenzione ai dati ha caratterizzato le applicazioni dell'informatica fin dalle sue origini, ma software dedicati alla loro gestione sono stati realizzati a partire dalla fine degli anni Sessanta.
L'approccio convenzionale alla gestione dei dati sfrutta la presenza di archivi o *file* per memorizzare i dati in modo persistente sulla memoria di massa. Un file consente di memorizzare e ricercare dati, ma fornisce semplici meccanismi di accesso e condivisione. Secondo questo approccio, le procedure scritte in un linguaggio di programmazione sono autonome, ciascuna utilizza uno o piu' file privati e dati di interesse sono replicati con ridondanza e possibilità di incoerenza.
Le basi di dati sono state concepite per superare questi inconvenienti, gestendo in modo integrato e flessibile le informazioni di interesse per diversi soggetti, limitando ridondanza e incoerenza.
Una base di dati è una collezione di dati, di interesse per una o piu' applicazioni.
Un *sistema di gestione di basi di dati (Data Base Management System, DBMS)* è un sistema software in grado di gestire collezione di dati che siano *grandi*, *condivise* e *persistenti*, assicurando la loro *affidabilità* e *privatezza*. Come ogni prodotto informatico, un DBMS deve essere *efficiente* ed *efficace*. Una base di dati è una collezione di dati gestita da un DBMS.
Chiariamo con precisione alcune caratteristiche dei DBMS e delle basi di dati:

- le basi di dati sono *grandi*, nel senso che possono avere dimensioni molto maggiori della memoria centrale disponibile, di conseguenza i DBMS devono prevedere una gestione dei dati in memoria secondaria, con architettura molto sofisticate;

- le basi di dati sono *condivise*, nel senso che diversi utenti e applicazioni possono accedervi secondo opportune modalità. In questo modo si riduce la ridondanza dei dati e la conseguente inconsistenza in quanto se esistono copie diverse di uno stesso dato è possibile che queste non siano uguali. Per garantire l'accesso condiviso ai dati, il DBMS dispone di un meccanismo apposito, detto *controllo di concorrenza*;
- le basi di dati sono *persistenti*, cioè hanno un tempo di vita non limitato a quello delle singole esecuzioni dei programmi che le utilizzano;
- i DBMS garantiscono *affidabilità*, cioè la capacità di conservare intatto il contenuto della base di dati in caso di malfunzionamenti hardware o software. A questo scopo i DBMS offrono funzionalità di *salvataggio* e *ripristino*;
- i DBMS garantiscono la *privatezza* dei dati; ciasun utente, opportunamente riconosciuto, viene abilitato a svolgere solo determinate azioni sui dati, attraverso meccanismi di *autorizzazione*;
- i DBMS sono *efficienti*, cioè sono capaci di svolgere le operazioni utilizzando un insieme di risorse, tempo e spazio, che sia accettabile per gli utenti;
- i DBMS sono *efficaci* in quanto sono capaci di rendere produttive le attività dei loro utenti.

La gestione di collezioni di dati grandi e persistenti è possibile anche con strumenti meno sofisticati dei DBMS, anche con i file, introdotti per gestire insiemi di dati localmente ad una specifica applicazione. I DBMS sono stati concepiti per estendere le funzioni del file system fornendo la possibilità di accesso condiviso agli stessi dati da parte di piu' utenti e applicazioni.
Inoltre, nei programmi tradizionali che accedono a file, ogni programma contiene una descrizione della struttura del file con rischi di incoerenza fra le varie descrizioni e i file stessi, nei DBMS, invece, c'è una porzione di basi di dati, chiamata *dizionario* o *catalogo* che contiene una descrizione centralizzata dei dati che è accessibile dai vari programmi.
Diverse sono le tipologie di basi di dati e DBMS, come:

- basi di dati multimediali;
- data warehouse;

- GIS;
- basi di dati in tempo reale;
- basi di dati attive;
- NoSQL;
- Access;
- Oracle;
- PostgreSQL;
- MySql

e tante altre.

2.1 Modelli di dati

Un *modello di dati* è un insieme di concetti utilizzati per organizzare i dati di interesse e descriverne la struttura in modo che risulti comprensibile a un elaboratore.
Ogni modello di dati fornisce *meccanismi di strutturazione* analoghi ai costruttori di tipo dei linguaggi di programmazione, che permettono di definire nuovi tipi sulla base di tipi predefiniti e costruttori di tipo.
Il *modello relazionale* dei dati, il piu' diffuso, permette di definire tipi per mezzo del costruttore *relazione*, che consente di organizzare i dati in insiemi di record a struttura fissa. Una relazione viene spesso rappresentata per mezzo di una tabella, le cui righe rappresentano dei record e le colonne corrispondono ai campi dei record, ad esempio:

Docenza	
Corso	NomeDocente
Basi di dati	Rossi
Reti	Neri
Linguaggi	Verdi

Oltre al modello relazionale sono stati definiti altri tipi di modelli:

- il *modello gerarchico* basato sull'uso di strutture ad albero;
- il *modello reticolare* basato sull'uso di grafi;
- il *modello a oggetti*, sviluppato come evoluzione del modello relazionale che estende alle basi di dati il paradigma di programmazione ad oggetti;
- il *modello XML* sviluppato come rivisitazione del modello gerarchico, in cui i dati vengono presentati assieme alla loro descrizione e non devono sottostare rigidamente a un'unica struttura logica;
- modelli semi-strutturati e flessibili, sviluppati nel contesto dei sistemi NoSQL che cercano di superare alcune limitazioni dei sistemi relazionali in termini di prestazioni e di rigidità di organizzazione.

I modelli dei dati vengono detti *logici* per sottolineare il fatto che le strutture utilizzate pur essndo astratte riflettono una particolare organizzazione (ad albero, grafo, ecc.). Piu' recentemente ai modelli logici, sono stati introdotti i modelli *concettuali* utilizzati per descrivere i dati indipendenmente dalla scelta del modello logico. Il loro nome deriva dal fatto che essi tendono a descrivere i *concetti* del mondo reale e vengono usati nella fase preliminare del processo di progettazione di basi di dati.

2.1.1 Schemi e istanze

Nelle basi di dati esiste una parte invariante nel tempo, detto *schema* della base di dati, costituita dalle caratteristiche dei dati, e una parte variabile nel tempo, detta *istanza* o *stato* della base di dati, costituita dai valori effettivi. Lo schema di una relazione è costituito dalla sua intestazione, ad esempio:

$$Docenza \triangledown Corso, NomeDocente \partial$$

Le righe della tabella, invece, variano nel tempo, l'istanza di una relazione è costituita dall'insieme delle sue righe,

2.1.2 Livelli di astrazione nei DBMS

La notazione di modello descritta in precedenza puo' essere sviluppata tenendo presente di altri fattori nella descrizione dei dati. In particolare, esiste un'architettura standard per DBMS articolata su tre lvielli, detti: *esterno*, *logico* e *interno* e per ogni livello esiste uno schema.

- Lo *schema logico* costituisce una descrizione dell'intera basi di dati per mezzo del modello logico adottato dal DBMS;
- Lo *schema interno* costituisce la rappresentazione dello schema logico per mezzo di strutture fisiche di memorizzazione, ad esempio una relazione puo' essere realizzata tramite file sequenziale, file hash o altri approcci;
- Uno *schema esterno* costituisce la descrizione di una porziona della base di dati di interesse, per mezzo del modello logico.

Nei sistemi moderni il livello esterno non è esplicitamente presente, ma è possibile definire relazioni derivate, o *viste*.

2.1.3 Indipendenza dei dati

L'architettura a livelli cosi' definita garantisce l'*indipendenza dei dati* che è la principale prorietà dei DBMS. L'indipendenza dei dati permette a utenti e programmi applicativi che utlizzano una base di dati di interagire a un elevato livello di astrazione, cioè prescinde dai dettagli realizzativi.
L'indipendenza dei dati puo' essere caratterizzata come indipendenza fisica e logica:

- l'*indipendenza fisica* consente di interagire con il DBMS in modo indipendente dalla struttura fisica dei dati. è possibile modificare le strutture fisiche, come le modalità di orgaizzazione dei file gestiti dal DBMS, senza influire sulle descrizioni dei dati ad alto livello e quindi sui programmi che utilizzano i dati.
- l'*indipendenza logica* consente di interagire con il livello esterno della base di dati in modo indipendente dal livello logico. Ad esempio, è possibile aggiungere uno schema esterno in base alle esigenze di un nuovo utente o modificare uno schema esterno senza dover modificare lo schema logico e quindi l'organizzazione fisica dei dati.

Gli accessi alla base di dati avvengono solo attraverso il livello esterno ed è il DBMS che traduce le operazioni in termini dei livelli sottostanti.

2.2 Linguaggi e utenti delle basi di dati

I DBMS sono caratterizzati dalla presenza di diversi linguaggi per la gestione dei dati e dalla presenza di molteplici utenti.

2.2.1 Linguaggi per basi di dati

è possibile effettuare diverse operazioni su un DBMS, in particolare quelle relative agli schemi e alle istanze. I linguaggi per basi di dati si distinguono in due categorie:

- *linguaggi di definizione dei dati (Data Definition Language, DDL)*, utilizzati per definire gli schemi logici, esterni e fisici e le autorizzazioni per l'accesso;
- *linguaggi di manipolazione dei dati (Data Manipulation Language, DML)*, utilizzati per l'interrogazione e l'aggiornamento delle istanze di basi di dati.

L'accesso ai dati puo' essere effettuato con varia modalità:

- tramite linguaggi testuali interattivi, come il linguaggio SQL. Ad esempio, possiamo definire la struttura della relazione *Docenza*, sottoponendo a un interprete SQL la seguente istruzione che appartiene al DDL:

```
create table Docenza (
        Corso         character(20),
        NomeDocente   character(30)
)
```

- tramite comandi simili a quelli interattivi immersi in linguaggi di programmazione. Questi linguaggi di programmazione si dicono *linguaggi ospite* perchè ospitando comandi scritti nel linguaggio per basi di dati. Questo approccio si utilizza spesso nella realizzazione di applicazioni parametriche e di farle eseguire ad un utente che non conosce SQL;
- tramite comandi simili a quelli interattivi immersi in linguaggi di sviluppo ad hoc, spesso con funzionalità specifiche;
- tramite interfacce amichevoli che permettono di sintetizzare interrogazioni senza usare un linuaggio testuale.

2.2.2 Utenti e progettisti

Diverse categorie di persone possono interagire con una base di dati o con un DBMS.

- l'*amministratore della base di dati (DataBase Administrator, DBA)* è la persona o gruppo di persone responsabile della progettazione, controllo e amministrazione della base di dati, l'acquisizione di software e risorse hardware e controlla l'efficienza delle operazioni;
- i *progettisti e programmatori di applicazioni* definiscono e realizzano i programmi che accedono alla base di dati. Essi utilizzano i linguaggi DML;
- gli *utenti* utilizzano la base di dati per le proprie attività e gli utenti si dividono in due categorie:
  - *utenti finali* o *terminalisti* che utilizzano *transazioni*, cioè programmi che realizzano attività predefinite e di frequenza elevata;
  - *utenti casuali* in grado di impiegare i linguaggi interattivi per l'accesso alla base di dati, formulando interrogazioni o aggiornamenti di vario tipo. Essi possono essere specializzati e interagire frequentemente con la base di dati.

2.3 Vantaggi e svantaggi dei DBMS

Dei DBMS possiamo elencare i seguenti vantaggi:

- I DBMS permettono di considerare i dati come una risorsa comune di un'organizzazione, a disposizione di tutte le sue componenti;
- la base di dati fornisce un modello unificato e preciso dalla parte del mondo reale di interesse per l'organizzazione;
- utilizzando un DBMS è possibile un controllo centralizzato dei dati, che puo' essere arricchito da forme di standardizzazione e beneficiare di "economie di scala";
- la condivisione permette di ridurre ridondanze e inconsistenze;
- l'indipendenza dei dati, fornisce lo sviluppo di applicazioni piu' flessibili e facilmente modificabili.

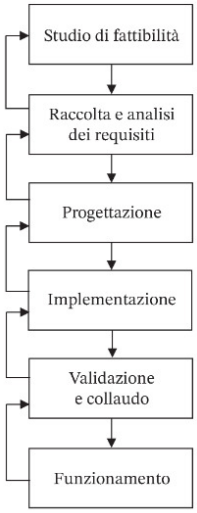
In casi di applicazioni con uno o pochi utenti, senza accessi concorrenti possono essere utlilizzati file rispetto ai DBMS in quanto puo' risultare sconveniente.

Capitolo 3

Metodologie e modelli per il progetto

3.1 Il ciclo di vita dei sistemi informativi

La progettazione di una basd di dati costituisce solo una delle componenti del processo di sviluppo di un sistema informativo complesso e va quindi inquadrata in un contesto piu' ampio, quello del *ciclo del vita* dei sistemi informativi.
Il ciclo di vita di un sistema informativo comprende le seguenti attività.



- **Studio di fattibilità**. Serve a definire, nel modo piu' preciso possibile, i

costi delle varie alternative possibili e a stabilire le priorità di realizzazione delle varie componenti del sistema.

- **Raccolta e analisi dei requisiti**. Consiste nell'individuazione e nello studio delle proprietà e delle funzionalità che il sistema informativi dovrà avere. Questa fase richiede un'iterazione con gli utenti del sistema e produce una descrizione completa, ma informale, dei dati coinvolti e delle operazioni su di essi. Vengono inoltre stabiliti i requisiti software e hardware del sistema informativo.

- **Progettazione**. Si divide generalmente in *progettazione di dati* e *progettazione delle applicazioni*. Nella prima si individua la struttura e l'organizzazione che i dati dovranno avere, nell'altra si definiscono le caratteristiche dei programmi applicativi.
  Le due attività sono complementari e possono procedere in parallelo o in cascata. Le descrizioni dei dati e delle applicazioni prodotti sono formali e fanno riferimento a specifici modelli.

- **Implementazione**. Consiste nella realizzazione del sistema informativo secondo la struttura e le caratteristiche definite nella fase di progettazione. Viene costruita e popolata la base di dati e viene prodotto il codice dei programmi.

- **Validazione e collaudo**. Serve a verificare il corretto funzionamento e la qualità del sistema informativo. La sperimentazione deve prevedere tutte le condizioni operative.

- **Funzionamento**. In questa fase il sistema informativo diventa operativo ed esegue i compiti per i quali era stato progettato. Se non si verificano malfunzionamenti delle funzionalità del sistema, questa attività richiede solo operazioni di gestione e manutenzione.

Il processo non è quasi mai strettamente sequenziale in quanto, durante l'attività di una delle attività citate, bisogna rivedere decisioni prese nell'attività precedente. Inoltre, oltre alle attività descritte, si aggiunge quella di *prototipizzazione*, che consiste nell'uso di specifici strumenti software pe la realizzazione rapida di una versione semplificata del sistema informativo, con la quale sperimentare le sue funzionalità. La verifica del prototipo puo' portare ad un'eventuale revisione del progetto.

I dati hanno un ruolo centrale nei sistemi informativi e questo giustifica uno studio autonomo relativo alla progettazione delle basi di dati.

3.2 Metodologie di progettazione e basi di dati

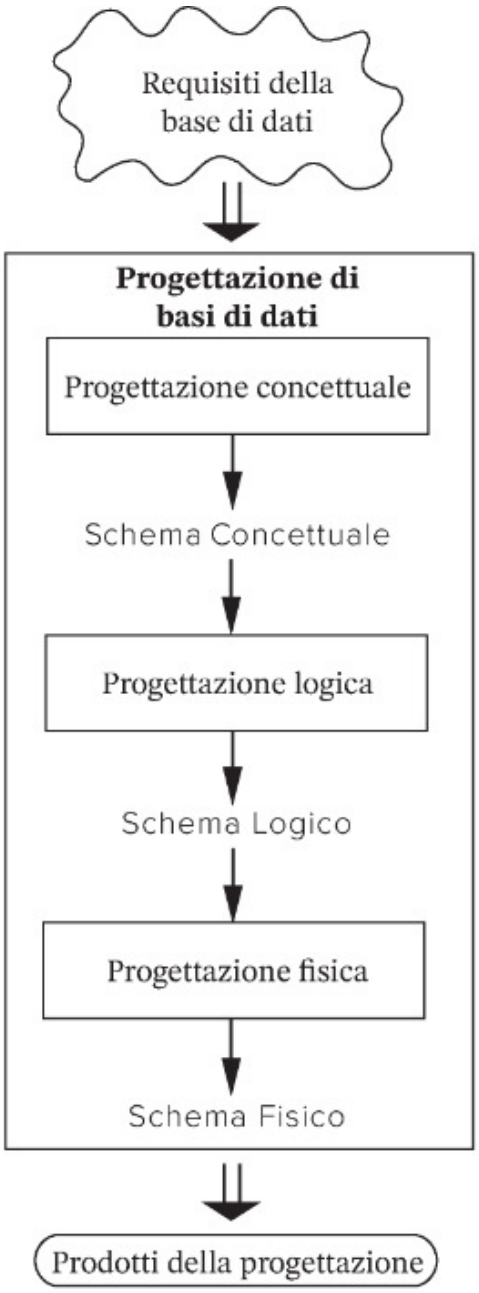
Bisogna precisare cosa si intende per *metodologia di progettazione* e quali sono le propriotà che deve garantire. Una metodologia di progettazione consiste in:

- una *decomposizione* dell'interva attività di progetto in passi successivi indipendenti tra loro;
- una serie di *strategie* da seguire nei vari passi e alcuni *criteri* per la scelta in caso di alternative;
- alcuni *modelli di riferimento* per descrivere i dati di ingresso e uscita delle varie fasi.

Le proprietà che una metodologie deve garantire sono principalmente:

- la *generalità* rispetto alle applicazioni e ai sistemi di gioco (e quindi la possibilità di utilizzo indipendentemente dal problema allo studio e dagli strumenti a disposizione);
- la *qualità del prodtto* in termini di correttezza, completezza ed efficienza rispetto alla risorse impiegato;
- la *facilità d'uso* delle strategie e dei modelli di riferimento.

Nelle basi di dati, nel corso degli anni si è consolidata una metodologia di progetto che è articolata in tre fasi principali da effettuare in cascata. Questa metodologia separa le decisioni relative a *cosa* rappresentare in una base di dati, da quelle relative a *come* farlo.



- **Progettazione concettuale**. Il prodotto di questa fase viene chiamato *schema concettuale* e fa riferimento a un *modello concettuale* dei dati. I modelli concettuali ci consentono di descrivere l'organizzazione dei dati

a un alto livello di astrazione senza tenere conto degli aspetti implementativi. In questa fase, il progettista deve cercare di rappresentare il *contenuto informativo* delle base di dati non pensando a dettagli implementativi. Il modello concettuale piu' noto è il modello **Entity-Relationship**.

- **Progettazione logica**. Consiste nella traduzione dello schema concettuale definito nella fase precedente, adottando il modello di rappresentazione di dati del DBMS a disposizione. Il prodotto di questa fase viene denominato *schema logico* delle base di dati e fa riferimento a un *modello logico* dei dati. Il modello logico ci consente di descrivere i dati prescindendo ancora da dettagli fisici, ma concreta perchè disponile nei sistemi di gestione di basi di dati. In questa fase, le scelte progettuali si basano su criteri di ottimizzazione delle operazioni da effettuare sui dati. Inoltre, si fa uso di tecniche formali di verifica della qualità dello schema logico ottenuto. Nel caso del modello relazione dei dati, la tecnica usata è quella della *normalizzazione*

- **Progettazione fisica**. In questa fase lo schea logico viene completato con la specifica dei parametri fisici di memorizzazione dei dati. Il prodotto di questa fase viene chiamato *schema fisico* e fa riferimento a un *modello fisico* dei dati che dipende dal DBMS scelto e si basa sui criteri di organizzazione fisica dei dati.

Analizziamo come i requisiti della base di dati vengono utilizzati nelle varie fasi della progettazione. Dobbiamo prima distinguere le *specifiche sui dati* che riguardano il contenuto della base di dati, e *specifiche sulle operazioni* che riguardano l'uso che utenti e applicazioni fanno della base di dati. Nella progettazione concettuale si fa uso delle specifiche sui dati mentre le specifiche sulle operazioni servono solo a verificare che lo schema concettuale sia completo, cioè che contenga le informazioni necessarie per eseguire tutte le operazioni previste.

Capitolo 4

Il modello relazionale

4.1 Struttura del modello relazionale

4.1.1 Modelli logici nei sistemi di basi di dati

Il *modello relazionale* si basa su due concetti fondamentali, quello di *relazione* e *tabella*.
La nozione di *relazione* proviene dalla teoria di insiemi in matematica mentre il concetto di *tabella* è intuitivo.
Il modello relazionale risponde al requisito d'indipendenza dei dati che prevede una distinzione, nella descrizione dei dati, fra il livello *fisico* e il livello *logico*; gli utenti che accedono ai dati e i programmatori che sviluppano le applicazioni fanno riferimento solo al livello logico e i dati descritti al livello logico sono concretamente organizzati per mezzo di strutture fisiche, ma non è necessario conoscerne i dettagli per accedervi. Invece, il modello reticolare e gerarchico, includevano riferimenti alla struttura realizzativa, attraverso l'uso di puntatori e l'ordinamento fisico dei dati.
Il termine *relazione* viene utilizzato in tre accezioni:

1. *relazione matematica* secondo la definizione della teoria degli insiemi;
2. *relazione* secondo la definizione del modello relazionale che presenta differenze a quella della teoria degli insiemi;
3. *relazione* come traduzione di *relationship* costrutto del modello concettuale *Entità-Relazione* utilizzato per descrivere legami tra entità del mondo reale.

4.1.2 Relazioni e tabelle

In matematica, dati due insiemi $\mathcal{D}_1$ e $\mathcal{D}_2$, si chiama *prodotto cartesiano* di $\mathcal{D}_1$ e $\mathcal{D}_2$, l'insieme di coppie ordinate $\nabla v_1, v_2 \partial$, tali che v_1 è un elemento di $\mathcal{D}_1$ e v_2 è un elemento di $\mathcal{D}_2$.

Esempio : Prodotto cartesiano

Dati due insiemi $A = =:1, 2, 3, 4=:$ e $B = =:a, b=:$, il prodotto cartesiano è:

$$A \times B = =:\nabla 1, a\partial, \nabla 1, b\partial, \nabla 2, a\partial, \nabla 2, b\partial, \nabla 3, a\partial, \nabla 3, b\partial, \nabla 4, a\partial, \nabla 4, b\partial=:$$

ovvero l'insieme di tutti le possibili coppie dove il primo elemento appartiene ad A e il secondo a B.

Una *relazione matematica* sugli insiemi $\mathcal{D}_1$ e $\mathcal{D}_2$, detti *domini* della relazione, è un sottoinsieme di $\mathcal{D}_1 \times \mathcal{D}_2$.
Le relazioni possonoe essere espresse graficamente sotto forma tabellare, ad esempio prendendo spunto dall'esempio del prodotto cartesiano, possiamo rappresentare la relazione cosi:

1	a
1	b
2	a
2	b
4	a
4	b

1	a
1	b
4	b

Le definizioni di prodotto cartesiano e relazione date precedentemente hanno fatto riferimento a due insiemi, ma possono essere generalizzate rispetto al numero di insiemi. Dati $n > 0$ insiemi $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$, anche non distinti, il prodotto cartesiano di $\mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_n$ è costituito dalle n-uple $\nabla v_1, v_2, \ldots, v_n\partial$ tali che $v_1 \in \mathcal{D}_i \quad 1 \leq i \leq n$.
Una relazione matematica sui domini $\mathcal{D}_1$, $\mathcal{D}_2$, ... , $\mathcal{D}_n$ è un sottoinsieme del prodotto cartesiano $\mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_n$. Il numero delle n componenti del prodotto cartesiano viene detto *grado* del prodotto cartesiano e della relazione. Il numero di elementi della relazione viene detto *cardinalità* della relazione.

4.1.3 Relazioni con attributi

Una relazione matematica è un insieme di n-uple ordinate $\nabla v_1, v_2, \dots, v_n \partial$ ove $v_1 \in \mathcal{D}_1$, $v_2 \in \mathcal{D}_2, \dots$, $v_n \in \mathcal{D}_n$. Ciascuna n-upla contiene dati fra loro collegati, stabilisce un legame fra loro, ad esempio una n-upla siffatta: "Juventus, Lazio, 3, 1" indica che il risultato della partita tra Juventus e Lazio è 3 a 1.
Una relazione è un insieme quindi:

- non è definito un ordinamento fra le n-uple;
- le n-uple di una relazione sono distinte l'una dall'altra, in quanto tra gli elementi di un insieme non ce ne possono essere due uguali fra loro.

Ciascuna n-upla è, al proprio interno, *ordinata*: l'i-esimo valore di ciascuna proviene dall'i-esimo dominio. L'ordinamento appena descritto corrisponde ad una caratteristica insoddisfacente del concetto di relazione matematica rispetto alla possibilità di organizzare e utilizzare i dati. Infatti, in vari contesti dell'informatica, si tende a privilegiare notazioni *non-posizionali* rispetto a quelli posizionali: si tende a utilizzare quelli posizionali solo quando l'ordinamento corrisponde ad una esigenza intrinseca, come accade nei problemi di natura matematica, in cui gli array permettono di rappresentare in modo ovvio vettori e matrici. Le basi di dati, invece, hanno una struttura che si puo' ricondurre a quella dei record. Nel caso dei record, ad ogni campo, è associato un nome e allo stesso modo, ad ogni dominio possiamo associare un nome detto *attributo* che descrive il ruolo svolto dal dominio. Per esempio, nella relazione precedente delle squadre, possiamo usare nomi come "SquadraCasa, SquadraOspite, RetiCasa, RetiOspite".
Modificando la definizione di relazione con l'introduzione degli attributi e quindi l'ordinamento degli attributi risulta irrilevante, non è piu' necessario parlare di primo dominio, secondo dominio ecc.
Per formalizzare i concetti, indichiamo con $\mathcal{D}$ l'insieme dei domini e specifichiamo la corrispondenza fra attributi e domini, nell'ambito di una relazione, per mezzo di una funzione $dom : X \to \mathcal{D}$, che associa a ciascun attributo $A \in X$ un dominio $dom \nabla A \partial \in \mathcal{D}$. Poi diciamo che una *tupla* su un insieme di attributi X è una funzione t che associa a ciascun attributo $A \in X$ un valore del dominio $dom \nabla A \partial$. Quindi una *relazione* su X è un insieme di tuple su X. La differenza nella definzione precedente sta nel fatto che gli elementi delle tuple sono indivuati per mezzo degli attributi e non per posizione.
Se t è una tupla su X e A un attributo nell'insieme X allora $t[A]$ indica il valore di t su A, ad esempio:

$$t[SquadraOspite] = Lazio$$

4.1.4 Relazioni e basi di dati

Una base di dati è costituita da piu' relazioni, le cui tuple, contengono valori comuni per stabilire delle corrispondenze. Ad esempio date le seguenti relazioni:

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	25/11/2000
485745	Neri	Anna	23/04/2001
200768	Verdi	Fabio	12/02/2001
587614	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Corso
276545	28	01
276545	27	04
937653	25	01
200768	24	04

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	25/11/2000
485745	Neri	Anna	23/04/2001
200768	Verdi	Fabio	12/02/2001
587614	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Corso
276545	28	01
276545	27	04
937653	25	01
200768	24	04

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

- la relazione *Studenti* contiene informazioni relative a un insieme di studenti, con numero di matricola, cognome, nome e data di nascita;
- la relazione *Esami* contiene informazioni relative a esami: il numero di matricola dello studente, il voto e il corso; questa relazione fa riferimento ai dati contenute nelle altre due, agli studenti con il numero di matricola e ai corsi con il codice di corso;
- la relazione *corsi* contiene relazioni relative ai corsi con il codice del corso, il titolo e il docente.

Questo esempio mostra una delle caratteristiche fondamentali del modello relazionale, il fatto che esso è basato su valori: i riferimenti fra dati in relazioni sono rappresentato per mezzi di valori che compaiono nelle tuple. Rispetto a un modello basato su record e puntatori, il modello relazionale, basato su valori, presenta diversi vantaggi:

- esso richiede di rappresentare solo cio che è rilevante dal punto di vista dell'applicazione: i puntatori sono qualcosa di aggiuntivo legato ad aspetti realizzativi, nei modelli con puntatori, il programmatore fa riferimento a dati non significativi per l'applicazione;
- la rappresentazione logica dei dati non fa alcun riferimento con quella fisica che puo' anche subire cambiamenti, il modello relazionale permette di ottenere l'indipendenza fisica dei dati;
- dato che l'informazione è contenuta nei valori, è semplice trasferire i dati da un contesto ad un altro; con i puntatori l'operazione è piu' complessa perchè i puntatori hanno un significato locale al singolo sistema.

Anche in una base di dati relazionale, a livello fisico, i dati possono essere rappresentati con modalità che prevedono l'uso dei puntatori. La differenza rispetto

ai modelli basati su puntatore è che qui i puntatori non sono visibili a modello logico.
Possiamo riassumere le definizioni relative al modello relazionale con piu' precisione, distinguendo il livello degli schemi a quello delle istanze.

- Uno *schema di relazione* è costituito da un simbolo R, detto *nome della relazione* e da un insieme di attributi $X = =:A_1, A_2, \dots, A_n=:$;
- Uno *schema di base di dati* è un insieme di schemi di relazione con nomi diversi:
$$\mathbf{R} = =:R_1 \nabla X_1 \partial, R_2 \nabla X_2 \partial, \dots, R_n \nabla X_n \partial =:$$
- Un'*istanza di relazione* o *relazione* su uno schema $R \nabla X \partial$ è un insieme di r di tuple su X. Talvolta si usa la notazione di $r \nabla X \partial$;
- Un'*istanza di base di dati* o semplicemente *base di dati* è un insieme di relazioni $\mathbf{r} = =:r_1, r_2, \dots, r_n=:$ dove ogni r_i per $1 \leq i \leq n$ è uno schema di relazione $R_i \nabla X_i \partial$

Il modello relazionale permette di rappresentare informazioni strutturate in modo articolato, nella figura sottostante, sono schematizzate tre ricevute emesse da un ristorante. Esse hanno una struttura che prevede alcune informazioni fisse come numero, data e totale e un numero di righe variabile ognuna relativa a un insieme di portate omogenee. Possiamo rappresentare queste informazioni per mezzo di due relazioni, la relazione Ricevute con numero, data e totale e la relazione Dettaglio con quantità, descrizione e importo, associate alla ricevuta stessa tramite il numero.

"DA MARIO"		
Ricevuta n. 1357		
del 5/5/2021		
3	coperti	6,00
2	antipasti	12,00
3	primi	27,00
2	bistecche	36,00
Totale		**81,00**

"DA MARIO"		
Ricevuta n. 2334		
del 4/7/2021		
2	coperti	4,00
1	antipasti	6,00
2	primi	15,00
2	orate	50,00
2	caffè	3,00
Totale		**78,00**

“DA MARIO”		
Ricevuta n. 3002		
del 4/8/2021		
3	coperti	6,00
2	antipasti	14,00
3	primi	20,00
1	orate	25,00
1	caprese	8,00
2	caffè	3,00
Totale		**76,00**

La base di dati che rappresenta le ricevute soprastanti è la seguente:

Ricevute

Num	Data	Totale
1357	5/5/2021	81,00
2334	4/7/2021	78,00
3002	4/8/2021	76,00

Dettaglio

Num	Q.tà	Descr	Costo
1357	3	coperti	6,00
1357	2	antipasti	12,00
1357	3	primi	27,00
1357	2	bistecche	36,00
2334	2	coperti	4,00
2334	1	antipasti	6,00
2334	2	primi	15,00
2334	2	orate	50,00
2334	2	caffè	3,00
3002	3	coperti	6,00
3002	2	antipasti	14,00
3002	3	primi	20,00
3002	1	orate	25,00
3002	1	caprese	8,00
3002	2	caffè	3,00

In questa situazione, la base di dati descritta non tiene traccia della riga dei singoli ordini come accade nelle ricevute, se vogliamo tener traccia dell'impaginazione della ricevuta basta aggiungere un attributo nella relazione Dettaglio che indica la posizione all'interno della ricevuta, come segue:

Dettaglio

Num	Riga	Q.tà	Descr	Costo
1357	1	3	coperti	6,00
1357	2	2	antipasti	12,00
1357	3	3	primi	27,00
1357	4	2	bistecche	36,00
2334	1	2	coperti	4,00
2334	2	1	antipasti	6,00
2334	3	2	primi	15,00
2334	4	2	orate	50,00
2334	5	2	caffè	3,00
3002	1	3	coperti	6,00
3002	2	2	antipasti	14,00
3002	3	3	primi	20,00
3002	4	1	orate	25,00
3002	5	1	caprese	8,00
3002	6	2	caffè	3,00

4.1.5 Informazione incompleta e valori nulli

La struttura del modello relazione è molto semplice e potente. Essa pero' impone un certo grado di rigidità, in quanto le informazioni devono essere rappresentate per mezzo di tuple di dati omogenee, in ogni relazione possiamo rappresentare solo tuple corrispondenti allo schema di relazione stessa. In molti casi, i dati disponibili possono non corrispondere al formato previsto, per esempio, data la relazione:

$$Persone\nabla Cognome, Nome, Indirizzo, Telefono\partial$$

il valore dell'attributo Telefono potrebbe non essere disponibile per tutte le tuple. Quindi, non sarebbe opportuno utilizzare un valore del dominio per rappresentare l'assenza di informazione. In questo caso, supponendo che i numeri telefonici sono rappresentati per mezzo di numeri interi, potremmo utilizzare lo zero per

indicare l'assenza. In generale, questa scelta non risulta soddisfacente. In primo luogo perchè, richiede l'esistenza di un valore nel dominio mai utilizzato per valori significativi, in secondo luogo l'uso di valori del dominio puo' generare confusione: la distinzione fra valori veri e fittizzi è nascosta e quindi, i programmatori che accedono alla base di dati devono tenerne conto distinguendo opportunamente.
Per rapprsentare in modo semplice e comodo l'assenza di informazione, il concetto di relazione viene esteso prevedendo che una tupla possa assumere, su ciascun attributo, o un valore del dominio, o un valore speciale, detto *valore nullo*. Nelle rappresentazioni tabellari il valore nullo viene indicato con il simbolo *null*.
Nei sistemi di basi di dati relazionali è prevista una gestione molto semplice del valore nullo, sul quale non viene fatta alcuna ipotesi e quindi rappresenta il valore senza informazione.

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	null
null	Neri	Anna	23/04/2001
null	Verdi	Fabio	12/02/2001

Esami

Studente	Voto	Corso
276545	28	01
null	27	null
200768	24	null

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	null
null	Chimica	Belli

Il valore nullo sulla data di nascita è ammissibile perchè si puo' pensare che l'informazione non sia rilevante, ma un valore nullo sul numero di matricola o sul codice del corso genera problemi in quanto quei valori sono utilizzati per stabilire correlazioni fra tuple di relazioni diverse. Inoltre, la presenza di valori nulli rende inutilizzabili le informazioni. Infine, la presenza di molteplici valori nulli in una relazione puo' generare dubbi sulla significatività e identità delle tuple. è necessario quindi moderare oppurtunamente la presenza dei valori nulli nelle nostre relazioni, in genere, quando si definisce una relazione è possibile specifi-

care quali attributi possono assumere valori nulli o meno.

4.2 Vincoli di integrità

In molti casi, non è vero che l'insieme di tuple sullo schema rappresenti informazioni corrette per l'applicazione, dopo aver descritto i problemi relativi alla presenza di valori nulli, analizziamo i problemi relativi alla presenza di valori non nulli. Analizziamo la relazione in figura:

Studenti

Matricola	Cognome	Nome	Data di nascita
200768	Verdi	Fabio	12/02/2001
937653	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Lode	Corso
200768	36		05
937653	28	lode	01
937653	30	lode	04
276545	25		01

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

- Nella prima tupla della relazione *Esami* abbiamo un voto pari a 36 che, nel sistema italiano, non è ammissibile;

- Nella seconda tupla della relazione *Esami* viene indicato che è stata attribuita la lode ad un voto di 28;
- Le ultime due tuple della relazione *Studenti* troviamo lo stesso numero di matricola che è impossibile in quanto univoco;
- La quarta tupla della relazione *Esami* presente, per l'attributo **Studente**, un numero di matricola che non compare nella relazione *Studenti*, analogamente compare un codice di corso non presente nella relazione *Corsi*.

In una base di dati, bisogna evitare situazioni di questo tipo, quindi è stato introdotto il concetto di *vincolo di integrità*, come proprietà che deve essere soddisfatta dalle istanze che rappresentano informazioni corrette per l'applicazione. Ogni vincolo puo' essere visto come un predicato che associa a ogni istanza il valore *vero* o *falso*. In uno schema a base di dati associamo un insieme di predicati e consideriamo corrette le istanze che soddisfano tutti i vincoli. In ciascuno dei casi discussi sopra possiamo introdurre un vincoli che vieti la situazione indesiderata.
Distinguiamo due categorie di vincoli a seconda degli elementi che sono coinvolti in una base di dati.

- Un vincolo è *intrarelazionale* se è il soddisfacimento è definito rispetto a singole relazioni della base di dati; i primi tre casi corrispondono a vincoli intrarelazionali. Talvolta, il coinvolgimento riguarda le tuple separatamente le une dalle altre:
  - un *vincolo di tupla* è un vincolo che puo' essere valutato su ciascuna tupla indipendentemente dalle altre; i primi due casi rientrano in questa categoria.
  - un *vincolo su valori* o *vincolo di dominio* impone una restrizione sul dominio; il primo caso rientra in questa categoria.
- Un vincolo è *interrelazionale* se coinvolge piu' relazioni; questo è il caso del quarto esempio, in cui la situazione indesiderata puo' essere vietata richiedendo che un numero di matricola compaia nella relazione *Esami* solo se presente nella relazione *Studenti*.

4.2.1 Vincoli di tupla

I vincoli di tupla esprimono condizioni sui valori di ciascuna tupla indipendentemente dalle altre tuple.
Una possibile sintassi per esprimere condizioni di questo tipo è quella che permette di definire espressioni booleane, con connettivi *and, or* e *not* con atomi che confrontano, con operatori di uguaglianza, disuguaglianza e ordinamento, valori di attributo o espressioni aritmetiche su valori di attributo. Ad esempio i vincoli violati negli esempi precedenti possono essere descritti con le espressioni:

$$\nabla Voto \geq 18\partial \; and \; \nabla Voto \leq 30\partial$$

$$\nabla not \nabla Lode \; = \; 'lod :: \partial\partial \; or \; \nabla Voto \; = \; 30\partial$$

Il secondo indica che è ammissibile la lode solo se il voto è pari a 30.

4.2.2 Chiavi

Analizziamo i vincoli di chiave che sono i piu' importanti del modello relazionale. Negli esempi precedenti il numero di matricola identifica univocamente uno studente, ma ci sono casi in cui anche un insieme di attributi identifica univocamente una tupla.
Quindi, intuitivamente, una chiave è un insieme di attributi utilizzato per identificare univocamente le tuple di una relazione. Per formalizzare la definizione, procediamo in due passi:

1. un insieme di K attributi è *superchiave* di una relazione r se r non contiene due tuple distinte t_1 e t_2 con $t_1[K] = t_2[K]$

2. K è *chiave* di r se è un superchiave minimale di r

Ad esempio, consideriamo la seguente relazione:

Matricola	Cognome	Nome	Nascita	Corso
4328	Rossi	Luigi	29/04/2000	Ing. Informatica
6328	Rossi	Dario	29/04/2000	Ing. Informatica
4766	Rossi	Luca	01/05/2001	Ing. Civile
4856	Neri	Luca	01/05/2001	Ing. Meccanica
5536	Neri	Luca	05/03/1999	Ing. Meccanica

- l'insieme $=:Matricola=$ è superchiave ed è anche una superchiave minimale in quanto contiene un solo attributo;

- l'insiene $=:Cognome, Nome, Nascita=$ è superchiave e nessuno dei suoi sottoinsiemi è superchiave; infatti esistono due tuple uguali su Cognome e Nascita, due uguali su Cognome e Nome e due uguali su Nome e Nascita;
- l'insieme $=:Matricola, Corso=$ èsuperchiave, ma non è superchiave minimale perchè esiste un suo sottoinsieme proprio $=:Matricola=$, che è superchiave minimale, e quindi $=:Matricola, Corso=$ non è una chiave;
- l'insieme $=:Nome, Corso=$ non è superchiave perchè nella relazione compaiono due tuple uguali su Nome e Corso.

Possiamo notare come ciascuna relazione e ciascuno schema di relazione abbiano sempre una chiave. Una relazione è un insieme e quindi, è costitua da elementi diversi tra loro; di conseguenza per ogni relazione $r\nabla X\partial$, l'insieme X di attributi è una superchiave per la relazione. Ci possono essere due possbili casi: o l'insieme è anche chiave, oppure non è chiave, perchè esiste un'altra superchiave in esso contenuta; allora si puo' ripetere lo stesso ragionamento per questo sottoinsieme e cosi via.
Il fatto che su ogni schema di relazione possa essere definita almeno una chiave garantisce l'accessibilità a tutti i valori di una base di dati e la loro univoca identificabilità. Inoltre, permette di stabilire efficacemente quelle corrispondenze fra dati contenuti in relazioni diverse che caratterizzano il modello relazionale come modello basato su valori. I valori attraverso cui vengono stabilite le corrispondenze fra tuple di relazioni diverse sono valori delle chiavi delle relazioni cui si fa riferimento dall'esterno.

4.2.3 Chiavi e valori nulli

Consideriamo la relazione in figura:

Matricola	Cognome	Nome	Nascita	Corso
null	Rossi	Mario	null	Ing. Informatica
4766	Rossi	Luca	01/05/2001	Ing. Civile
4856	Neri	Luca	null	null
null	Neri	Luca	05/03/1999	Ing. Civile

Esaminando questa relazione, con due chiavi Matricola e (Cognome, Nome, Nascita), notiamo problemi di due tipi: la prima tupla ha valori nulli su Matricola e Nascita e quindi questa tupla non è identificabile in alcun modo. Inoltre, non

è possibile in altre relazioni, fare riferimento a questa tupla, visto che andrebbe fatto con il valore di una chiave. Anche le ultime due tuple presentano un problema: nonostante ciascuna delle due tuple abbia una chiave completamente specificata, la presenza di valori nulli rende impossibile capire se le due tuple facciano riferimento allo stesso studente.
Questo esempio suggerisce la necessità di porre dei limiti alla presenza di valori nulli nelle chiavi delle relazioni. In pratica, si adotta una soluzione semplice, che permette di garantire l'identificazione univoca di tutte le tuple e la possibilità di far riferimento a esse: su una delle chiavi, detta *chiave primaria*, si vieta la possibilità di avere valori nulli, sulle altre i valori nulli sono ammessi. Gli attributi che costituiscono la chiave primaria vengono evidenziati attraverso la sottolineatura. La maggior parte dei riferimenti tra relazioni avviene attraverso la chiave primaria.
In quasi tutti i casi reali è possibile trovare attributi i cui valori sono identificanti e sempre disponibili. Se questo non accade è necessario introdurre un attributo aggiuntivo, solitamente un codice, che viene generato e assegnato ad ogni tupla all'atto dell'inserimento.

4.2.4 Vincoli di integrità referenziale

Per discutere la piu' importante classe di vincoli interrelazionali, consideriamo la base di dati in figura:

Infrazioni

<u>Codice</u>	Data	Agente	Articolo	Stato	Numero
143256	25/10/2021	567	44	I	AB 234 ZK
987554	26/10/2021	456	34	I	AB 234 ZK
987557	26/10/2021	456	34	I	CB 123 AA
630876	15/10/2021	456	53	F	CB 123 AA
539856	12/10/2021	567	44	F	CB 123 AA

Agenti

<u>Matricola</u>	CF	Cognome	Nome
567	RSSM ...	Rossi	Mario
456	NREL ...	Neri	Luigi
638	NREP ...	Neri	Piero

Auto

Stato	Numero	Proprietario	Indirizzo
I	CB 123 AA	Verdi Piero	Via Tigli ...
I	DE 834 ZZ	Verdi Piero	Via Tigli ...
I	AB 234 ZK	Bini Luca	Via Aceri ...
F	CB 123 AA	Beau Marcel	Rue Louis XIV ...

La prima relazione, contiene informazioni relative a un insieme di infrazioni del codice della strada, la seconda agli agenti di polizia che le hanno rilevate e la terza a un insieme di autoveicoli di Paesi Europei. Le informazioni delle *Infrazioni* sono rese significative attraverso il riferimento alla relazione *Agenti* per mezzo della matricola di essi e alla relazione *Auto* per tramite degli attributi Stato e Numero. I riferimenti sono significativi in quanto i valori nella relazione *Infrazioni* sono uguali a quelli presenti nelle altre due relazioni.
Un *vincolo di integrità referenziale* o *foreign key* fra un insieme di attributi X di una relazione R_1 e un'altra relazione R_2 è soddisfatto se i valori di X di ciascuna tupla dell'istanza di R_1 compaiono come valori della chiave dell'istanza di R_2. Questa definizione richiede particolare attenzione, nei casi in cui la relazione alla quale ci riferiamo è composta da piu' attributi e nel caso in cui vi siano piu' chiavi.
Vedendo prima il caso in cui la chiave di R_2 è unica e composta da un solo attributo B allora il vincolo di integrità referenziale fra l'attributo di A di R_1 e la relazione R_2 è soddisfatto se, per ogni tupla t_1 in R_1 per cui $t_1[A]$ non è nullo, esiste una tupla t_2 in R_2 tale che $t_1[A] = t_2[B]$. Nel caso piu' generale, dobbiamo fare attenzione al fatto che ciascuno degli attributi X deve corrispondere a un preciso attributo della chiave K in R_2. è necessario specificare un ordinamento sia nell'insieme X sia di K. Indicando gli attributi $X = A_1 A_2 \dots A_n$ e $K = B_1 B_2 \dots B_n$ il vincolo è soddisfatto se per ogni tupla t_1 in R_1 senza nulli su X esiste una tupla t_2 in R_2 con $t_1[A_i] = t_2[B_i] \quad \forall i \in [1, p]$.
Una considerazione va fatta riguardo le relazioni con piu' chiavi. In questo caso, è opportuno identificarne uno come chiave primaria e riferimenti vengono fatti attraverso essa. Pero', va notato che non tutti i sistemi di gestione di base di dati permettono di indicare esplicitamente la chiave primaria. In questi casi, il vincolo di integrità referenziale deve indicare esplicitamente gli attributi che compongono la chiave cui si fa riferimento.

Capitolo 5

Algebra relazionale

Come detto precedentemente, i linguaggi per basi di dati si suddividono in due categorie.

- Operazioni sullo schema:
  - **DDL: Data Definition Language**;
- Operazioni sui dati:
  - **DML: Data Manipulation Language**

I linguaggi di interrogazione per basi di dati relazionali, invece, possono essere:

- **dichiarativi** che specificano le proprietà del risultato quindi cosa si vuole ottenere;
- **procedurali** che specificano le modalità di generazione del risultato quindi come ottenere il risultato.

I linguaggi di interrogazione possono essere:

- **Algebra relazione** che è di tipo procedurale;
- **Calcolo relazionale** che è dichiarativo e teorico;
- **SQL (Structed Query Language)** che è parzialmente dichiarativo (reale);
- **QBE (Query By Example)** che è di tipo dichiarativo (reale).

In questo capitolo analizzaremo l'algebra relazione che come abbiamo detto è un linguaggio di tipo procedurale basato su concetti di tipo algebrico. Esso è costituito da un insieme di operatori, definiti su relazioni e che producono ancora relazioni come risultati ed è possibile anche formare espressioni che coinvolgono piu' operatori allo scopo di formulare anche interrogazioni piu' complesse. Gli operatori sono:

- operatori insiemistici tradizionali: *unione, intersezione* e *differenza* che possono essere anche definiti, con qualche differenza, sulle relazioni;
- operatori di *ridenominazione, selezione* e *proiezione* che sono piu' specifici;
- operatore di *join* in varie forme: *join naturale, prodotto cartesiano, semijoin* e *theta-join*;
- operatore di *divisione*, che è ridondante perchè esprimibile per mezzo di altri operatori.

5.1 Algebra relazionale

5.1.1 Unione, intersezione e differenza

Notiamo che le relazioni sono insiemi e quindi ha senso definire su di esse gli operatori insiemistici tradizionali di unione, differenza e intersezione, quest'ultima è esprimibile per mezzo della differenza in quanto $r \cap s = r - \nabla r - s\partial$. Pero', dobbiamo prestare attenzione sul fatto che una relazione non è genericamente un insieme di tuple, ma un insieme di tuple *omogenee*, quindi pur potendo, il linea di principio, definire delle operazioni insiemistiche sulle relazioni, non ha senso, dal punto di vista del modello relazionale, definirli con riferimento a relazioni su attributi diversi. Pertanto, consideriamo ammissibili, nell'algebra relazionale, solo applicazioni degli operatori di unione, differenza e intersezione a coppie di operandi definite sugli stessi attributi. La seguente figura mostra degli esempi di applicazione dei tre operatori, che confermano le classiche definizioni ma ridefinite sul nostro contesto:

- l'*unione* di due relazioni r_1 e r_2 definite sullo stesso insieme di attributi X è indicata con $r_1 \cup r_2$ ed è una relazione su X contenente le tuple che appartengono a r_1 oppure a r_2 o a entrambe;

- l'*intersezione* di $r_1 \nabla X \partial$ e $r_2 \nabla X \partial$ è indicata con $r_1 \cap r_2$ ed è una relazione su X contente le tuple che appartengono sia a r_1 che a r_2;
- la *differnza* di r_1 e r_2 è indicata con $r_1 - r_2$ ed è una relazione su X contenente le tuple che appartengono a r_1 e non appartengono a r_2.

Laureati

Matricola	Cognome	Età
7274	Rossi	37
7432	Neri	39
9824	Verdi	38

Dirigenti

Matricola	Cognome	Età
9297	Neri	56
7432	Neri	39
9824	Verdi	38

Laureati ∪ Dirigenti

Matricola	Cognome	Età
7274	Rossi	37
7432	Neri	39
9824	Verdi	38
9297	Neri	56

Laureati ∩ Dirigenti

Matricola	Cognome	Età
7432	Neri	39
9824	Verdi	38

Laureati − Dirigenti

Matricola	Cognome	Età
7274	Rossi	37

5.1.2 Ridenominazione

La limitazione imposta agli operatori insiemistici, anche se giustificata, risulta molto restrittiva. Consideriamo infatti le relazioni in figura:

Paternità

Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele

Maternità

Madre	Figlio
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

Paternità ∪ Maternità ??

Sarebbe sensato eseguire su di esse una sorta di unione per avere tutte le coppie genitore-figlio ma questo non è possibile, perchè gli attributi sono diversi, in una troviamo l'attributo *Padre* e nell'altra troviamo l'attributo *Madre*.
Per risolvere il problema utilizziamo un operatore che ha lo scopo di adeguare il nome degli attributi in base alle necessità. Questo si chiama operatore di *ridenominazione* che cambia il nome degli attributi lasciando invariato il contenuto delle relazioni. Consideriamo le relazioni in figura:

Paternità

Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Isacco	Giacobbe

$\rho_{\text{Genitore}\leftarrow\text{Padre}}$(Paternità)

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Isacco	Giacobbe

Come possiamo notare, la ridenominazione agisce solo sullo schema, cambiando il nome dell'attributo *Padre* in *Genitore* e si denota con:

$$\rho_{Genitore\leftarrow Padre}$$

Ora definiamo questo operatore in forma generale. Sia r una relazione definita su un insieme di attributi X e sia Y un altro insieme di attributi con la stessa cardinalità. Siano $A_1, A_2, \dots, A_k$ e $B_1, B_2, \dots, B_k$ rispettivamente un ordinamento per gli attributi in X e un ordinamento per quelli in Y. Allora la ridenominazione:

$$\rho_{B_1B_2\dots B_k\leftarrow A_1A_2\dots A_k}\nabla r\partial$$

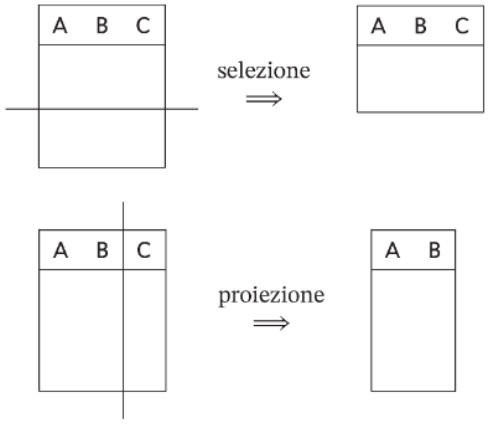
contiene una tupla t' per ciascuna tupla t in r, definita come segue: t' è una tupla su Y e $t'[B_i] = t[A_i] \quad \forall i = 1, \dots, k$. La definizione conferma che cio' che cambia sono i nomi degli attributi, mentre i valori rimangono inalterati e vengono associati ai nuovi attributi. In pratica, nelle due liste $A_1A_2 \dots A_k$ e $B_1B_2 \dots B_k$ indicheremo solo gli attributi che vengono ridenominati, cioè quelli per cui $A_i \neq B_i$.
Quindi ritornando alle relazioni precedenti, l'unione delle due relazioni è la seguente:

$\rho_{\text{Genitore}\leftarrow\text{Padre}}(\text{Paternità}) \cup \rho_{\text{Genitore}\leftarrow\text{Madre}}(\text{Maternità})$

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

5.1.3 Selezione

Esaminiamo, ora, gli operatori più tipici dell'algebra relazionale, che permettono di manipolare le relazioni. Si tratta di tre operatori: selezione, proiezione e join con diverse varianti.
È bene precisare che la selezione e la proiezione svolgono funzioni "complementari", sono entrambe definire su un operando e producono come risultato una porzione dell'operando. Più precisamente la selezione produce un sottinsieme delle tuple, su tutti gli attributi, mentre la proiezione dà un risultato cui contribuiscono tutte le tuple, ma su un sottoinsieme degli attributi. Quindi, possiamo dire che la selezione genera "decomposizioni orizzontali" e la proiezione "decomposizioni verticali". La situazione descritta è mostrata nella seguente figura:



La selezione è denotata dal simbolo σ a pedice del quale viene indicata la "condizione di selezionè' opportune. Il risultato contiene le tuple dell'operando che soddisfano la condizione di selezione. Le condizioni di selezione possono prevedere confronti tra attributi e confronti tra attributi e costanti e possono essere ottenuta combinando condizioni semplici con i connettivi logici: $\vee$ (or), $\wedge$ (and) e $\neg$ (not).
In termini più precisi data una relazione $r\nabla X\partial$, una *formula proposizionale* F su X è una formula ottenuta combinando, con i connetivi $\vee$, $\wedge$ e $\neg$, condizioni atomiche del tipo $A\Theta B$ o $A\Theta c$, dove:

- Θ è un oepratore di confronto $\nabla=, \neq, <, >, \leq, \geq\partial$;
- A e B sono attributi in X sui cui valori il confronto Θ abbia senso;
- c è una costante con il dominio di A.

Seguono due esempi di selezione:

Cittadini

Cognome	Nome	CittàDiNascita	Residenza
Rossi	Mario	Roma	Milano
Neri	Luca	Roma	Roma
Verdi	Nico	Firenze	Firenze
Rossi	Marco	Napoli	Firenze

$\sigma_{\text{CittaDiNascita=Residenza}}(\text{Cittadini})$

Cognome	Nome	CittàDiNascita	Residenza
Neri	Luca	Roma	Roma
Verdi	Nico	Firenze	Firenze

Impiegati

Cognome	Nome	Età	Stipendio
Rossi	Mario	25	2000,00
Neri	Luca	40	3000,00
Verdi	Nico	36	4500,00
Rossi	Marco	40	3900,00

$\sigma_{\text{Eta>30}\wedge\text{Stipendio>4000,00}}(\text{Impiegati})$

Cognome	Nome	Età	Stipendio
Verdi	Nico	36	4500,00

Date una formula F e una tupla t, è definito un avlore di verità per F su t:

- $A\Theta B$ è vera su t se $t[A]$ è in relazione Θ con $t[B]$, altrimenti è falsa;
- $A\Theta c$ è vera su t se $t[A]$ è in relazione Θ con c, altrimenti è falsa;
- $F_1 \vee F_2, F_1 \wedge F_2, \neg F_1$ hanno l'usuale significato.

Possiamo quindi completare la definizione di selezione: $\Theta_F \nabla r \partial$, in cui r è una relazione e F è una formula proposizionale, produce una relazione sugli stessi attributi di r che contiene le tuple di r su cui F è vera.
Seguono le proprietà dell'operatore di selezione:

- *Cardinalità:* $\triangleq \Theta_F \nabla r \partial \triangleq \leq \triangleq r \triangleq$
- *Grado*: grado$\nabla \Theta_F \nabla r \partial \partial$ =grado$\nabla r \partial$
- *Proprietà commutativa*: $\Theta_{F_1} \nabla \Theta_{F_2} \nabla r \partial \partial = \Theta_{F_2} \nabla \Theta_{F_1} \nabla r \partial \partial$
- *Idempotenza*: $\Theta_{F_1} \nabla \Theta_{F_2} \nabla r \partial \partial = \Theta_{F_1 \wedge F_2} \nabla r \partial$

5.1.4 Proiezione

L'opertore di proiezione presenta una definizione ancora più semplice di quella di selezione: dati una relazione $r \nabla X \partial$ e un sottoinsieme Y di X, la *proiezione* di r su Y è l'insieme di tuple su Y ottenute dalle tuple di r considerando solo i valori su Y:

$$\pi_Y \nabla r \partial = =: t[Y] \mid t \in r =$$

Seguono due esempi di proiezione:

Impiegati

Cognome	Nome	Reparto	Capo
Rossi	Mario	Vendite	Gatti
Neri	Luca	Vendite	Gatti
Verdi	Mario	Personale	Lupi
Rossi	Marco	Personale	Lupi

$\pi_{\text{Cognome,Nome}}(\text{Impiegati})$

Cognome	Nome
Rossi	Mario
Neri	Luca
Verdi	Mario
Rossi	Marco

In questo esempio viene illustrato chiaramento il concetto citato.

Impiegati

Cognome	Nome	Reparto	Capo
Rossi	Mario	Vendite	Gatti
Neri	Luca	Vendite	Gatti
Verdi	Mario	Personale	Lupi
Rossi	Marco	Personale	Lupi

$\pi_{\text{Reparto,Capo}}(\text{Impiegati})$

Reparto	Capo
Vendite	Gatti
Personale	Lupi

In questo esempio, invece, troviamo un numero di tuple inferiore a quelle dell'operando, perchè alcune tuple, avendo valori uguali sugli attributi della proiezione, danno lo stesso contributo alla proiezione. Essendo le relazioni definite come insiemi, non possono apparire più tuple uguali fra loro. Esiste, quindi, un legame fra i vincoli di chiave e le proiezioni relativamente a questo problema: $\pi_Y \nabla r \partial$ contiene lo stesso numero di tuple di r se e solo se Y è superchiave per r. Infatti:

- se Y è superchiave, allora r non contiene tuple uguali su Y, quindi ogni tupla da un contributo diverso alla proiezione;
- se la proiezione ha tante tuple quante l'operando, allora ciascuna tupla contribuisce alla proiezione con valori diversi, quindi r non contiene coppie di tuple uguali su Y e questa è la definizione di superchiave.

5.1.5 Join

Esaminiamo l'operatore più caratteristico dell'algebra relazionale, l'operatore di join, che permette di correlare dati contenuti in relazioni diverse, confrontando i valori contenuti in esse e utilizzando la caratteristica fondamentale del modello, cioè quella di essere basato su valori. Esistono due versioni dell'operatore: il *join naturale* utile per riflessioni astratte e il *theta-join* che è più rilevante dal punto di vista pratico.

Join naturale

Il *join naturale* è un operatore che correla dati in relazioni diverse, sulla bae di valori uguali in attributi con lo stesso nome. L'operatore è denotato con il simbolo $\bowtie$. Il risultato del join è costituito da una relazione sull'unione degli insiemi di attributi degli operandi e le sue tuple sono ottenute combinando le tuple degli operandi con valori uguali sugli attributi comuni. Un esempio di join è mostrato nella seguente figura:

r_1

Impiegato	Reparto
Rossi	vendite
Neri	produzione
Bianchi	produzione

r_2

Reparto	Capo
produzione	Mori
vendite	Bruni

$r_1 \bowtie r_2$

Impiegato	Reparto	Capo
Rossi	vendite	Bruni
Neri	produzione	Mori
Bianchi	produzione	Mori

In generale, il join naturale $r_1 \bowtie r_2$ di $r_1 \nabla X_1 \partial$ e $r_2 \nabla X_2 \partial$ è una relazione definita su $X_1 X_2$ come segue:

$$r_1 \bowtie r_2 = =: t \text{ su } X_1X_2 \mid \text{esistono } t_1 \in r_1 \text{ e } t_2 \in r_2 \text{ con } t[X_1] = t_1 \text{ e } t[X_2] = t_2 =:$$

Più sinteticamente, possiamo scrivere:

$$r_1 \bowtie r_2 = =: t \text{ su } X_1X_2 \mid t[X_1] \in r_1 \text{ e } t[X_2] \in r_2 =:$$

La definizione conferma che le tuple del risultato sono ottenute combinando tuple degli operandi con valori uguali su attributi comuni. Il grado della relazione ottenuta come risultato di un join è minore o uguale della somma dei gradi dei due operandi, perchè gli attributi omonimi degli opernadi compaiono una sola volta nel risultato.
Molto frequentemente, si eseguono join sulla base di valori di una chiave delle relazioni coinvolte, esplicitando i riferimenti fra tuple. In molti di questi casi, è anche definito, fra gli attributi coinvolti, un vincolo di integrità referenziale. Consideriamo la relazione in figura:

Infrazioni

Codice	Data	Agente	Articolo	Stato	Numero
143256	25/10/2021	567	44	I	AB 234 ZK
987554	26/10/2021	456	34	I	AB 234 ZK
987557	26/10/2021	456	34	I	CB 123 AA
630876	15/10/2021	456	53	F	CB 123 AA
539856	12/10/2021	567	44	F	CB 123 AA

Auto

Stato	Numero	Proprietario	Indirizzo
I	CB 123 AA	Verdi Piero	Via Tigli ...
I	DE 834 ZZ	Verdi Piero	Via Tigli ...
I	AB 234 ZK	Bini Luca	Via Aceri ...
F	CB 123 AA	Beau Marcel	Rue Louis ...

Infrazioni ⋈ Auto

Codice	Data	Ag	Art	Stato	Numero	Proprietario	Indirizzo
143256	25/10/2021	567	44	I	AB 234 ZK	Bini Luca	Via Aceri ...
987554	26/10/2021	456	34	I	AB 234 ZK	Bini Luca	Via Aceri ...
987557	26/10/2021	456	34	I	CB 123 AA	Verdi Piero	Via Tigli ...
630876	15/10/2021	456	53	F	CB 123 AA	Beau Marcel	Rue Louis ...
539856	12/10/2021	567	44	F	CB 123 AA	Beau Marcel	Rue Louis ...

Notiamo che ciascuna delle tuple di *Infrazioni* è stata combinata con una e una sola delle tuple di *Auto*: una sola perchè **Prov** e **Numero** formano una chiave di

Auto, almeno una perchè è definito il vincolo di integrità referenziale fra **Prov** e **Numero** in *Infrazioni* e *Auto*. Il join, quindi, ha esattamente tante tuple quante la relazione *Infrazioni*.

Join completi e incompleti

Analizziamo diversi esempi di join, in modo da effettuare delle valutazioni sulla dimensione del risultato e al contributo a esso portato dalle tuple degli operandi. Nel primo esempio di join mostrato, ciasuna tupla di ciasuno degli operandi contribuisce ad almeno una tupla del risultato, quindi ci troviamo in un caso di *join completo*: per ogni tupla t_1 di r_1, esiste una tupla t in $r_1 \bowtie r_2$ tale che $t[X_1] = t$, analogamente per r_2. Questa proprietà non è sempre verificata, perchè richeide una corrispondenza fra le tuple delle due relazioni.
Consideriamo la relazione in figura:

r_1

Impiegato	Reparto
Rossi	vendite
Neri	produzione
Bianchi	produzione

r_2

Reparto	Capo
produzione	Mori
acquisti	Bruni

$r_1 \bowtie r_2$

Impiegato	Reparto	Capo
Neri	produzione	Mori
Bianchi	produzione	Mori

Figura 5.1: Un join con tuple *dangling*

La figura mostra un join in cui alcune tuple degli operandi non contribuiscono al risultato, perchè l'altra relazione non contiene tuple con gli stessi valori sull'attributo comune. Queste tuple vengono chiamate *dangling*. Inoltre, è possibile che nessuna tupla degli operandi sia combinabile, e allora il risultato del join è la relazione vuota.

r_1

Impiegato	Reparto
Rossi	vendite
Neri	produzione
Bianchi	produzione

r_2

Reparto	Capo
concorsi	Mori
acquisti	Bruni

$r_1 \bowtie r_2$

Impiegato	Reparto	Capo

Figura 5.2: Un join vuoto

All'estremo opposto, è possibile che ciascuna delle tuple di ciascuno degli operandi sia combinabile con tutte le tuple dell'altro. In tal caso, il risultato contiene un numero di tuple pari al prodotto della cardinalità degli operandi e cioè $\triangleq r_1 \triangleq \times \triangleq r_2 \triangleq$.

r_1

Impiegato	Progetto
Rossi	A
Neri	A
Bianchi	A

r_2

Progetto	Capo
A	Mori
A	Bruni

$r_1 \bowtie r_2$

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	A	Mori
Bianchi	A	Mori
Rossi	A	Bruni
Neri	A	Bruni
Bianchi	A	Bruni

Figura 5.3: Un join con $\triangleq r_1 \triangleq \times \triangleq r_2 \triangleq$ tuple

Quindi il join di r_1 e r_2 contiene un numero di tuple compreso fra 0 e $\triangleq r_1 \triangleq \times \triangleq r_2 \triangleq$. Inoltre:

- se il join di r_1 e r_2 è completo, allora contiene almeno un numero di tuple pari al massimo fra $\triangleq r_1 \triangleq$ e $\triangleq r_2 \triangleq$;

- se $X_1 \cap X_2$ contiene una chiave per r_2, allora il join di $r_1 \nabla X_1 \partial$ e $r_2 \nabla X_2 \partial$ contiene al più $\triangleq r_1 \triangleq$ tuple;
- se $X_1 \cap X_2$ coincide con una chiave per r_2 e sussite il vincolo referenziale fra $X_1 \cap X_2$ in r_1 e la chiave di r_2, allora il join di $r_1 \nabla X_1 \partial$ e $r_2 \nabla X_2 \partial$ contiene esattamente $\triangleq r_1 \triangleq$ tuple.

Join esterni

La caratteristica dell'operatore di join di "tralasciarè' le tuple di una relazione che non hanno controparte nell'altra è utile in molti casi ma pericolosa in altri in quanto potrebbe omettere informazioni utili. Consideriamo la figura 5.1, supponendo di essere interessati a tutti gli impiegati, con l'indicazione del capo, se noto. Per questo, è stata proposta una variante dell'operatore di join chiamta *join esterno* o *outer join*, che prevede che tutte le tuple diano un contributo al risultato, eventualmente estese con valori nulli ove non vi siano controparti opportune. Esistono tre varianti dell'opearatore di join esterno:

1. il join esterno *sinistro*, che estende solo le tuple del primo operando;
2. il join esterno *destro*, che estende solo le tuple del secondo operando;
3. il join esterno *completo*, che le estende tutte.

La seguente figura mostra degli esempi di join esterno.

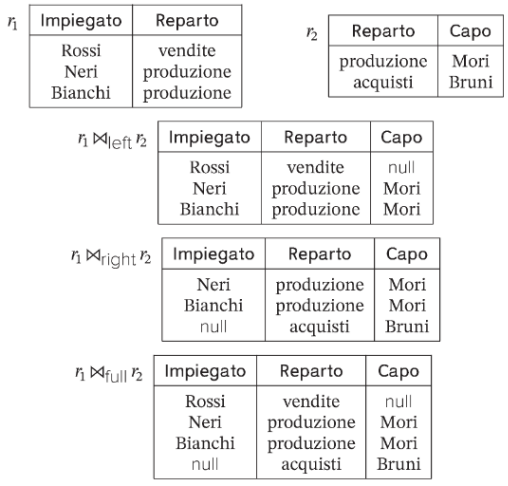


Figura 5.4: Join esterni

Semijoin

Un operatore derivato da quello di join è il *semijoin*, che restituisce le tuple di una relazione che partecipano al join naturale di tale relazione con un'altra. La definizione formale dell'operatore, che viene indicano con il simbolo $\ltimes$, è la seguente:

date due relazioni di $r_1 \nabla X_1 \partial$ e $r_2 \nabla X_2 \partial$ il semijoin $r_1 \ltimes r_2$ è una relazione definita su X_1 come segue:

$$r_1 \ltimes r_2 = =: t \mid t \in r_1 \text{ ed esiste } t_2 \in r_2 \text{ con } t[X_1 \cap X_2] = t_2[X_1 \cap X_2] =:$$

Sulla base della definizione, è facile osservare che l'operatore può essere espresso per mezzo del join e della proiezione:

$$r_1 \ltimes r_2 = \pi_{X_1} \nabla r_1 \bowtie r_2 \partial$$

Un esempio di semijoin è mostrato nella seguente figura:

r_1

Impiegato	Reparto
Rossi	vendite
Neri	produzione
Bianchi	produzione

r_2

Reparto	Capo
produzione	Mori
acquisti	Bruni

$r_1 \ltimes r_2$

Impiegato	Reparto
Neri	produzione
Bianchi	produzione

Figura 5.5: Semijoin

Join n-ario, intersezione e prodotto cartesiano

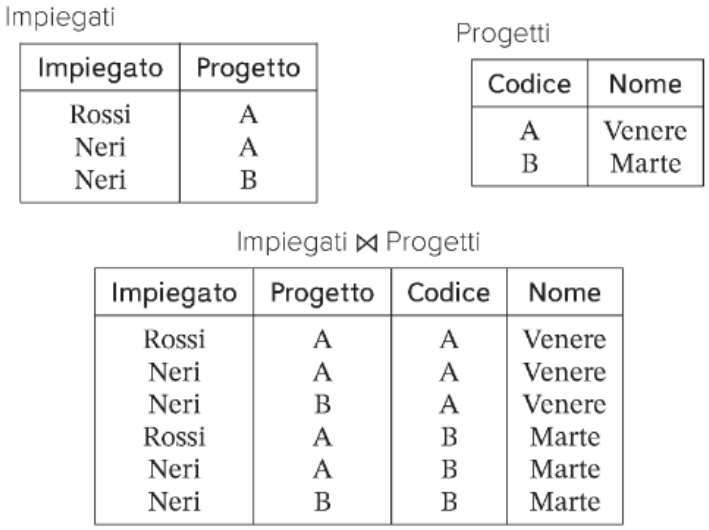
Vediamo alcune proprietà dell'operatore di join naturale. In primo luogo, osserviamo che l'operatore di join è commutativo e associativo. Pertanto, possiamo scrivere, ove necessario, sequenza di join senza parentesi:

$$r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n \text{ oppure } \bowtie_{i=1}^{n} r_i$$

Se gli insiemi degli attributi, X_1 e X_2, sono uguali o disgiunti si applica la definizione generale vistasopra, ma in particolare, se $X_1 = X_2$ allora il join coincide con l'intersezione:

$$r_1 \nabla X_2 \partial \bowtie r_2 \nabla X_2 \partial = r_1 \nabla X_1 \partial \cap r_2 \nabla X_2 \partial \quad \iff X_1 = X_2$$

Il caso in cui gli insiemi degli attributi sono disgiunti merita ancora maggiore attenzione. Il risultato è sempre definito sull'unione X_1X_2, e ciascuna tupla deriva sempre da due tuple, una per ciascuno degli operandi, ma dato che le tuple non hanno attributi comuni, non viene richiesta a esse nessuna condizione per partecipare insieme al join. Quindi, il risultato del join contiene le tuple ottenute comninando, in tutti i modi possibili, le tuple degli operandi. In questo caso allora il join diventa un *prodotto cartesiano*.



Impiegati

Impiegato	Progetto
Rossi	A
Neri	A
Neri	B

Progetti

Codice	Nome
A	Venere
B	Marte

Impiegati ⋈ Progetti

Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	A	Venere
Rossi	A	B	Marte
Neri	A	B	Marte
Neri	B	B	Marte

Figura 5.6: Un prodotto cartesiano

Theta-join ed equi-join

Osservando la figura 5.6 possiamo notare che un prodotto cartesiano di solito ha poca utilità, in quanto concatena tuple non correlate dal punto di vista semantico. Spesso, il prodotto cartesiano viene seguito da una selezioen, che c'entra l'attenzione su tuple correlate secondo le esigenze. Per esempio, sulle relazioni `Impiegati` e `Progetti` ha senso definire un prodotto cartesiano seguito dalla selezione che mantiene solo le tuple con valori uguali sull'attributo **Progetto** di `Impiegati` e su **Codice** di `Progetti`, come mostrato nella seguente figura:

Impiegati

Impiegato	Progetto
Rossi	A
Neri	A
Neri	B

Progetti

Codice	Nome
A	Venere
B	Marte

$\sigma_{\text{Progetto=Codice}}(\text{Impiegati} \bowtie \text{Progetti})$

Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

Figura 5.7: Un prodotto cartesiano seguito da una selezione

Per questo viene definito un operatore derivato, il *theta-join*, come prodotto cartesiano seguito da una selezione, nel modo seguente:

$$r_1 \underset{F}{\bowtie} r_2 = \sigma_F \nabla r_1 \bowtie r_2 \partial$$

ove F è una formula proposizionale. La relazione nella figura 5.7 può essere ottenuta per mezzo del theta-join:

$$\texttt{Impiegati} \bowtie_{\textbf{Progetto=Codice}} \texttt{Progetti}$$

Un theta-join in cui la condizione è una congiunzione di atomi di uguaglianza viene chiamato *equi-join*.
Il theta-join, in particolare l'equi-join, hanno un importante risvolto pratico, in quanto la maggior parte dei sistemi di basi di dati non utilizzano i nomi di attributi per correlare relazioni e quindi non utilizzano il join naturale ma il theta-join e l'equi-join. Il join naturale può essere simulato per mezzo della ridenominazione, dell'equi-join e della proiezione.

Esempio : Simulazione join naturale

Date due relazioni $r_1 \nabla ABC \partial$ e $r_2 \nabla BCD \partial$, il join naturale di r_1 e r_2 può essere espresso per mezzo degli altri operatori, in tre passi:

- ridenominando gli attributi in modo di ottenere relazioni su schemi disgiunti:

$$\rho_{B'C' \leftarrow BC} \nabla r_2 \partial$$

- effettuando l'equi-join, con condizioni di uguaglianza sugli attributi corrispondenti:

$$r_1 \underset{B=B' \wedge C=C'}{\bowtie} \rho_{B'C' \leftarrow BC} \nabla r_2 \partial$$

- concludendo con una proiezione che elimina gli attributi "doppioni", che presentano valori identici a quelli di altri attributi:

$$\pi_{ABCD} \nabla r_1 \underset{B=B' \wedge C=C'}{\bowtie} \rho_{B'C' \leftarrow BC} \nabla r_2 \partial \partial$$

5.1.6 Divisione

L'operatore di divisione, da un certo punto di vista, può essere visto come l'inverso del prodotto cartesiano. Dati due insiemi di attributi disgiunti X_1, X_2, una relazione r su X_1X_2 e una relazione r_2 su X_2, la divisione $r \div r_2$ è una relazione su X_1 che contiene le tuple ottenute come "proiezione" di tuple di r che si combinano con tutte le tuple di r_2 per formare tuple di r:

$$r \div r_2 = =: t_1 \; su \; X_1 \mid \forall t_2 \in r_2 \; \exists t \in r \; con \; t[X_1] = t_1 \; e \; t[X_2] = t_2 =:$$

Un esempip di divisione è mostrato nella seguente figura:

Sedi

Filiale	Ufficio
Roma	Acquisti
Roma	Vendite
Roma	Studi
Milano	Acquisti
Milano	Vendite
Milano	Studi
Napoli	Acquisti
Napoli	Vendite

Uffici

Ufficio
Acquisti
Vendite
Studi

Sedi ÷ Uffici

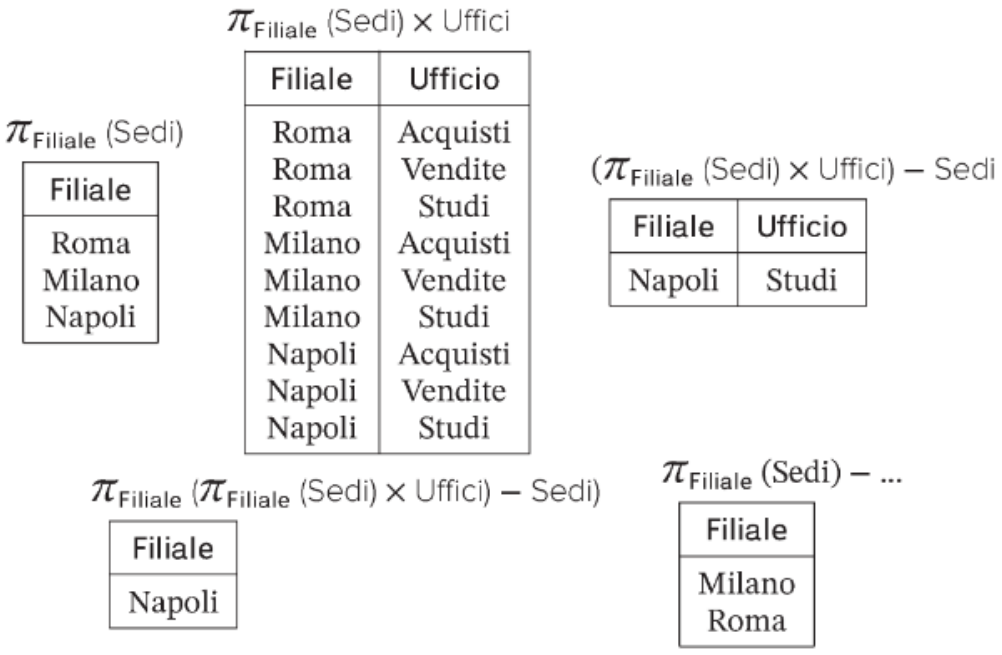
Filiale
Milano
Roma

L'operatore di divisione non è essenziale perchè può essere espresso in termini di altri operatori nel modo seguente:

$$r \div r_2 = \pi_{X_1} \nabla r \partial - \pi_{X_1} \nabla \nabla \pi_{X_1} \nabla r \partial \times r_2 \partial - r \partial$$

$\pi_{X_1} \nabla r \partial \times r_2$ contiene le tuple di $\pi_{X_1} \nabla r \partial$ estese con tutti i possibili valori di r_2: quindi la differenza $\nabla \pi_{X_1} \nabla r \partial \times r_2 \partial - r$ contiene le estensioni di $\pi_{X_1} \nabla r \partial$ che non

compaiono in r e infine, la proiezione $\pi_{X_1}\nabla\nabla\pi_{X_1}\nabla r\partial \times r_2\partial - r\partial$ contiene le tuple di $\pi_{X_1}\nabla r\partial$ per le quali un qualche completamento con r_2 non compare in r. Togliendo a queste $\pi_{X_1}\nabla r\partial$ otteniamo le tuple di $\pi_{X_1}\nabla r\partial$ che si combinano con tutte quelle di r_2.
L'operatore di divisione è il suo rapporto con il prodotto cartesiano, è facile verificare che, se applichiamo la divisione al risultato di un prodotto cartesiano, riotteniamo le relazioni originarie. Specificatamente date due relazioni r_1 e r_2, su insiemi di attributi disgiunti, abbiamo che $\nabla r_1 \times r_2\partial \div r_2 = r_1$.
La seguente figura mostra i risultati della divisione ottenuta con l'operatore di proiezione e differenza:



5.1.7 Interrogazioni in algebra relazionale

Un'interrogazione può essere definita come una funzione che, applicata a istanze di base di dati, produce relazioni. Più precisamente, dato uno schema **R** di base di dati, un'interrogazione è una funzione che, per ogni istanza di **r** di **R**, produce una relazione su un dato insieme di attributi X. Le espressioni dei vari linguaggi di interrogazione, come l'algebra relazionale, rappresentano delle interrogazioni: ogni espressione definisce una funzione. Indichiamo con $E\nabla\mathbf{r}\partial$ il risultato dell'applicazione dell'espressione E alla base di dati **r**.

In algebra relazionale, le interrogazioni su uno schema di basi di dati **R** vengono formulate con espressioni i cui atomi sono relazioni in **R**.
Di seguito, sono mostrate alcune interrogazioni di crescente complessità, che fanno riferimento allo schema contenente le due relazioni:

`Impiegati(`<u>`Matr`</u>`, Nome, Età, Stipendio)`

`Superivisione(Capo,` <u>`Impiegato`</u>`)`

La prima interrogazione chiede di trovare matricola, nome ed età degli impiegati che guadagnano più di 40000 mila euro:

$$\pi_{Matr,Nome,Et\|}\nabla\sigma_{Stipendio\ >\ 40}\nabla\texttt{Impiegati}\partial\partial$$

La seconda interrogazione coinvolge entrambe le relazioni, in modo molto naturale: trovare le matricole dei capi degli impiegati che guadagnano più di 40mila euro:

$$\pi_{Capo}\nabla\texttt{Supervisione}\underset{Impiegato=Matr}{\bowtie}\sigma_{Stipendio\ >\ 40}\nabla\texttt{Impiegati}\partial\partial$$

La terza interrogazione chiede di trovare nome e stipendio dei capi degli impiegati che guadagnano più di 40mila euro. In questo caso, possiamo far uso dell'espressione precedente, ma dobbiamo poi produrre, per ciascuna tupla del risultato, le informazioni richieste sul capo che vanno estratte dalla relazione `Impiegati`. Ogni tupla del risultato è costruita a partire da tre tuple: la prima di `Impiegati`, relativa a un impiegato che guadagna più di 30mila euro, la seconda di `Supervisione`, che indica la matricola del capo dell'impiegato in questione, e la terza di `Impiegati`, con le informazioni relative al capo.

$$\pi_{NomeC,StipC}\left(\begin{array}{l}\rho_{MatrC,NomeC,StipC,EtaC\leftarrow Matr,Nome,Stip,Eta}\nabla\text{Impiegati}\partial\\ \bowtie_{MatrC=Capo}\ \text{Supervisione}\\ \bowtie_{Imp=Matr}\ \sigma_{Stip>40}\nabla\text{Impiegati}\partial\end{array}\right)$$

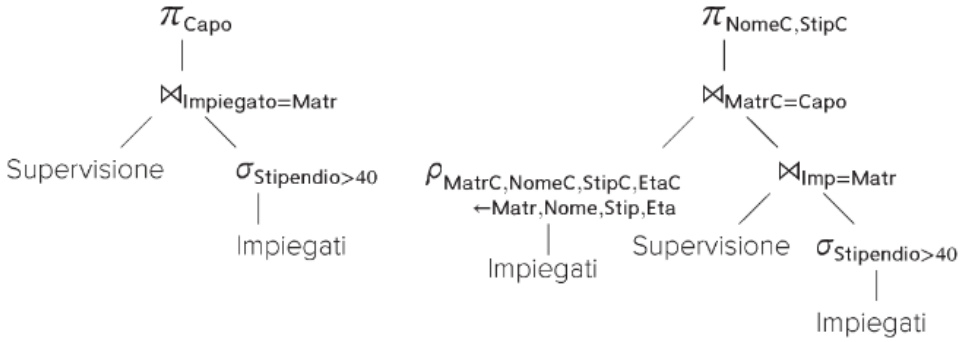
La quarta interrogazione è una variante del precedente, in quanto richiede il confronto di due valori dello stesso attributo, di tuple diverse: trovare gli impiegati che guadagnano più del rispettivo capo, mostrando matricola, nome e stipendio di ciascuno di essi e del capo.

$$\pi_{\text{Matr,Nome,Stip,MatrC,NomeC,StipC}}$$
$$(\sigma_{\text{Stip>StipC}}(\rho_{\text{MatrC,NomeC,StipC,EtaC}\leftarrow\text{Matr,Nome,Stip,Eta}}(\text{Impiegati})$$
$$\bowtie_{\text{MatrC=Capo}}\text{Supervisione}\bowtie_{\text{Imp=Matr}}\ \text{Impiegati}))$$

L'ultimo esempio richiede più attenzione: trovare matricola e nome dei capi i cui impiegati guadagnano tutti più di 40mila euro.

$$\pi_{\text{Matr,Nome}}(\text{Impiegati} \bowtie_{\text{Matr=Capo}} (\pi_{\text{Capo}}(\text{Supervisione}) - \pi_{\text{Capo}}(\text{Supervisione} \bowtie_{\text{Imp=Matr}} \sigma_{\text{Stip}\leq 40}(\text{Impiegati}))))$$

A volte, può essere conveniente utilizzare per l'algebra relazionale, una notazione grafica, in cui un'espressione è rappresentata per mezzo di un albero le cui foglie sono operandi e gli altri nodi corrispondono a degli operatori. Ogni nodo che non sia una foglia ha un numero di figli pari al numero di operandi dell'operatore, quindi i nodi con proiezione, selezioni e ridenominazioni hanno un solo figlio, mentre quelli relativi agli altri operatori ne hanno due. Di seguito viene mostrato un esempio di questa notazione:



5.1.8 Equivalenza di espressioni algebriche

L'algebra relazionale permette di formulare espressioni fra loro *equivalenti*, cioè che producono lo stesso risultato. Ad esempio vale la seguente equivalenza:

$$x \times \nabla y + z\partial \equiv x \times y + x \times z$$

Nell'algebra relazionale possiamo dare una definizione analoga, ma dobbiamo fare attenzione al fatto che l'equivalenza dipende dallo schema, oppure essere assoluta:

- $E_1 \equiv_{\mathbf{R}} E_2 \iff E_1 \nabla \mathbf{r} \partial = E_2 \nabla \mathbf{r} \partial$ per ogni istanza di $\mathbf{r}$ in $\mathbf{R}$.

- $E_1 \equiv E_2 \iff E_1 \equiv_{\mathbf{R}} E_2$ per ogni schema $\mathbf{R}$.

La distinzione è dovuta al fatto che gli schemi degli operandi non vengono esplicitati nelle espressioni e il comportamento può variare a seconda degli attributi nei vari schemi di relazione. Un esempio di equivalenza assoluta è:

$$\pi_{AB}\nabla\sigma_{A>0}\nabla R\partial\partial \equiv \sigma_{A>0}\nabla\pi_{AB}\nabla R\partial\partial$$

mentre la seguente equivalenza:

$$\pi_{AB}\nabla R_1\partial \bowtie \pi AC\nabla R_2\partial \equiv_{\mathbf{R}} \pi_{ABC}\nabla R_1 \bowtie R_2\partial$$

sussite se e solo se nello schema $\mathbf{R}$ l'intersezione fra gli insiemi di attributi R_1 e R_2 è pari ad A. Infatti, se ci fossero anche altri attributi, il join operebbe solo su A nella prima espressione e su A e gli altri attributi nella seconda, con risultati diversi.
L'equivalenza di espressioni dell'algebra risulta molto importante dal punto di vista applicativo, infatti, le interrogazioni specificate in linguaggio SQL vengono tradotte in algebra relazionale e viene valutato il costo in termini di risultati intermedi. In presenza di alternative equivalenti, viene scelta quella con costo minore. Vengono, quindi, utilizzate *trasformazioni di equivalenza* cioè operazioni che sostituiscono un'espressione con un'altra a essa equivalente. In particolare, risultano interessanti le trasformazioni che riducono le dimensioni dei risultati intermedi e quelle che preparano un'espressione all'applicazione di una trasformazione che riduce le dimensioni dei risultati intermedi. Vediamo alcune trasformazioni interessanti.

1. **Atomizzazione delle selezioni**: una selezione congiuntiva può essere sostituita da una cascata di selezioni atomiche:

   $$\sigma_{F_1\wedge F_2}\nabla E\partial \equiv \sigma_{F_1}\nabla\sigma_{F_2}\nabla E\partial\partial$$

   dove E è una qualunque espressione. Questa trasformazione permette di applicare successive trasformazioni che operano su selezioni con condizioni atomiche.

2. **Idempotenza delle proiezioni**: una proiezione può essere trasformata in una cascata di proiezioni che "eliminano" i vari attributi in fasi successive:

   $$\pi_X\nabla E\partial \equiv \pi_X\nabla\pi_{XY}\nabla E\partial\partial$$

   Anche questa è una trasformazione preliminare rispetto ad altre.

3. **Anticipazione della selezione rispetto al join**, detta anche *pushing selections down*, con riferimento al fatto che, nella rappresentazione di espressioni sotto forma di alberi, la selezione viene spostata verso il basso:

$$\sigma_F \nabla E_1 \bowtie E_2 \partial \equiv E_1 \bowtie \sigma_F \nabla E_2 \partial$$

se la condizione F fa riferimento solo ad attributi nella sottoespressione E_2.

4. **Anticipazione della proiezione rispetto al join**, detta anche *pushing projections down*: siano $E_!$ e E_2 definite rispettivamente su X_1 e X_2: se $Y_2 \subseteq X_2$ e $Y_2 \supseteq \nabla X_1 \cap X_2 \partial$, cioè gli attributi $X_2 - Y_2$ non sono coinvolti nel join, allora vale l'equivalenza:

$$\pi_{X_1, Y_2} \nabla E_1 \bowtie E_2 \partial \equiv E_1 \bowtie \pi_{Y_2} \nabla E_2 \partial$$

Combinando questa regola con quella dell'idempotenza, possiamo ottenere la seguente equivalenza:

$$\pi_Y \nabla E_1 \underset{F}{\bowtie} E_2 \partial \equiv \pi_Y \nabla \pi_{Y_1} \nabla E_1 \partial \underset{F}{\bowtie} \pi_{Y_2} \nabla E_2 \partial \partial$$

dove indicando con X_1 ed X_2 gli attributi di E_1 ed E_2 rispettivamente e con J_1 ed J_2 i rispettivi sottoinsiemi coinvolti nella condizione F di join:

$$Y_1 = \nabla X_1 \cap Y \partial \cup J_1$$

$$Y_2 = \nabla X_2 \cap Y \partial \cup J_2$$

In sostanza, possiamo eliminare da ciascuna relazione gli attributi che non compaiono nel risultato finale e non sono coinvolti nel join.

5. **Inglobamento di una selezione in un prodotto cartesiano a formare un join**:

$$\sigma_F \nabla E_1 \bowtie E_2 \partial \equiv E_1 \underset{F}{\bowtie} E_2$$

con $X_1 \cap X_2 = \emptyset$, indicando con X_1 ed X_2 rispettivamente gli attributi di E_1 ed E_2

Data la seguente base di dati:

Impiegati

Matr	Nome	Età	Stipendio
101	Mario Rossi	34	40
103	Mario Bianchi	23	35
104	Luigi Neri	38	61
105	Nico Bini	44	38
210	Marco Celli	49	60
231	Siro Bisi	50	60
252	Nico Bini	44	70
301	Sergio Rossi	34	70
375	Mario Rossi	50	65

Supervisione

Capo	Impiegato
210	101
210	103
210	104
231	105
301	210
301	231
375	252

Figura 5.8: Esempio di una base di dati per le espressioni

Supponiamo di voler trovare i numeri di matricola dei capi di impiegati con meno di trent'anni.

$$\pi_{Capo}\nabla\sigma_{Matr=Imp\wedge Eta<30}\nabla\texttt{Impiegati} \bowtie \texttt{Supervisione}\partial\partial$$

La qualità di quest'espressione è molto bassa, perchè per calcolare pochi valori effettua un prodotto cartesiano. Utilizzando le regole precedenti possiamo migliorarla significativamente. Con la regola 1. spezziamo la selezione:

$$\pi_{Capo}\nabla\sigma_{Matr=Imp}\nabla\sigma_{Eta<30}\nabla\texttt{Impiegati} \bowtie \texttt{Supervisione}\partial\partial\partial$$

Possiamo poi fondere la prima selezione con il prodotto cartesiano e formare un join, la regola 5., e anticipare la seconda selezione rispetto al join, la regola 3.:

$$\pi_{Capo}\nabla\sigma_{Eta<30}\nabla\texttt{Impiegati}\partial \underset{Matr=Imp}{\bowtie} \texttt{Supervisione}\partial$$

Infine, possiamo eliminare dal primo argomenti del join gli attributi non necessari, utilizzando la regola 4.:

$$\pi_{Capo}\nabla\pi_{Matr}\nabla\sigma_{Eta<30}\nabla\texttt{Impiegati}\partial\partial \underset{Matr=Imp}{\bowtie} \texttt{Supervisione}\partial$$

Vediamo ora altre trasformazioni interessanti:

6. **Distributività della selezione rispetto all'unione**:

$$\sigma_F\nabla E_1 \cup E_2\partial \equiv \sigma_F\nabla E_1\partial \cup \sigma_F\nabla E_2\partial$$

7. **Distributività della selezione rispetto alla differenza**:

$$\sigma_F \nabla E_1 - E_2 \partial \equiv \sigma_F \nabla E_1 \partial - \sigma_F \nabla E_2 \partial$$

8. **Distributività della selezione rispetto all'unione**:

$$\pi_F \nabla E_1 \cup E_2 \partial \equiv \pi_F \nabla E_1 \partial \cup \pi_F \nabla E_2 \partial$$

Vale la pena notare che la proiezione non è distributiva rispetto alla differenza, come si può verificare applicando le espressioni:

$$\pi_A \nabla R_1 - R_2 \partial \text{ e } \pi_A \nabla R_1 \partial - \pi_A \nabla R_2 \partial$$

a due relazioni su AB che contengano tuple uguali su A e diverse su B.
Un altro gruppo di trasformazioni contiene quelle che si basano sull'intersezione fra gli operatori insiemistici e le selezioni complesse:

9. $\sigma_{F_1 \vee F_2} \nabla R \partial \equiv \sigma_{F_1} \nabla R \partial \cup \sigma_{F_2} \nabla R \partial$
10. $\sigma_{F_1 \wedge F_2} \nabla R \partial \equiv \sigma_{F_1} \nabla R \partial \cap \sigma_{F_2} \nabla R \partial \equiv \sigma_{F_1} \nabla R \partial \bowtie \sigma_{F_2} R$
11. $\sigma_{F_1 \wedge \neg \nabla F_2 \partial} \nabla R \partial \equiv \sigma_{F_1} \nabla R \partial - \sigma_{F_2} \nabla R \partial$

Abbiamo poi la proprietà commutativa e associativa di tutti gli operatori binari, esclusa la differenza, e la proprietà distributiva del join rispetto all'unione:

12. $E \bowtie \nabla E_1 \cup E_2 \partial \equiv \nabla E \bowtie E_1 \partial \cup \nabla E \bowtie E_2 \partial$

Vale la pena notare che la presenza di risultati intermedi vuoti permette di semplificare le espressioni in modo abbastanza naturale. Notiamo che un join in cui uno degli operandi sia la relazione vuota produce un risultato vuoto, perchè non è possibile concatenare con alcuna tupla le tuple dell'altra relazione.

5.1.9 Algebra con valori nulli

Finora abbiamo supposto che le espressioni dell'algebra venissero applicate a relazioni prive di valori nulli, ma abbiamo studiato l'importanza dei valori nulli nelle applicazioni reali e quindi dobbiamo studiare l'impatto che essi hanno nei linguaggi trattati.
Consideriamo la seguente relazione:

Persone

Nome	Età	Reddito
Aldo	35	15
Andrea	27	21
Maria	null	42

Figura 5.9: Una relazione con valori nulli

Consideriamo la seguente selezione:

$$\sigma_{Eta>30}\nabla\texttt{Persone}\partial$$

Sappiamo che la prima tupla fa parte del risultato e la seconda no. Ma per quanto riguarda il terzo? A proposito di queste interrogazioni è stato proposto di utilizzare una logica a tre valori, in cui un predicato può essere vero oppure falso oppure assumere un terzo, nuovo valore di verità che chiameremo *unknown* e rappresentiamo il simbolo con *U*. Quindi ritornando al nostro esempio, la prima tupla appartiene al risultato quindi è vera, la seconda non appartiene quindi è falsa e la terza forse appartiene e forse no. Le tabelle dei connettivi logici *not, or* e *and* si estendono nel modo seguente:

not	
F	V
U	U
V	F

and	V	U	F
V	V	U	F
U	U	U	F
F	F	F	F

or	V	U	F
V	V	V	V
U	V	U	U
F	V	U	F

Una selezione su relazioni con valori nulli produce come risultato le tuple per cui il risultato predicato risulta vero. Il valore *unknown* rappresenta un valore di verità intermedio tra vero e falso, e il significato dei tre connettivi in questo caso diventa: il *not* è vero solo se il valore di partenza è falso, l'*and* è vero solo se tutti i termini sono veri e l'*or* è vero se almeno uno dei termini è vero.
La logica a tre valori risulta significativa nel caso di espressioni complesse, in cui il risultato presenta proprietà poco soddisfacenti. Per esempio, consideriamo la seguente espressione:

$$\sigma_{Eta>30}\nabla\texttt{Persone}\partial \cup \sigma_{Eta\leq 30}\nabla\texttt{Persone}\partial$$

Questa, a livello logico, dovrebbe restituire la relazione `Persone`, ma se le due sottoepressioni sono valutate seperatamente, la terza tupla ha un'appartenenza sconosciuta a ciascuna sottoespressione e quindi all'unione.
Il modo migliore per superare gli incovenienti appena discussi consiste nel trattare i valori nulli da un punto di vista meramente sintattico, tralasciando cosa essi potrebbero rappresentare. Allo scopo, si introducono due nuove forme di condizioni atomiche di selezione, che valutano se un valore è specificato oppure nullo:

- *A is null* assume valore vero se una tupla t se il valore di t su A è nullo e falso se esso è specificato;
- *A is not null* assume valore vero su una tupla t se il valore di t su A è specificato e falso se il valore è nullo.

Quindi, l'espressione:

$$\sigma_{Eta>30}\nabla\texttt{Persone}\partial$$

restituisce le persone la cui età è nota e maggiore di 30, mentre per ottenere quelle che hanno o potrebbero avere più di trent'anni possiamo utilizzare l'espressione:

$$\sigma_{Eta>30 \vee Eta\ is\ null}\nabla\texttt{Persone}\partial$$

5.1.10 Viste

A volte può risultare mettere a disposizione degli utenti rappresentazioni diverse per gli stessi dati. Nel modello relazionale la tecnica prevista a questo scopo è quella delle *relazioni derivate*, relazioni il cui contenuto è funzione del contenuto di altre relazioni. In una base di dati relazionale, quindi, possono esistere le relazioni *di base*, il cui contenuto è autonomo, e relazioni derivate, il cui contenuto è funzione di quello di altre relazioni. È possibile che una relazione derivata sia funzione di altre relazioni derivate, a condizione che esista un ordinamento fra le relazioni derivate tale che ogni relazione sia definita solo in termini di relazioni di base e di relazioni derivate che la precedono nell'ordinamento.
Possono esistere due tipi di relazioni derivate:

- *viste materializzate*: relazioni effettivamente memorizzate nella base di dati;
- *relazioni virtuali*: esse sono anche chiamate *viste* e sono relazioni definite per mezzo di funzioni, non memorizzate nella base di dati, ma utilizzabili nelle interrogazioni come se le fossero.

Le viste materializzate hanno il vantaggio di essere immediatamente disponibili per le interrogazioni, ma spesso è oneroso mantenere il loro contenuto allineato con quello delle relazioni da cui derivano. Le relazioni virtuali devono essere calcolate per ogni interrogazione ma non presentano problemi di allienamento. Le viste materializzate sono convenienti quando gli aggiormamenti sono rari rispetto alle interrogazioni e il calcolo della vista è complesso. In generale, non esistono tecniche per mantenere l'allineamento e per questo, i sistemi atttuali forniscono solo meccanismi per la gestione di relazioni virtuali che chiameremo *viste*.
Le viste vengono definite nei sistemi relazioni per mezzo di espressioni del linguaggio di interrogazione. Eventuali interrogazioni che si riferiscono alle viste vengonor risolte sostituendo alla vista la sua definizione. Di solito, i sistemi relazionali stabiliscono la strategia di esecuzione delle interrogazioni dopo aver sostituito alla vista la sua definizione. Per esempio, supponiamo di avere una base di dati sulle seguenti relazioni:

$$R_1 \nabla ABC \partial \quad R_2 \nabla DEF \partial \quad R_3 \nabla GH \partial$$

con una vista definita per mezzo di un prodotto cartesiano seguito da una selezione:

$$R = \sigma_{A>D} \nabla R_1 \bowtie R_2 \partial$$

Su questo schema, l'interrogazione:

$$\sigma_{B=G} \nabla R \bowtie R_3 \partial$$

viene eseguita sostituendo a R la sua definizione:

$$\sigma_{B=G} \nabla \sigma_{A>D} \nabla R_1 \bowtie R_2 \partial \bowtie R_3 \partial$$

L'uso delle viste può essere vantaggioso per diversi motivi:

- Un utente interessato solo a una porzione della base di dati può evitare di considerare le componenti irrilevanti.
- Espressioni molto complesse possono essere definite tramite viste, con vantaggi rilevanti soprattutto se ci sono sottoespressioni ripetute.
- Attraverso la definzione di autorizzazioni di accesso rispetto alla viste, è possibile introdurre meccanismi di protezione della privatezza.

- In occasione di ristrutturazione di una base di dati, può risultare conveniente definire viste che corrispondano a relazioni sostituite da altre e perciò non più presenti dopo la ristrutturazione, ma ricavabili dalle nuove relazioni. Ad esempio, se uno schema $R\nabla ABC\partial$ viene sostituito dagli schemi $R_1\nabla AB\partial$ e $R_2\nabla BC\partial$ è possibile definire una vista $R = R_1 \bowtie R_2$ e mantenere inalterate le applicazioni che fanno riferimento a R.

Per quanto riguarda le interrogazioni le viste possono essere trattate come le relazioni di base, ma non si può dire lo stesso per le operazioni di aggiormento. Infatti, in molti casi non è possibile definire una semantica degli aggiornamenti sulle viste: dato un aggiormento su una vista, in generale non esiste uno e un solo aggiornamento delle relazioni di base che porti a un'istanza della base di dati cui corrisponda un'istanza della vista che sia il risultato effettivo dell'aggiornamento specificato sulla vista. Per questo, molti sistemi pongono forti limitazioni riguardo alla possibilità di specificare aggiornamenti sulle viste.

5.2 Calcolo relazionale

Il *calcolo relazionale* fa riferimento a una famiglia di linguaggi di interrogazione, basati sul calcolo dei predicati del primo ordine, che hanno la caratteristica di essere *dichiarativi*, cioè di specificare le proprietà del risultato delle interrogazioni, anziché la procedura seguita per generarlo. In contrasto, l'algebra relazionale è un linguaggio *procedurale*, in quanto le sue espressioni specificano passo passo la costruzione del risultato.
Esistono diverse versioni del calcolo relazionale:

- il **calcolo relazionale sui domini** che presenta in modo naturale tutte le caratteristiche originale di questi linguaggi;
- il **calcolo su tuple con dichiarazioni di range**, che costituisce la base per molti dei costrutti disponibili per le interrogazioni nel linguaggio SQL.

Rispetto alla definzione dei calcoli del predicato del primo ordine, nel calcolo relazionale ci sono alcune semplificazioni. Mentre nel calcolo dei predicati abbiamo simboli di predicato e di funzione, nel calcolo relazionale i simboli di predicato corrispondono alle relazioni nelle base di dati e non compaiono simboli di funzione perchè non sono necessari.
Inoltre, nel calcolo dei predicati interessano sia formule aperte, cioè con variabili libere, sia formule chiuse, con tutte variabili legate. Nel calcolo relazionale

interessano le formule aperte: un'interrogazione è definita per mezzo di una formula del calcolo e il risultato è costituito dalle tuple di valori che rendono vera la formula stessa.

5.2.1 Calcolo relazione su domini

Le espressioni del calcolo relazionale su domini hanno la forma:

$$=:A_1 \ : \ x_1, \ldots, A_k \ : \ x_k \mid f\mp$$

ove:

- $A_1, \ldots, A_k$ sono attributi distinti, che possono anche non comparire nello schema della base di dati rispetto a cui viene formulata l'interrogazione;
- $x_1, \ldots, x_k$ sono *variabili* che per comodità le consideriamo distinte ma non necessariamente sono distinte;
- f è una formula, secondo le seguenti regole:
  - Vi sono formule *atomiche* di due tipi:
    - $R\nabla A_1 \ : \ x_1, \ldots, A_p \ : \ x_p\partial$, dove $R\nabla A_1 \ \ldots \ A_p\partial$ è uno schema di relazione e $x_1, \ldots, x_p$ sono varaibili;
    - $x\theta y$ o $x\theta c$ con x e y variabili, c costante e θ operatore di confronto $\nabla=, \neq, \leq, \geq, <, >\partial$
  - Se f_1 e f_2 sono formule, allora $f_1 \vee f_2$, $f_1 \wedge f_2$ e $\neg f_1$ sono formule;
  - Se f è una formula e x una variabile allora $\exists x \nabla f \partial$ e $\forall x \nabla f \partial$ sono formule.

La lista di coppie $A_1 \ : \ x_1, \ldots A_k \ : \ x_k$ viene chiamata *target list* in quanto definisce la struttura del risultato, che è costituito dalla relazione su $A_1, \ldots, A_k$ che contiene le tuple i cui valori, sostituiti a $x_1, \ldots, x_k$, rendono vera la formula rispetto a un'istanza di base di dati a cui l'espressione viene applicata.
Definiamo il concetto di *valore di verità* informalmente seguento la struttura sintattica delle regole, ove per valore intendiamo "elemento del dominio":

- una formula atomica $R\nabla A_1 \ : \ x_1, \ldots, A_p \ : \ x_p\partial$ è vera sui valori di $x_1, \ldots, x_p$ che formano una tupla della relazione r sullo schema R nell'istanza di base di dati a cui l'espressione viene applicata;

- una formula atomica $x\theta y$ è vera sui valori a_1 e a_2 se il confronto $a_1\theta a_2$ è soddisfatto, analogamente per $x\theta c$;
- per congiunzione, disgiunzione e negazione valgono le usuali definizioni;
- per le formule con i quantificatori:
  - $\exists x \nabla f \partial$ è vera se esiste almeno un valore a che sostituito alla variabile x rende vera f;
  - $\forall x \nabla f \partial$ è vera se, per ogni possibile valore a per la variabile x, la formula f risulta vera.

Con riferimento allo schema di base di dati sulle relazioni:

$$\texttt{Impiegati}\nabla\underline{\text{Matr}}, \text{Nome, Età, Stipendio}\partial$$

$$\texttt{Supervisione}\nabla\text{Capo, }\underline{\text{Impiegato}}\partial$$

La prima interrogazione chiede di trovare matricola, nome, età e stipendio degli impiegati che guadagnano più di 40 mila euro:

$$\sigma_{Stipendio>40}\nabla\texttt{Impiegati}\partial$$

Nel calcolo relazionale questa interrogazione si formula con la seguente espressione:

$$=\!\!:\!\!\mathbf{Matr}\colon m, \mathbf{Nome:}\ n, \mathbf{Età:}\ e, \mathbf{Stipendio:}\ s \mid$$
$$\texttt{Impiegati}\nabla\mathbf{Matr}\ :\ m, \mathbf{Nome:}\ n, \mathbf{Età:}\ e, \mathbf{Stipendio:}\ s\partial \wedge s > 40\!\!=$$

Notiamo la presenza di due condizioni nella formula:

- la prima, $\texttt{Impiegati}\nabla\mathbf{Matr}\ :\ m, \mathbf{Nome:}\ n, \mathbf{Età:}\ e, \mathbf{Stipendio:}\ s\partial$, richiede che i valori rispettivamente sostituiti alle variabili m, n, e, s costituiscano una tupla della relazione `Impiegati`;
- la seconda richiede che il valore della variabile s sia maggiore di 40;

Un'altra interrogazione, appena più complessa richiede di trovare matricola, nome ed età degli impiegati che guadagnano più di 40 mila euro, che in algebra relazione si esprime con:

$$\pi_{Matr,Nome,Eta}\nabla\sigma_{Stipendio>40}\nabla\texttt{Impiegati}\partial\partial$$

Nel calcolo relazionale questa può essere formulata in vari modi. Il più diretto, ma non il più semplice, è basato sul fatto che ciò che ci interessa sono i valori di matricola, nome ed età che partecipano a tuple per le quali lo stipendio è maggiore di 40. Possiamo usare un quantificatore esistenziale:

$$=:\textbf{Matr:}\ m, \textbf{Nome:}\ n, \textbf{Età:}\ e \mid$$
$$\exists s \nabla \texttt{Impiegati} \nabla \textbf{Matr}\ :\ m, \textbf{Nome:}\ n, \textbf{Età:}\ e, \textbf{Stipendio:}\ s \partial \wedge s > 40 \partial =:$$

L'uso del quantificatore, in realtà, non è necessario, perchè scrivendo semplicemente:

$$=:\textbf{Matr}:\ m, \textbf{Nome:}\ n, \textbf{Età:}\ e \mid$$
$$\texttt{Impiegati} \nabla \textbf{Matr}\ :\ m, \textbf{Nome:}\ n, \textbf{Età:}\ e, \textbf{Stipendio:}\ s \partial \wedge s > 40 =:$$

otteniamo lo stesso risultato.
La stessa struttura si estende a interrogazioni più complesse. Ad esempio, l'interrogazione che vuole trovare le matricole dei capi degli impiegati che guadagnano più di 40mila euro, formulata in algebra relazionale come:

$$\pi_{Capo} \nabla \texttt{Supervisione} \bowtie_{Impiegato=Matr} \sigma_{Stipendio>40} \nabla \texttt{Impiegati} \partial \partial$$

può essere formulata nel calcolo relazionale con:

$$=:\textbf{Capo}:\ c \mid \texttt{Impiegati} \nabla \textbf{Matr}\ :\ m, \textbf{Nome:}\ n, \textbf{Età:} e, \textbf{Stipendio:}\ s \partial \wedge$$
$$\texttt{Supervisione} \nabla \textbf{Impiegato}\ :\ m, \textbf{Capo:}\ c \partial \wedge s > 40 =:$$

dove la variabile m, comune alle due condizioni atomiche, realizza la stessa correlazione fra tuple specificata nel join.
L'interrogazione che vuole trovare nome e stipendio dei capi degli impiegati che guadagnano più di 40 mila euro, scritta in algebra relazionale come:

$$\pi_{NomeC,StipC} \nabla \rho_{MatrC,NomeC,StipC,EtaC \leftarrow Matr,Nome,Stip,Eta} \nabla \texttt{Impiegati} \partial$$

$$\bowtie_{MatrC=Capo}$$

$$\texttt{Supervisione} \bowtie_{Impiegato=Matr} \sigma_{Stipendio>40} \nabla \texttt{Impiegati} \partial \partial$$

viene formulata nel calcolo, per ciascuna tupla del risultato, l'esistenza di tre tuple: una relativa a un impiegato che guadagna più di 40 mila euro, una seconda che indica chi è il suo capo e l'ultima che fornisce le informazioni di dettaglio sul capo:

$$=:\textbf{NomeC:}\ nc, \textbf{StipC:}\ sc \mid$$

$$\texttt{Impiegati}\nabla\textbf{Matr: } m, \textbf{Nome: } n, \textbf{Età: } e, \textbf{Stipendio: } s\partial \wedge s > 40$$

$$\texttt{Supervisione}\nabla\textbf{Impiegato: } m, \textbf{Capo: } c\partial\wedge$$

$$\texttt{Impiegati}\nabla\textbf{Matr: } c, \textbf{Nome: } nc, \textbf{Età: } ec, \textbf{Stipendio: } sc\partial=$$

L'ultimo esempio richiede una soluzione più complessa. Dobbiamo trovare matricola e nome dei capi i cui impiegati guadagnano tutti più di 40 mila euro. In algebra abbiamo utilizzato una differenza:

$$\pi_{Matr,Nome}\nabla\texttt{Impiegati} \bowtie_{Matr=Capo}$$

$$\nabla\pi_{Capo}\nabla\texttt{Supervisione}\partial-$$

$$\pi_{Capo}\nabla\texttt{Supervisione} \bowtie_{Imp=Matr} \sigma_{Stip\leq 40}\nabla\texttt{Impiegati}\partial\partial\partial$$

Nel calcolo dobbiamo utilizzare un quantificatore. Seguendo la stessa logica dell'espressione in algebra relazionale possiamo utilizzare un quantificatore esistenziale negato, ognuno per ogni variabile coinvolta, trovando i capi per quali non esiste un impiegato che guadagna non più di 40 mila euro:

$$\{\textsf{Matr} : c,\ \textsf{Nome} : n \mid$$
$$\textsf{Impiegati}(\textsf{Matr} : c,\ \textsf{Nome} : n,\ \textsf{Età} : e,\ \textsf{Stip} : s) \wedge$$
$$\textsf{Supervisione}(\textsf{Impiegato} : m,\ \textsf{Capo} : c) \wedge$$
$$\neg\exists m'(\exists n'(\exists e'(\exists s'(\textsf{Impiegati}(\textsf{Matr} : m',\ \textsf{Nome} : n',\ \textsf{Età} : e',\ \textsf{Stip} : s') \wedge$$
$$\textsf{Supervisione}(\textsf{Impiegato} : m',\ \textsf{Capo} : c) \wedge s' \leq 40))))\}$$

In alternativa, possiamo utilizzare quantificatori universali:

$$\{\textsf{Matr} : c,\ \textsf{Nome} : n \mid$$
$$\textsf{Impiegati}(\textsf{Matr} : c,\ \textsf{Nome} : n,\ \textsf{Età} : e,\ \textsf{Stip} : s) \wedge$$
$$\textsf{Supervisione}(\textsf{Impiegato} : m,\ \textsf{Capo} : c) \wedge$$
$$\forall m'(\forall n'(\forall e'(\forall s'(\neg(\textsf{Impiegati}(\textsf{Matr} : m',\ \textsf{Nome} : n',\ \textsf{Età} : e',\ \textsf{Stip} : s') \wedge$$
$$\textsf{Supervisione}(\textsf{Impiegato} : m',\ \textsf{Capo} : c)) \vee s' > 40))))\}$$

Questa espressione seleziona un capo c se, per ogni quadrupla di valori m', n', e' e s' relativi a impiegati di c, si ha che s' è maggiore di 40. La struttura $\neg f \vee g$

corrisponde alla condizione "se f allora g" in quanto è vera in tutti i casi escluso in cui f è vera e g è falsa. Notiamo le leggi di De Morgan che valgono per gli operatori dell'algebra di Boole:

$$\neg\nabla f \wedge g\partial = \neg\nabla f\partial \vee \neg\nabla g\partial$$

$$\neg\nabla f \vee g\partial = \neg\nabla f\partial \wedge \neg\nabla g\partial$$

valgon allo stesso modo anche per i quantificatori:

$$\exists x\nabla f\partial = \neg\nabla\forall x\nabla\neg\nabla f\partial\partial\partial$$

$$\forall x\nabla f\partial = \neg\nabla\exists x\nabla\neg\nabla f\partial\partial\partial$$

5.2.2 Pregi e difetti del calcolo su domini

Il calcolo relazionale presenta aspetti interessanti, soprattutto per la dichiaratività, ma anche alcuni difetti e limitazioni. Innanzitutto, notiamo che il calcolo ammette espressioni che hanno veramente poco senso. Ad esempio l'espressione:

$$=:A_1 \ : \ x_1, A_2 \ : \ x_2 \mid R\nabla A_1 \ : \ x_1\partial \wedge x_2 = x_2{=}$$

produce come risultato una relazione su A_1 e A_2 costituita da tuple il cui valore su A_1 compare nella relazione R e il valore su A_2 è un qualunque valore del dominio. In particolare, se cambia il dominio, cambia anche la risposta all'interrogazione. Se il dominio è infinito, anche la risposta è infinita, il che è indesiderabile.
Pertanto, può essere utile introdurre un concetto: un'espressione di un linguaggio di interrogazione è *indipendente dal dominio* se il suo risultato, su ciascuna istanza di base di dati, non varia al variare del dominio rispetto al quale l'espressione è valutata. Un linguaggio è indipendente dal dominio se tutte le sue espressioni sono indipendenti dal dominio. Il requisito di indipendenza dal dominio è fondamentale per i linguaggi reali, perchè la maggior parte delle volte le espressioni dipendenti dal dominio non hanno utilità pratica e possono produrre risultati di grandi dimensioni.
Il calcolo relazionale, dagli esempi visti prima non è indipendente dal dominio, mentre, l'algebra relazionale lo è, perchè costruisce i risultati a partire dalle relazioni nella base di dati, senza far riferimento ai domini degli attributi.
Inoltre, l'algebra relazione il calcolo non sono equivalenti, perchè non hanno espressioni equivalenti, infatti, il calcolo relazione ammette espressioni dipendenti dal dominio.

Se limitiamo la nostra attenzione al sottoinsieme del calcolo relazionale costituito dalle sole espressioni indipendenti dal dominio, otteniamo un linguaggio equivalente all'algebra relazionale. Infatti:

- per ogni espressione del calcolo relazionae che sia indipendente dal dominio esiste un'espressione dell'algebra relazione equivalente a essa;
- per ogni espressione dell'algebra relazione esiste un'espressione del calcolo relazionale equivalente a essa.

Oltre al problema della dipendenza dal dominio, un altro svantaggio è quello di richiedere numerose variabili. Quando poi sno necessarie quantificazioni, anche i quantificatori si moltiplicano.
Per superare i limiti sul calcolo dei domini, è stata proposta un'altra versione del calcolo relazione in cui le variabili, anzichè denotare singoli valori, denotano tuple. Molto spesso il numero di variabili si riduce di tanto, perchè si ha una variabile per ciascuna relazione coinvolta. Diventa poi necessario associare una struttura a ciascuna variabile e realizzare opportunamente le operazioni di confronto. Potremmo definire un *calcolo relazione su tuple* perfettamente corrispondente al calcolo su domini, ma preferiamo passare a un linguaggio che, recependo le caratteristiche del calcolo su tuple, superi le dipendenza dal dominio, attraverso l'associazione delle variabili alle relazioni della base di dati.

5.2.3 Calcolo su tuple con dichiarazioni di range

Le espressioni del *calcolo su tuple con dichiarazioni di range* hanno la forma:

$$=:T \mid L \mid f=$$

ove:

- T è la *target list*, con elementi del tipo $Y : x.Z$ (o semplicemente $x.Z$, abbreviazione per $Z : x.Z$) con x variabile e Y e Z sequenze di attributi; gli attributi in Z devono comparire nello schema della relazione che costituisce il *range* di x; si può anche scrivere $x.*$, come abbreviazione di $X : x.X$, dove il range della variabile x è una relazione sull'insieme di attributi X;
- L è la *range list*, che elenca le variabili libere della formula f con i relativi range: infatti, L è una lista di elementi del tipo $x\nabla R\partial$, con x variabile e R nome di relazione;

- f è una formula con:
  - atomi del tipo $x.A\theta c$ o $x_1A_1\theta x_2A_2$, che confrontano, rispettivametne, il valore di x sull'attributo A con la costante c e il valore di x_1 su A_1 con quello di x_2 su A_2;
  - connettivi come nel calcolo su domini;
  - quantificatori che associano i range alle relativi variabili:

    $$\exists x \nabla R \partial \nabla f \partial \quad \forall x \nabla R \partial \nabla f \partial$$

    $\exists x \nabla R \partial \nabla f \partial$ significa "esiste nella relazione R una tupla x che soddisfa la formula f".

Le dichiarazioni di range specificano che la variabili possono assumere come valore solo tuple nella relazione rispettivamente associata e garantiscono così l'indipendenza dal dominio. Di conseguenza, quest linguaggio non ha bisogno di condizioni atomiche come quelle viste nel calcolo su domini, che specificano l'appartenenza di una tupla a una relazione.
Vediamo come possono espresse in questo linguaggio le varie interrogazioni già formulate in algebra e in calcolo su domini.
La prima interrogazione chiede matricola, nome, età e stipendio degli impiegati che guadagnano più di 40 mila euro:

$$=\!:i.\ *|\ i \nabla \texttt{Impiegati} \partial \mid i.\textbf{Stipendio} > 40 \dashv\!\vdash$$

Per produrre solo alcuni degli attributi, matricola, nome ed età degli impiegati che guadagnano più di 40 mila euro, è sufficiente modificare la target list:

$$=\!:i.\nabla \textbf{Matr, Nome, Età} \partial \mid i \nabla \texttt{Impiegati} \partial \mid i.\textbf{Stipendio} > 40 \dashv\!\vdash$$

Per interrogazioni che coinvolgono più relazioni sono necessarie più variabili, con specifica delle condizioni di correlazione sugli attributi. L'interrogazione che vuole trovare la matricole dei capi degli impiegati che guadagnano più di 40 mila euro, può essere formulata con:

$$=\!:s.\textbf{Capo} \mid i \nabla \texttt{Impiegati} \partial, s \nabla \texttt{Supervisione} \partial \mid$$
$$i.\textbf{Matr} = s.\textbf{Impiegato} \wedge i.\textbf{Stipendio} > 40 \dashv\!\vdash$$

Nel caso delle espressioni corrispondenti al join di una relazione con se stessa, abbiamo più variabili aventi la stessa relazione come range. L'interrogazione: trovare nome e stipendio dei capi degli impiegati che guadagnano più di 40 mila euro può essere realizzata con l'espressione:

$$=\!:\mathbf{NomeC}, \mathbf{StipC} \ : \ i'.\nabla \mathbf{Nome,\ Stip}\partial \mid$$

$$i'\nabla \texttt{Impiegati}\partial, s\nabla \texttt{Supervisione}\partial, i\nabla \texttt{Impiegati}\partial \mid$$

$$i'.\mathbf{Matr} = s.\mathbf{Capo} \wedge s.\mathbf{Impiegato} = i.\mathbf{Matr} \wedge i.\mathbf{Stipendio} > 40\text{⧧}$$

Le interrogazioni con i quantificatori mostrano appieno la sinteticità e praticità del calcolo su tuple con dichiarazioni di range. L'interrogazione che richiede di trovare matricola e nome dei capi i cui impiegati guadagnano tutti più di 40 mila euro si può esprimere in diversi modi in base all'uso dei quantificatori e della negazione. Con quantificatori universali:

$$=\!:i.\nabla \mathbf{Matr,\ Nome}\partial \mid i\nabla \texttt{Impiegati}\partial, s\nabla \texttt{Supervisione}\partial \mid$$

$$i.\mathbf{Matr} = s.\mathbf{Capo} \wedge \forall i'\nabla \texttt{Impiegati}\partial\nabla\forall s'\nabla \texttt{Supervisione}\partial$$

$$\nabla\neg\nabla s.\mathbf{Capo} = s'.\mathbf{Capo} \wedge s'.\mathbf{Impiegato} = i'.\mathbf{Matr}\partial \vee i'.\mathbf{Stipendio} > 40\partial\partial\text{⧧}$$

Con quantificatori esistenziali negati:

$$=\!:i.\nabla \mathbf{Matr,\ Nome}\partial \mid i\nabla \texttt{Impiegati}\partial, s\nabla \texttt{Supervisione}\partial \mid$$

$$i.\mathbf{Matr} = s.\mathbf{Capo} \wedge \neg\nabla\exists i'\nabla \texttt{Impiegati}\partial\nabla\exists s'\nabla \texttt{Supervisione}\partial$$

$$\nabla s.\mathbf{Capo} = s'.\mathbf{Capo} \wedge s'.\mathbf{Impiegato} = i'.\mathbf{Matr} \wedge i'.\mathbf{Stipendio} \leq 40\partial\partial\partial\text{⧧}$$

Il calcolo su tuple con dichiarazioni di range non permette di esprimere tutte le interrogazioni che possono essere formulate in algebra relazionale. In particolare, le interrogazioni i cui risultati possono provenire indifferentemente da due o più relazioni, che in algebra realizziamo con l'operatore di unione, non possono essere espresse con questa versione del calcolo: infatti, i risultati sono costruiti a partire dalle variabili libere, i cui range sono definiti nella target list, e ogni variabile ha come range una sola relazione.
Consideriamo l'unione di due relazioni sugli stessi attributi: date $R_1\nabla AB\partial$ e $R_2\nabla AB\partial$, vogliamo formulare l'interrogazione che in algebra esprimeremmo con l'unione di R_1 e R_2. Se l'aspressione avesse due variabili libere, allora ogni tupla del risultato dovrebbe corrispondere a una tupla di ciascuna delle relazioni, il che non è necessario perchè l'unione richiede alle tuple di comparire in almeno uno degli

operandi.
Se potessimo associare a una variabile un range costituito da più relazioni, risolveremmo il problema dell'unione ma non riusciremmo a formulare unioni complesse i cui operandi siano sottoespressioni non direttamente corrispondenti a schemi di relazione. Ad esempio, date due relazioni $R_1 \nabla ABC \partial$ e $R_2 \nabla BCD \partial$, l'unione delle loro proiezioni su BC:

$$\pi_{BC} \nabla R_1 \partial \cup \pi_{BC} \nabla R_2 \partial$$

non potrebbe essere espressa, perchè le due relazioni hanno schemi diversi e una variabile non può essere associata ad entrambe.
Per quanto riguarda gli operatori di interesezione e differenza questi sono esprimibili.

- L'interesezione richiede che le tuple del risultato appartengano a entrambi gli opernadi, quindi si può costruire il risultato a partire da una relazione, richiedendo l'esistenza di una tupla uguale nell'altra relazione. Ad esempio:

  $$\pi_{BC} \nabla R_1 \partial \cap \pi_{BC} \nabla R_2 \partial$$

  può essere espressa con:

  $$=:x_1.BC \mid x_1 \nabla R_1 \partial \mid \exists x_2 \nabla R_2 \partial \nabla x_1.B = x_2.B \wedge x_1.C = x_2.C \partial =:$$

- La differenza produce le tuple di un operando non contenute nell'altro. Ad esempio:

  $$\pi_{BC} \nabla R_1 \partial - \pi_{BC} \nabla R_2 \partial$$

  può essere espressa con:

  $$=:x_1.BC \mid x_1 \nabla R_1 \partial \mid \neg \exists x_2 \nabla R_2 \partial \nabla x_1.B = x_2.B \wedge x_1.C = x_2.C \partial =:$$

Capitolo 6

Il modello Entità-Relazione

Il modello Entità-Relazione, il modello E-R, è un modello *concettuale* di dati e fornisce una serie di struttire, dette *costrutti*, atte a descrivere la realtà di interesse in maniera facile da comprendere e che prescinde dall'organizzazione dei dati nei calcolatori.
Questi costrutti sono usati per definire *schemi* che descrivono l'organizzazione e la struttura delle *occorrenze* dei dati, ovvero i valori assunti dai dati al variare del tempo.

6.1 I costrutti principali del modello

Analizziamo i costrutti principali di questo modello:

- le entità;
- le relazioni;
- gli attributi.

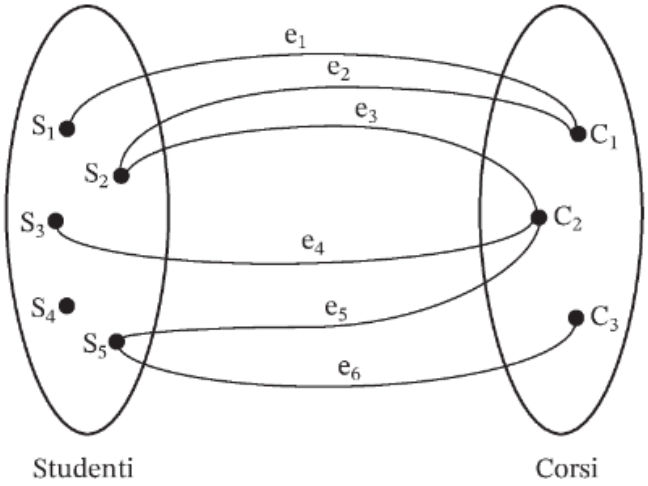
6.1.1 Entità

Le entità rappresentano classi di oggetti che hanno proprietà comuni ed esistenza "autonoma" ai fini dell'applicazione di interesse: `Città`, `Dipartimento`, `Impiegato` sono esempi di entità di un'applicazione aziendale. Una occorrenza di un'entità è un oggetto della classe che l'entità rappresenta, come Roma per l'entità `Città`. Un'occorrenza di entità non è un valore che identifica un oggetto

ma è l'oggetto stesso. Una conseguenza di questo è che un'occorrenza di entità ha un'identità indipendente dalle proprietà ad esso associate.
In questo il modello E-R presenta una grande differenza rispetto al modello relazionale nel quale non possiamo rappresentare un oggetto senza conoscere alcune sue proprietà.
In uno schema, ogni entità ha un nome che la identifica univocamente e viene rappresentata graficamente mediante un rettangolo con il nome dell'entità all'interno.

6.1.2 Relazioni o associazioni

Le relazioni o associazioni rappresentano legami logici, significativi per l'applicazione di interesse, tra due o più entità. `Residenza` è un esempio di relazione che può sussistere tra le entità `Città` e `Impiegato`. Un'occorenza di relazione è un'ennupla costuita da occorrenze di entità, una per ciascuna delle entità coinvolte. Segue un esempio di occorrenze della relazione `Esame`:



In uno schema E-R, ogni relazione ha un nome che la identifica univocamente e viene rappresentata graficamente mediante un rombo con il nome della relazione all'interno, e da linee che connettono la relazione con ciascuna delle sue componenti.

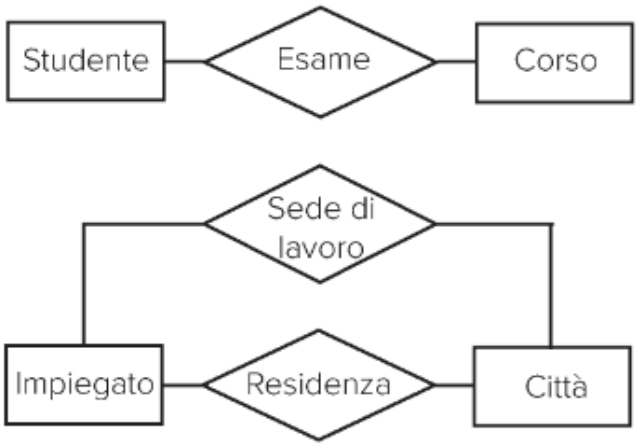
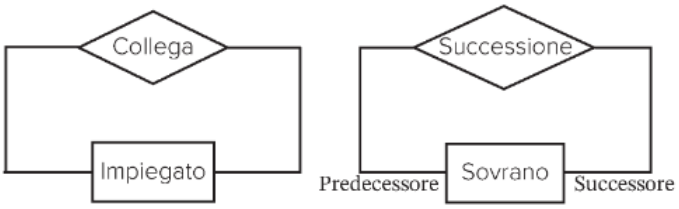


Figura 6.1: Esempi di relazioni tra entità nel modello E-R

Dalla figura notiamo che possono esistere relazioni diverse che coinvolgono le stesse entità. Nella scelta dei nomi di relazione è preferibile utilizzare sostantivi invece che verbi in modo da non "assegnare" un verso alla relazione.
L'insieme di occorrenza di una relazione del modello E-R è, a tutti gli effetti, una relazione matematica tra le occorrenze delle entità coinvolte, quindi, un sottoinsieme del loro prodotto cartesiano. Di conseguenza, non ci possono essere ennuple ripetute.
Inoltre, è anche possibile avere relazioni *ricorsive*, ovvero relazioni tra un'entità e se stessa, come mostrato nella seguente figura:



Per esempio la relazione `Collega` sull'entità `Impiegato` connette coppie di impiegati che lavorano insieme, mentre la relazione `Successione` sull'entità `Sovrano` associa a ogni sovrano di una dinastia il suo successore. In quest'ultimo esempio, però, è necessario stabilire i due *ruoli* che l'entità coinvolta gioca nella relazione.
Infine, è possibile avere relazioni n-arie, ovvero relazioni che coinvolgono più di due entità, come mostrato nella seguente figura:

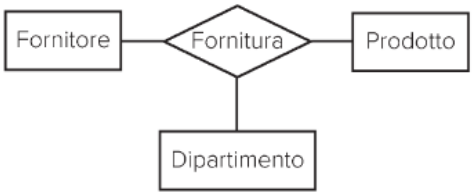
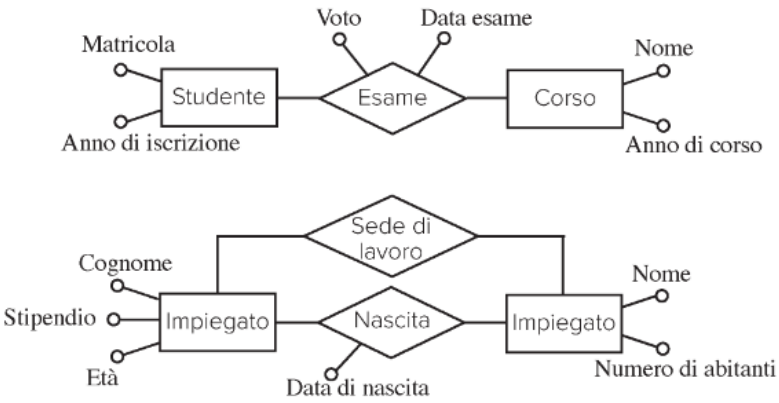


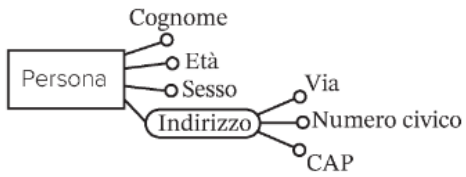
Figura 6.2: Esempio di relazione che coinvolge tre entità

6.1.3 Attributi

Gli attributi descrivono le proprietà elementari di entità o relazioni che sono di interesse ai fini dell'applicazione, ad esempio **Cognome, Stipendio** ed **Età** sono possibili attributi dell'entità `Impiegato`. Un attributo associa a ciascuna occorrenza di entità un valore appartenente a un insieme, detto *dominio*, che contiene i valori ammissibili per l'attributo. La seguente figura mostra come vengono rappresentati graficamente gli attributi:



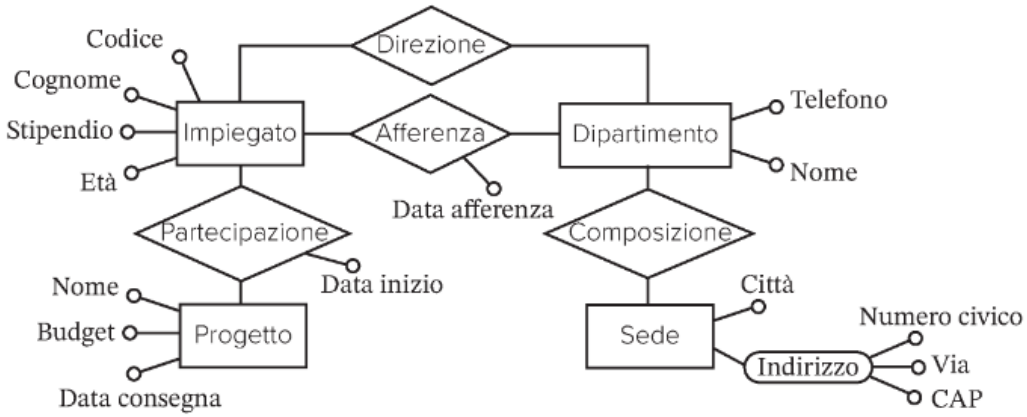
Di solito, i domini non vengono riportati nello schema, ma sono descritti nella documentazione associata.
A volte, può risultare comodo raggruppare attributi di una stessa entità che presentano affinità nel loro significato: l'insieme di attributi che si ottiene si chiama *attributo composto*. Un esempio di attributo composto viene mostrato nella seguente figura:



Tuttavia, per ridurre la complessità degli schemi, gli attributi composti vengono usati raramente utilizzando, preferibilemente, degli attributi atomici.

6.1.4 Costruzione di schemi con i costrutti di base

I tre costrutti descritti ci consentono già di costruire schemi per descrivere realtà di una certa complessità. Consideriamo, ad esempio, il seguente schema E-R:



Si comprende facilemente che questo schema rappresenta informazioni organizzative relative a un'azienza con diverse sedi. Partendo dall'entità `Sede` si può vedere che una sede dell'azienza è dislocata in una certa città con un certo indirizzo. Ogni sede è organizzata in dipartimenti con un nome e un telefono. A questi dipartimenti afferiscono, a partire da una certa data, gli impiegati dell'azienda e ci sono impiegati che dirigono i dipartimenti. Gli impiegati sono rappresentati tramite codice, cognome, stipendio ed età. Gli impiegati lavorano su progetti a partire da una certa data e ogni progetto ha un nome, un budget e una data di consegna.

6.2 Altri costrutti del modello

Esaminiamo altri costrutti del modello E-R:

- le cardinalità delle relazioni e degli attributi;
- gli identificatori delle entità;
- le generalizzazioni.

Le generalizzazioni è un costrutto *nuovo*, gli altri, invece, costituiscono dei *vincoli di integrità* sui costrutti visti.

6.2.1 Cardinalità delle relazioni

Le cardinalità delle relazioni vengono specificate per ciascuna partecipazione di entità a una relazione e descrivono il numero minimo e massimo di occorrenze di relazione a cui una occorrenza dell'entità può partecipare. Quindi, dicono quante volte, in una relazione tra entità, un'occorrenza di queste entità può essere legata a occorrenze delle altre entità coinvolte.
Consideriamo il seguente esempio:



Figura 6.3: Cardinalità di una relazione del modello E-R

In questo esempio, stiamo dicendo che un impiegato deve essere assegnato almeno un incarico ma non più di 5. Per quanto riguarda l'incarico, invece, l'incarico può non essere assegnato a nessun impiegato oppure può essere assegnato a un numero di impiegati ≤ 50.
È possibile assegnare un qualunque intero non negativo a una cardinalità di una relazione, con l'unico vincolo che la cardinalità minima deve essere minore o uguale della cardinalità massima. Nella maggior parte dei casi si utilizzano solo tre valori: 0, 1 e il simbolo N.

- per la cardinalità minima, 0 o 1; se 0 si dice che la partecipazione alla relazione è *opzionale*, se 1 si dice che la partecipazione è *obbligatoria*;

- per la cardinalità massima, 1 o N; se 1 si dice che la partecipazione alla relazione può essere vista come una funzione che associa a un'occorrenza dell'entità una sola occorrenza dell'altra entità che partecipa alla relazione. Se N c'è un'associazione con un numero arbitrario di occorrenza dell'altra entità.

Osservando le cardinalità massime è possibile classificare le relazioni binarie in base al tipo di corrispondenza che viene stabilita tra le occorrenza delle entità coinvolte. Le relazioni aventi cardinalità massima pari a 1 per entrambe le entità coinvolte, definiscono una corrispodenza uno a uno tra le occorrenze di tali entità e quindi vengono chiamate *relazioni uno a uno*. Analogamente, le relazioni aventi un'entità con cardinalità massima pari a 1 e l'altra con cardinalità massima pari a N sono denominate *relazioni uno a molti*. Infine, le relazioni aventi cardinalità massima pari a N per entrambe le entità coinvolte sono denominate *relazinoi molti a molti*.
Per quanto riguarda le cardinalità minime, va detto il caso di partecipazione obbligatoria è abbastanza raro, perchè quando si aggiunge una nuova occorrenza di entità molto spesso non sono note le corrispondenti occorrenze delle entità a esse collegate.
Nelle relazioni n-arie le entità coinvolte partecipano quasi sempre con cardinalità massima pari a N. Nel caso in cui un'entità partecipa a una relazione n-aria con cardinalità massima pari a 1, significa che ogni sua occorrenza può essere legata a una sola occorrenza della realzione, e quindi ad un'unica eenupla di occorenze delle altre entità coinvolte nella relazione. Questo significa che è possibile eliminare la relazione n-aria e legare direttamente tale entità con le altre entità mediante delle relazioni binarie di tipo uno a molti.

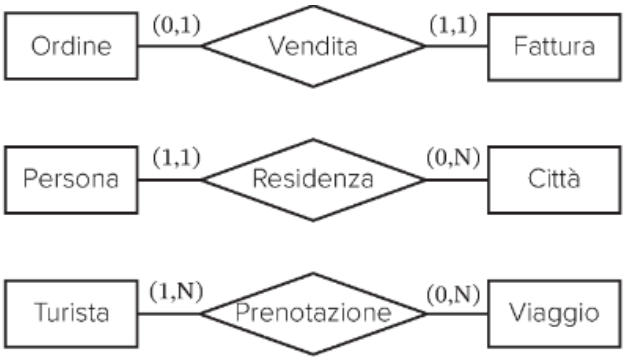


Figura 6.4: Esempio di relazioni con cardinalità

6.2.2 Cardinalità degli attributi

Le cardinalità sugli attributi possono essere specificate per gli attributi di entità o realzioni e descrivono il numero minimo e massimo di valori dell'attributo associati a ogni occorrenza di entità o relazione. Nella maggiore parte dei casi la cardinalità è (1, 1) e viene omessa. Il valore per un certo attributo può essere però nullo, oppure possono esistere diversi valori di un certo attributo per un'occorrenza di entità. Queste situazioni si possono rappresentare assocaindo all'attributo una cardinalità minima pari a 0 nel primo caso, e una cardinalità massima pari a molti N, nel secondo.
Come per le relazioni, un attributo con cardinalità minima pari a 0 è *opzionale* per la relativa entità o relazione, mentre è *obbligatorio* se la sua cardinalità minima è pari a 1. Un attributo si dice *multivalore* se la sua cardinalità massima è pari a N.

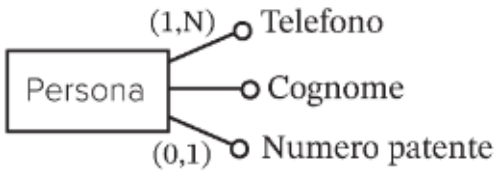
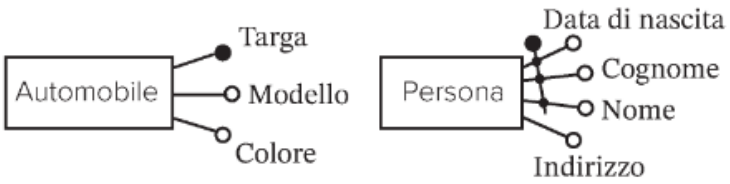


Figura 6.5: Esempio di attributi con cardinalità

6.2.3 Identificatori delle entità

Gli identificatori delle entità vengono specificati per ciascuna entità di uno schema e descrivono i concetti dello schema che permettono di identificare in maniera univoca ciascuna occorrenza delle entità. In molti casi, uno o più attributi di un'entità sono sufficienti ad individuare un identificatore e prende il nome di identificatore *interno*.
Alcune volte, però, gli attributi di un entità non sono sufficienti a identificare univocamente le sue occorrenze. In questo caso, per identificare univocamente un'occorrenza di un'entità si fa riferimento a uno o più attributi di un'altra entità. Un'entità E può essere identificata da altre entità solo se tali entità sono coinvolte in una relazione a cui E partecipa con cardinalità (1, 1). Nei casi in cui l'identificazione di un'entità è ottenuta utilizzando altre entità si parla di identificatore *esterno*. Seguono degli esempi di rappresentazione di identificatore interno ed esterno:

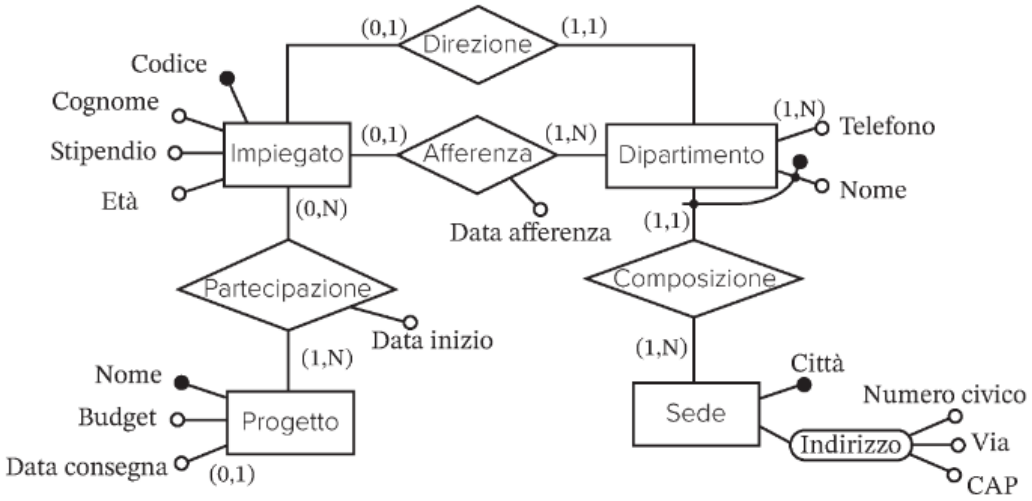




Facciamo alcune considerazioni:

- un identificatore può coinvolgere uno o più attributi, ognugno dei quali deve avere cardinalità (1, 1).
- un'identificazione esterna può coinvolgere una o più entità, ognuna delle quali deve essere membro di una relazione alla quale l'entità da inteficare partecipa con cardinalità (1, 1).
- un'identificazione esterna può coinvolgere un'entità che è a sua volta identificata esternamente, purchè non vengano generati cicli di identificazioni esterne.
- ogni entità deve avere almeno un identificatore, ma ne può avere in generale più di uno; nel caso di più identificatori, le restrizioni sopra descritte sono più concessive in quanto gli attributi e le entità coinvolte in alcune identificazioni, tranne una, possono essere opzionali.

Segue un esempio di schema con rappresentazione di identificatori interni ed esterni:



6.2.4 Generalizzazioni

Le generalizzazioni riguardano legami logici tra un'entità E, detta entità genitore, e una o più entità $E_1, \ldots, E_n$ dette entità *figlie*, di cui E è generale, nel senso che le comprende come caso particolare. Si dice in questo caso che E è generalizzazione di $E_1, \ldots, E_n$ e che le entità $E_1, \ldots, E_n$ sono *specializzazioni* dell'entità E. Per esempio, l'entità `Persona` è una generalizzazione delle entità `Uomo` e `Donna`. Di conseguenze, esse sono delle specializzazioni dell'entità `Persona`.
Tra le entità coinvolte in una generalizzazione valgono le seguenti proprietà.

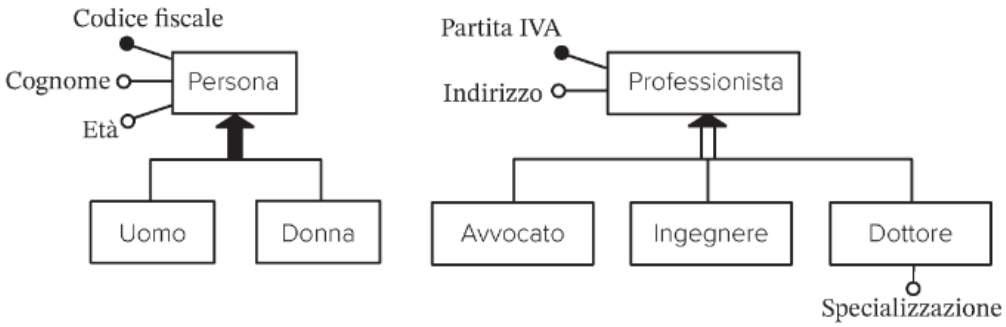
- Ogni occorrenza di un'entità figlia è anche occorrenza dell'entità genitore.
- Ogni proprietà dell'entità genitore, come attributi, identificatori, relazioni e altre generalizzazioni, è anche proprietà delle entità figlie. Questa proprietà è nota sotto il nome di *ereditarietà*.

Le generalizzazioni vengono rappresentate graficamente mediante delle frecce che congiungono le entità figlie con l'entità genitore. Per le entità figlie, le proprietà ereditate non vanno rappresentate esplicitamente.
Le generalizzazioni possono essere classificate sulla base di due proprietà tra loro ortogonali.

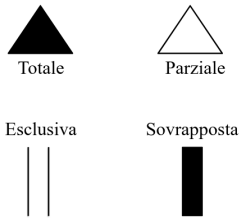
- Una generalizzazione è *totale* se ogni occorrenza dell'entità genitore è un'occorrenza di almeno una delle entità figlie, altrimenti è *parziale*;

- Una generalizzazione è *esclusiva* se ogni occorrenza dell'entità genitore è al più un'occorenza di una delle delle entità figlie, altrimenti è *sovrapposta*.

Ad esempio, la generalizzazione tra `Persona`, `Uomo` e `Donna` è totale perchè gli uomini e le donne costituiscono tutte le persone, ed è anche esclusiva perchè una persone è uomo o donna. Una generalizzazione tra l'entità `Professionista` e le entità `Ingegnere` e `Dottore` è invece parziale ed esclusiva. Tra l'entità `Persona` e le entità `Studente` e `Lavoratore` esiste una generalizzazione parziale e sovrapposta, perchè esistono studenti che sono anche lavoratori.
Seguono degli esempi di generalizzazioni tra entità:



Le generalizzazioni sovrapposte possono essere facilmente trasformate in generalizzazioni esclusive aggiungendo una o più entità figlie per rappresentare i concetti che costituiscono le "intersezioni" delle entità che si sovrappongono. Nel caso degli studenti e dei lavoratori è sufficiente aggiungere l'entità `StudenteLavoratore`.
Una stessa entità può essere coinvolta in più generalizzazioni diverse. Possono anche esserci generalizzazioni su più livelli, in questo caso si parla di *gerarchia* di generalizzazioni. Infine, una generalizzazione può avere una sola entità figlia: si parla in questo caso di *sottoinsieme*.
Le proprietà di esclusività e sovrapposta e di generalizzazione totale e parziale venegono rappresentate nel seguente modo:



Capitolo 7

Progettazione concettuale

La progettazione concettuale di una base di dati consiste nella costruzione di uno schema Entità-Relazione in grado di descrivere al meglio le specifiche sui dati di un'applicazione. Anche nel caso di applicazioni non particolarmente complesse, lo schema che si ottiene può contenere molti concetti correlati in una maniera piuttosto complicata. La costruzione finale dello schema è un processo graduale: lo schema concettuale viene progressivamente raffinato e arricchito attraverso una serie di trasformazioni ed eventuali correzioni.

7.1 La raccolta e l'analisi dei requisiti

Il reperimento e l'analisi dei requisiti di un'applicazione sono attività difficilmente standardizzabili perchè dipendono molto dall'applicazione con cui si ha a che fare.
Per *raccolta dei requisiti* si intende la completa individuazione dei problemi che l'applicazione da realizzare deve risolvere e le caratteristiche che tale applicazione dovrà avere.
Per caratteristiche del sistema si intendono sia gli aspetti statistici, i dati, sia gli aspetti dinamici, le operazioni sui dati. I requisiti vengono inizialmente raccoolti in specifiche espresse generalmente in linguaggio naturale e, per questo motivo, spesso ambigue e disorganizzate.
L'*analisi dei requisiti* consiste nel chiarimento e nell'organizzazione delle specifiche dei requisiti. Queste sono attività fortmente interconnesse: l'attività di analisi inizia con i i primi requisiti ottenuti per poi procedere di pari passo con l'attività di raccolta.

I requisiti di un'applicazione provengono, nella maggior parte dei casi, da fonti diverse. Le principali fonti di informazione sono, in genere, le seguenti.

- Gli *utenti dell'applicazione*. In questo caso le informazioni si acquisiscono mediante opportune interviste, anche ripetute, oppure attraverso una documentazione scritta che gli utenti possono aver predisposto appositamente per questo scopo.

- Tutta la *documentazione esistente* che ha qualche attinenza con il problema allo studio come moduli, regolamenti interni, procedure aziendali e normative. È richiesta, in questo caso, un'attività di raccolta e selezione che viene assistita dagli utenti, ma è a carico del progettista.

- *Realizzazioni preesistenti*, ovvero applicazioni che si devono rimpiazzare o che devono interagire in qualche maniera con il software da realizzare. La conoscenza delle caratteristiche di questi pacchetti software può fornirci importanti informazioni anche in relazione ai problemi esistenti che è necessario risolvere.

Nella fase di acquisizione delle specifiche, gioca un importante ruolo l'interazione con gli utenti del sistema informativo. Durante questa interazione può avvenire che utenti diversi forniscano informazioni diverse, spesso complementari ma contraddittorie. Quindi, nel corso delle interviste, è opportuno effettuare con l'utente verifiche di comprensione e consistenza sulle informazioni che si stanno raccogliendo, attraverso esempi oppure richiedendo definizioni e classificazioni precise.
Inoltre, in questa fase, è importante individuare gli aspetti essenziali rispetto a quelli marginali e procedere per raffinamenti successivi.
La specifica dei requisiti avviene spesso, facendo uso di descrizioni in linguaggio naturale e, data la sua ambiguità, è bene effettuare una profonda analisi del testo che descrive le specifiche per filtrare eventuali inesattezze e i termini ambigui presenti. Ad esempio, supponiamo di dover progettare una base di dati per una società di formazione e di aver raccolto, sulla base di alcune interviste fatte al personale di questa società, le specifiche dei dati espresse in linguaggio naturale.

Esempio : Società di formazione

	Società di formazione
1	*Si vuole realizzare una base di dati per una società che eroga corsi, di*
2	*cui vogliamo rappresentare i dati dei partecipanti ai corsi e dei docenti.*
3	*Per i partecipanti (circa 5000), identificati da un codice, si vuole me-*
4	*morizzare il codice fiscale, il cognome, l'età, il sesso, il luogo di nascita,*
5	*il nome dei loro attuali datori di lavoro, i posti dove hanno lavorato*
6	*in precedenza insieme al periodo, l'indirizzo e il numero di telefono, i*
7	*corsi che hanno frequentato (i corsi sono in tutto circa 200) e il giudizio*
8	*finale. Rappresentiamo anche i seminari che stanno attualmente fre-*
9	*quentando e, per ogni giorno, i luoghi e le ore dove sono tenute le lezioni.*
10	*I corsi hanno un codice, un titolo e possono avere varie edizioni con*
11	*date di inizio e fine e numero di partecipanti. Se gli studenti sono liberi*
12	*professionisti, vogliamo conoscere l'area di interesse e, se lo possiedono,*
13	*il titolo. Per quelli che lavorano alle dipendenze di altri, vogliamo co-*
14	*noscere invece il loro livello e la posizione ricoperta. Per gli insegnanti*
15	*(circa 300), rappresentiamo il cognome, l'età, il posto dove sono nati, il*
16	*nome del corso che insegnano, quelli che hanno insegnato nel passato e*
17	*quelli che possono insegnare. Rappresentiamo anche tutti i loro recapiti*
18	*telefonici. I docenti possono essere dipendenti interni della società o*
19	*collaboratori esterni.*

È facile rendersi conto che tale testo presenta un certo problema di ambiguità e imprecisioni. Ad esempio, si utilizzano i termini *partecipante* e *studente* per indicare lo stesso concetto. La stessa cosa accade per i termini *docente* e *professore* e per i termini *corso* e *seminario*.
Proviamo a fissare alcune regole generali per ottenere una specifica dei requisiti più precisa e senza ambiguità.

- **Scegliere il corretto livello di astrazione**. È bene evitare di utilizzare termini troppo generici o troppo specifici che rendono poco chiaro un concetto. Nell'esempio della società di formazione sono stati utilizzati i termini *titolo*, alla riga 13, con riferimento ai partecipanti che sono liberi professionisti (che viene usato per indicare un concetto diverso alla riga 10) e *giudizio*, alla riga 7, con riferimento alla valutazione dei corsi, che andrebbero specificati meglio come *titolo professionale* o *votazione in decimi*.

- **Standardizzare la struttura delle frasi**. Nella specifica di requisiti è preferibile utilizzare sempre lo stesso stile sintattico. Per esempio, per "per

<dato> rappresentiamo <insieme di proprietà>".

- **Evitare frasi contorte**. Le definizioni devono essere semplici e chiare. Per esempio, *lavoratori dipendenti*, o più semplicemente dipendenti, è da preferire a *quelli che lavorano alle dipendenze di altri* alla riga 13.

- **Individuare sinonimi/omonimi e unificare i termini**. I *sinonimi* indicano termini diversi con lo stesso significato, per esempio *docente* alla riga 2 e *insegnante* alla riga 14, oppure *partecipante* alla riga 2 e *studente* alla riga 11; gli *omonimi* indicano termini uguali ma con diversi significati, ad esempio *posto*, riferito a "impiego" alla riga 5, *città* alla riga 15 e *luogo* riferito a "città" e ad "aula" alla riga 4.
  Queste situazioni possono generare ambiguità e vanno chiarite: nel caso di sinonimi unificando i termini, nel caso di omonimi utlizzando termini diversi o specificandoli meglio.

- **Rendere esplicito il riferimento tra termini**. Può succedere che l'assenza di un contesto di riferimento renda alcuni concetti ambigui: in questi casi bisogna esplicitare il riferimento tra termini. Per esempio, nella riga 6, non è chiaro se i termini *indirizzo* e *numero di telefono* sono relativi ai partecipanti o ai loro datori di lavoro; nella riga 13, nella frase *per quelli che lavorano...* si deve chiarire esplicitamente a chi ci stiamo riferendo, se ai partecipanti o ai docenti, per evitare confusione.

- **Costruire un glossario dei termini**. È molto utile, per la comprensione e la precisazione dei termini usati, definire un glossario che, per ogni termine, contenga: una breve descrizione, possibile sinonimi e altri termini contenuti nel glossario con i quali esiste un legame logico. Un glossario per la società di formazione può essere il seguente.

Esempio : Glossario della società di formazione

Termine	Descrizione	Sinonimi	Collegamenti
Partecipante	Partecipante ai corsi. Può essere un dipendente o un professionista	Studente	Corso, Datore
Docente	Docente dei corsi. Possono essere collaboratori esterni	Insegnante	Corso
Corso	Corsi offerti. Possono avere varie edizioni	Seminario	Docente, Partecipante
Datore	Datori di lavoro attuali e passati dei partecipanti ai corsi	Posto	Partecipante

Dopo aver individuato le varie ambiguità e le imprecisioni, esse vanno eliminate sostituendo i termini non corretti con i termini più adeguati. In caso di dubbio, è necessario intervistare nuovamente colui che ha fornito il dato o consultare la documentazione relativa.
Nell'esempio della società di formazione si possono apportare diverse modifiche. Il *luogo* di nascita dei partecipanti, alla riga 4, è un omonimo del luogo in cui si tengono le lezioni e va sostituito da *città* di nascita, cosi come *posto* alla riga 5 che va sostituito con *datore di lavoro*. Va poi chiarito che a riga 6 l'*indirizzo* e il *numero di telefono* fanno riferimento ai datori di lavoro dei partecipanti. Il *giudizio* alla riga 7 deve essere interpretato come *votazione in decimi*, mentre *periodo* alla riga 6 va interpretato come *dato di inizio e fine rapporto*. Bisogna inoltre specificare che i partecipanti frequentano o hanno frequentato specifiche *edizioni* di corsi. Per quanto riguarda gli altri termini che fanno riferimento ai corsi: *seminario* alla riga 8 è un sinonimo e va sostituito con *edizione di corso*; *giorno* alla riga 9, riferito alle lezioni, è troppo astratto e va utilizzato *giorno della settimana*; *luogo* alla riga 9 è un omonimo, che va sostituito da *aula*. Il termine *studente* alla riga 11, va sostituito con *partecipante*. Per *titolo*, alla riga 13, di un partecipante che è libero professionista si intende il suo *titolo professionale*. Per quello che riguarda i docenti abbiamo che *insegnante*, alla riga 14, è sinonimo di *docente*, *posto* indica la *città* di nascta, il *nome* del corso che insegnano, alla riga 16, è un sinonimo di *titolo* del corso e il *recapito telefonico*, alle righe 17-18, è sinonimo di *numero di telefono*.
A questo punto possiamo riscrivere le nostre specifiche apportando le modifiche proposte. È utile, in questa fase, decomporre il testo in gruppi di frasi omogenee, relative cioè agli stessi concetti. Otteniamo così la strutturazione delle specifiche

sui dati.

Esempio : Strutturazione delle specifiche della società di formazione

Frasi di carattere generale
Si vuole realizzare una base di dati per una società che eroga corsi, di cui vogliamo rappresentare i dati dei partecipanti ai corsi e dei docenti.

Frasi relative ai partecipanti
Per i partecipanti *(circa 5000), identificati da un codice, rappresentiamo il codice fiscale, il cognome, l'età, il sesso, la città di nascita, i nomi dei loro attuali datori di lavoro e di quelli precedenti (insieme alle date di inizio e fine rapporto), le edizioni dei corsi che stanno attualmente frequentando e quelli che hanno frequentato in passato, con la relativa votazione finale in decimi.*

Frasi relative ai datori di lavoro
Relativamente ai datori di lavoro *presenti e passati dei partecipanti, rappresentiamo il nome, l'indirizzo e il numero di telefono.*

Frasi relative ai corsi
Per i corsi *(circa 180), rappresentiamo il titolo e il codice, le varie edizioni con date di inizio e fine e, per ogni edizione, rappresentiamo il numero di partecipanti e il giorno della settimana, le aule e le ore dove si sono tenute le lezioni.*

Frasi relative a tipi specifici di partecipanti
Per i partecipanti *che sono liberi professionisti, rappresentiamo l'area di interesse e, se lo possiedono, il titolo professionale. Per i partecipanti che sono dipendenti, rappresentiamo invece il loro livello e la posizione ricoperta.*

Frasi relative ai docenti
Per i docenti *(circa 300), rappresentiamo il cognome, l'età, la città di nascita, tutti i numeri di telefono, il titolo del corso che insegnano, di quelli che hanno insegnato in passato e di quelli che possono insegnare. I docenti possono essere dipendenti interni della società di formazione o collaboratori esterni.*

Inoltre, accanto alle specifiche sui dati vanno raccolte le specifiche sulle operazioni da effettuare su questi dati. Bisgona cercare di impiegare la stessa terminologia usata per i dati e informarci anche sulla frequenza con la quale le varie operazinoi vengono eseguite. Nell'esempio della società di formazione, le operazioni sui dati potrebbero essere le seguenti:

Esempio : Operazioni sui dati nella società di formazione

- **Operazione 1**: inserisci un nuovo partecipante indicando tutti i suoi dati (operazione da effettuare in media 40 volte al giorno).
- **Operazione 2**: assegna un partecipante a un'edizione di corso (circa 50 volte al giorno);
- **Operazione 3**: inserisci un nuovo docente indicando tutti i suoi dati e i corsi che può insegnare (2 volte al giorno).
- **Operazione 4**: assegna un docente abilitato a un'edizione di corso (15 volte al giorno).
- **Operazione 5**: stampa tutte le informazioni sulle edizioni passate di un corso con titolo, orari lezioni e numero partecipanti (10 volte al giorno).
- **Operazione 6**: stampa tutti i corsi offerti, con informazioni sui docenti che possono insegnarli (20 volte al giorno).
- **Operazione 7**: per ogni docente, trova i partecipanti a tutti i corsi da lui/lei insegnati (5 volte a settimana).
- **Operazione 8**: effettua una statistica su tutti i partecipanti a un corso con tutte le informazioni su di essi, sull'edizione alla quale hanno partecipato e sulla rispettiva votazione (10 volte al mese).

Dopo questa strutturazione dei requisiti si avvia la prima fase della progettazione che consiste nella costruzione di uno schema concettuale in grado di descrivere in maniera adeguata tutte le specifiche dei dati raccolte.

7.2 Rappresentazione concettuale di dati

Cerchiamo di stabilire alcune pratiche per una corretta rappresentazione concettuale dei dati, analizzando prima alcuni criteri generali e poi alcuni classici *design patter*, ovvero soluzioni progettuali a problemi comuni.

7.2.1 Criteri generali di rappresentazione

Va precisato che non esiste una rappresentazione univoca di un insieme di specifiche, perchè le stesse informazioni possono essere rappresentate in modi dif-

ferenti e non comparabili. Comunque, quando ci si trova davanti a diverse possibilità, è utile avere delle indicazioni sulle scelte più opportune. Nel caso della progettazione concettuale conviene seguire le "regole concettuali" del modello E-R.

- *Se un concetto ha proprietà significative e/o descrive classi di oggetti con esistenza autonoma, è opportuno rappresentarlo con un'entità.* Ad esempio, nel caso delle specifiche relative alla società di formazione, è naturale rappresentare il concetto di *docente* con un'entità, in quanto possiede diverse proprietà e la sua esistenza è indipendente da altri concetti.
- *Se un concetto ha una struttura semplice e non possiede proprietà rilvanti associate, è opportuno rappresentarlo con un attributo di un altro concetto a cui si riferisce.* Ad esempio, nel caso della società di formazione, il concetto di età è certamente da rappresentare come attributo.
- *Se sono state individuate due (o più) entità e nei requisiti compare un concetto che le associa, questo concetto può essere rappresentato da una relazione.* Ad esempio, nel caso della società di formazione, il concetto di *partecipazione a un corso* è rappresentabile da una relazione tra le entità che rappresentano i partecipanti e i corsi.
- *Se uno o più concetti risultano essere casi particolari di un altro, è opportuno rappresentarli facendo uso di una generalizzazione.* Ad esempio, i concetti di *professionista* e *dipendente* costituiscono dei casi particolari del concetto di *partecipante* ed è quindi indicato definire una generalizzazione tra le entità che rappresentano questi concetti.

7.3 Strategie di progetto

Lo sviluppo di una schema concettuale a partire dalle sue specifiche può essere considerato a tutti gli effetti un processo di ingegnerizzazione e, come tale, risultano a esso applicabili strategie di progetto utilizzate anche in altre discipline.

7.3.1 Strategia top-down

In questa strategia, lo schema concettuale viene prodotto mediante una serie di raffinamenti successivi, a partire da uno schema iniziale che descrive tutte le specifiche con pochi concetti molto astratti. Lo schema viene poi via via raffinato

mediante opportune trasformazioni che aumentano il dettaglio dei vari concetti presenti.
Nel passaggio da un livello di raffinamento a un altro, lo schema viene modificato facendo uso di alcune trasformazioni elementari che vengono denominate *primitive di trasformazione top-down*.

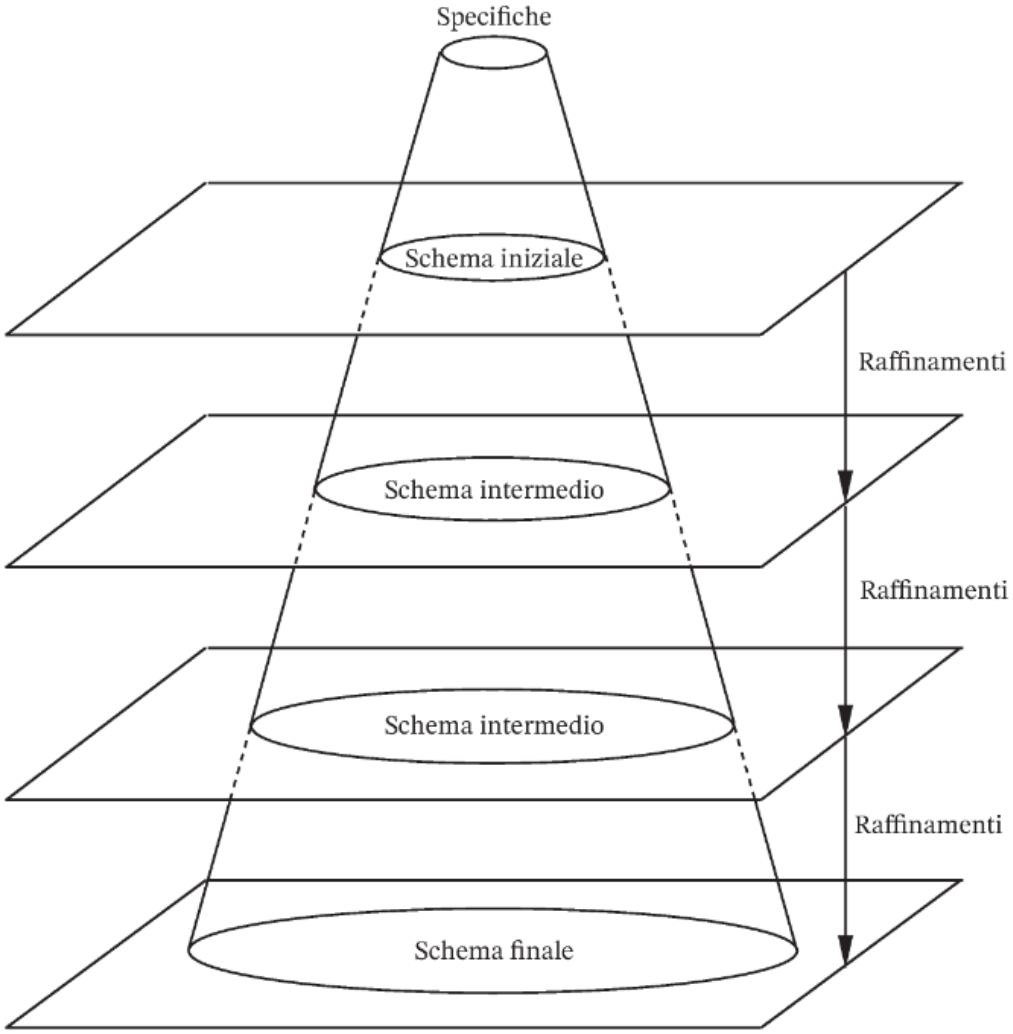


Figura 7.1: La strategia top down

Esempi di primitive di trasformazione top-down sono:

- la definizione degli attributi di un'entità o di una relazione; per esempio, la specifica per un'entità `Persona` di tutti gli attributi di interesse come il codice fiscale, il cognome, l'età, il sesso ecc.;
- la reificazione di un attributo o di un'entità; ad esempio se un'entità `Impiegato` contiene come attributo il nome dell'azienda, quest'ultima viene rappresentata come entità e sussiste una relazione tra le due;
- la decomposizione di una relazione in due relazioni distinte, come l'esempio descritto prima;
- la trasformazione di un'entità in una gerarchia di generalizzazione; come per l'anagrifico di un'azienda, dove può esistere un anagrafico storico e uno corrente.

Il vantaggio della strategia top-down è che il progettista può descrivere inizialmente tutte le specifiche dei dati trascurandone i dettagli, per poi entrare nel merito di un concetto alla volta. Questo, però, è possibile solo quando si possiede una visione globaile e astratta di *tutte* le componenti del sistema, che è una situazione rara quando si ha a che fare con problemi di una certa complessità.

7.3.2 Strategia bottom-up

In questa strategia, le specifiche iniziali sono suddivise in componenti via via sempre più piccole, fino a quando queste componenti descrivono un frammento elementare della realtà di interesse. A questo punto, le varie componenti vengono rappresentante da semplici schemi concettuali che possono consistere anche in singoli concetti. I vari schemi ottenuti vengono poi fusi fino a giungere, attraverso una completa integrazione di tutte le componenti, allo schema concettuale finale. Anche in questo caso lo schema finale si ottiene attraverso alcune trasformazioni elementari che vengono denominate *primite di trasformazione bottom-up* che introducuno in uno schema nuovi concetti non presenti precedentemente e in grado di descrivere aspetti della realtà di interesse che non erano ancora stati rappresentati.

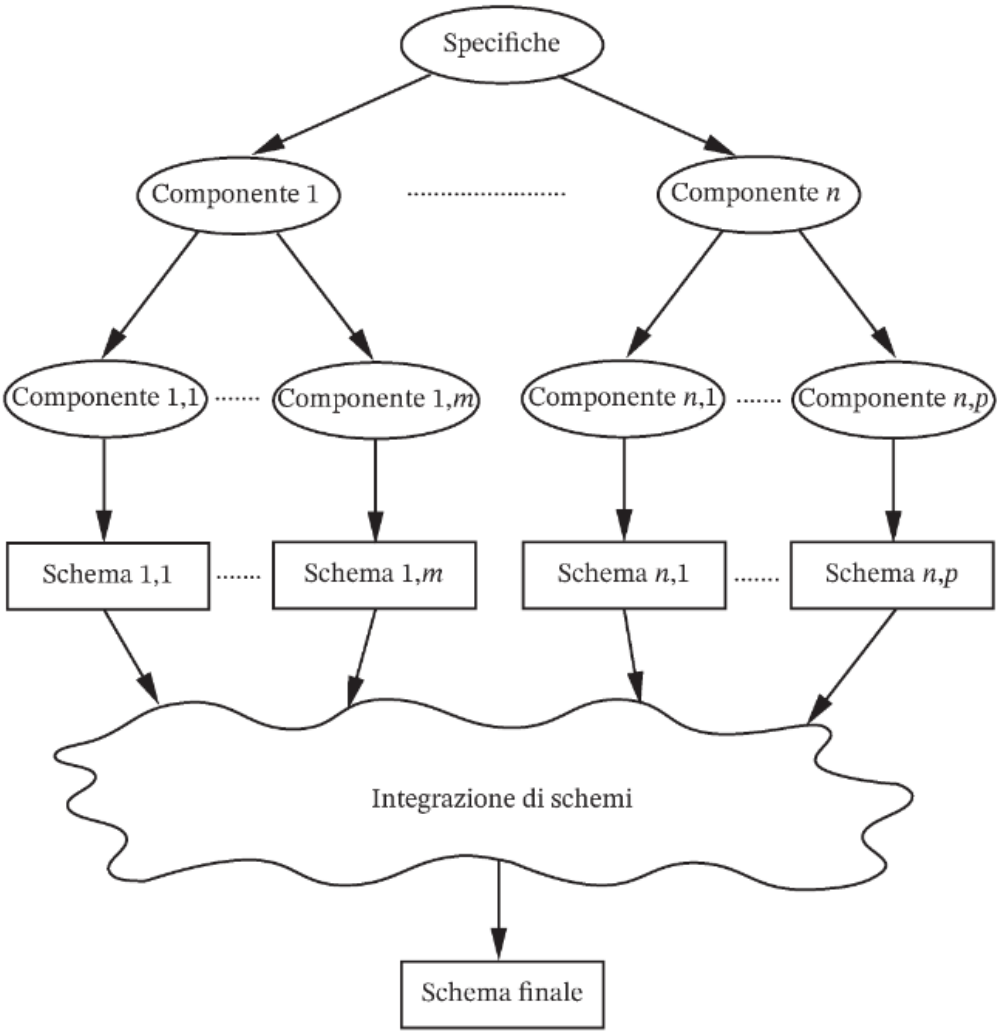


Figura 7.2: La strategia bottom-up.

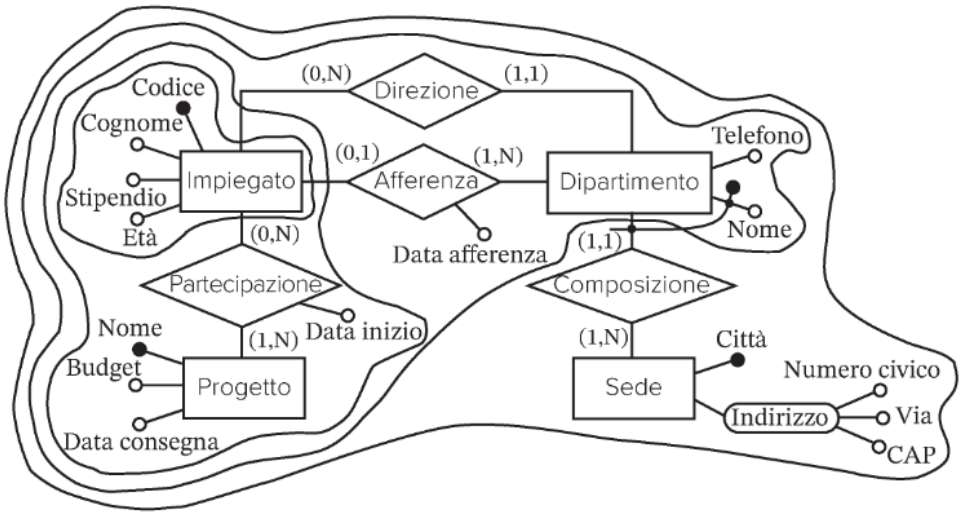
Esempi di primitive di trasformazione bottom-up sono:

- l'introduzione di una nuova entità o di una relazione dell'analisi delle specifiche;
- l'individuazione nelle specifiche di un legame tra diverse entità riconducibile a una generalizzazione;
- l'aggregazione di una serie di attributi in un'entità o in una relazione.

Il vantaggio della strategia bottom-up è che si adatta a una decomposizione del problema in componenti più semplici, facilmente individuabili, il cui progetto può essere affrontato anche da progettisti diversi. Lo svantaggio di questa strategia è invece i fatto che richiede delle operazioni di integrazione di schemi concettuali diversi che, nel caso di schemi complessi, presentano quasi sempre grosse difficoltà.

7.3.3 Strategia inside-out

Questa strategia può essere vista come un caso particolare della strategia bottom-up. Si individuano inizialmente solo alcuni concetti importanti e poi si procede, a partire da questi, a "macchia d'olio". Si rappresentano cioè prima i concetti in relazione con i concetti iniziali, per poi muoversi verso quelli più lontano attraverso una "navigazione" tra le specifiche.
Di seguito viene mostrato un esempio di applicazione della strategia inside-out:



Si può osservare che è stata individuata inizialmente l'entità `Impiegato` con i suoi attributi. A partire da questa sono state rapprsentate la partecipazione degli impiegati ai progetti e tutte le proprietà dei progetti. Successivamente, sono state analizzate le correlazioni esistenti tra gli impiegati e i dipartimenti dell'azienda, individuando le relazioni `Direzione` e `Afferenza` e l'entità `Dipartimento` con i relativi attributi. Infine, partendo da quest'ultima, sono state rappresentati le sedi dell'azienda e l'appartenenza dei dipartimenti alle relative sedi.

Questa strategia ha il vantaggio di non richiedere passi di integrazione. D'altro canto è necessario, di volta in volta, esaminare tutte le specifiche per individuare concetti non ancora rappresentati e descrivere nuovi concetti nel dettaglio. Non è quindi possibile procedere per livelli di astrazione.

7.3.4 Strategia mista

La strategia mista cerca di combinare i vantaggi della strategia top-down con quelli della strategia bottom-up. Il progettista suddivide i requisiti in compoenenti separate, come nella strategia bottom-up, ma allo stesso tempo definisce uno *schema scheletro* contenente, a livello astratto, i concetti principali dell'applicazione. Questo schema scheletro fornisce una visione unitaria, sia pure astratta, dell'intero progetto e favorisce le fasi di integrazione degli schemi sviluppati separatamente.
La strategia mista è probabilmente la più flessibile tra le strategia viste perchè si adatta bene a esigenze contrapposte: quella di suddivdere un problema complesso in sottoproblemi e qualla di procedere per raffinamenti successivi. Questa strategia ingloba anche la strategia inside-out che è solo un caso particolare della strategia bottom-up.
Inoltre, in quasi tutti i casi pratici di una certa complessità, la strategia mista è l'unica che si può effettivamente adottare perchè è spesso necessario cominciare la progettazione quando non sono ancora disponibili tutti i dati e, dei dati noti, abbiamo delle conoscenze a livello di dettaglio non omogenei.

7.4 Qualità di uno schema concettuale

Nella costruzione di uno schema concettuale vanno garantite alcune proprietà generali che uno schema concettuale di buona qualità deve possedere. Analizziamo le qualità più importanti e vediamo come è possibile verificare, durante la progettazione concettuale, queste qualità.

- **Correttezza**. Uno schema concettuale è *corretto* quando utilizza propriamente i costrutti messi a disposizione dal modello concettuale di riferimento. Gli errori possono essere *sintattici* o *semantici*. Gli errori sintattici riguardano un uso non ammesso di costrutti, come una generalizzazione tra relazioni invece che tra entità. Gli errori semantici riguardano un uso di costrutti che non rispetta la loro definizione. Ad esempio, l'uso di una relazione per descrivere il fatto che un'entità è specializzazione di un'altra.

La correttezza di uno schema si può verificare per ispezione, confrantando i concetti presenti nello schema in via di costruzione con le specifiche e con le definizioni dei costrutti del modello concettuale usato.

- **Completezza**. Uno schema concettuale è *completo* quando rappresenta tutti i dati di interesse e quando tutte le operazioni possono essere eseguite a partire dai concetti descritti nello schema. La completezza di uno schema si può descrivere controllando che tutte le specifiche sui dati siano rappresentate da qualche concetto presente nello schema in costruzione, e che tutti i concetti coinvolti in un'operazione presente nelle specifiche siano raggiungibili "navigando" attraverso lo schema.

- **Leggibilità**. Uno schema concettuale è *leggibile* quando rappresenta i requisiti in maniera naturale e facilmente comprensibile. Per garantire questa proprietà è necessario rendere lo schema autoesplicativo, per esempio, mediante una scelta opportuna dei nomi da dare ai concetti. La leggibilità dipende anche da criteri puramente estetici: la compresione di uno schema è facilitata anche se tracciamo il relativo diagramma su una griglia nella quale i vari costrutti hanno stesse dimensioni. Alcuni suggerimenti per rendere lo schema più leggibile sono:

  - disporre i costrutti su una griglia scegliendo come elementi centrali quelli con più relazioni con altri;
  - tracciare solo linee perpendicolari e cercare di minimizzare le interesezioni;
  - disporre le entità che sono genitori di generalizzazioni sopra le relative entità figlie.

- **Minimalità**. Uno schema è *minimale* quando tutte le specifiche sui dati sono rappresentate una sola volta nello schema. Uno schema, quindi, non è minimale quando esistono delle *ridondanze*, ovvero concetti che possono essere derivati da altri. La minimalità di uno schema si può verificare per ispezione, contrallando se esistono concetti che possono essere eliminati dallo schema che stiamo costruendo senza inficiare la sua completezza. Per quanto detto, si deve prestare particolare attenzione ai cicli presenti nello schema.

7.5 Una metodologia generale

Per quanto riguarda le strategia di progetto va precisato che, in pratica, non accade quasi mai che un progetto proceda *sempre* in maniera top-down o bottom-up. Indipendentemente dalla strategia scelta, nelle sitauzioni reali capita di modificare lo schema in via di costruzione sia con trasformazioni che raffinano un concetto presente, e quindi top-down, sia con trasformazioni che aggiungono un concetto non presente, e quindi bottom-up. Presentiamo quindi una metodologia per la progettazione concettuale con il modello E-R con riferimento alla strategia mista. La metodologia è composta dai seguenti passi.

1. **Analisi dei requisiti**
    1.1. Costruire un glossario dei termini.
    1.2. Analizzare i requisiti ed eliminare le ambiguità presenti.
    1.3. Raggruppare i requisiti in insiemi omogenei.
2. **Passo base**
    2.1. Individuare i concetti più rilevanti e rappresentarli in uno schema scheletro.
3. **Passo di decomposizione**, da effettuare se necessario.
    3.1. Effettuare una decomposizione dei requisiti con riferimento ai concetti presenti nello schema scheletro.
4. **Passo iterativo** da ripetere, per tutti i sottoschemi, se presenti, finchè ogni specifica è stata rappresentata.
    4.1. Raffinare i concetti presenti sulla base delle loro specifiche.
    4.2. Aggiungere nuovi concetti allo schema per descrivere specifiche non ancora descritte.
5. **Passo di integrazione**, da effettuare se è stato eseguito il passo 3.
    5.1. Verificare la correttezza dello schema ed eventualmente ristrutturare lo schema.
    5.2. Verificare la completezza dello schema ed eventualmente ristrutturare lo schema.

5.3. Verificare la minimalità, documentare le ridondanze ed eventualemente ristrutturare lo schema.

5.4. Verificare la leggibilità dello schema ed eventualmente ristrutturare lo schema.

Capitolo 8

Progettazione logica

8.1 Fasi della progettazione logica

Le attività principali della progettazione logica sono la riorganizzazione dello schema concettuale e la traduzione in un modello logico. La riorganizzazione può essere discussa indipendentemente dal modello logico ed è utile articolare la progettazione logica in due fasi:

1. **Ristrutturazione dello schema E-R**: è una fase indipendente dal modello logico scelto e si basa su criteri di ottimizzazione dello schema e di semplificazione della fase successiva.

2. **Traduzione verso il modello logico**: fa riferimento a uno specifico modello logico e può includere un'ulteriore ottimizzazione che si basa sulle caratteristiche del modello logico stesso.

I dati di ingresso della prima fase sono le schame concettuale prodotto e il *carico applicativo*.

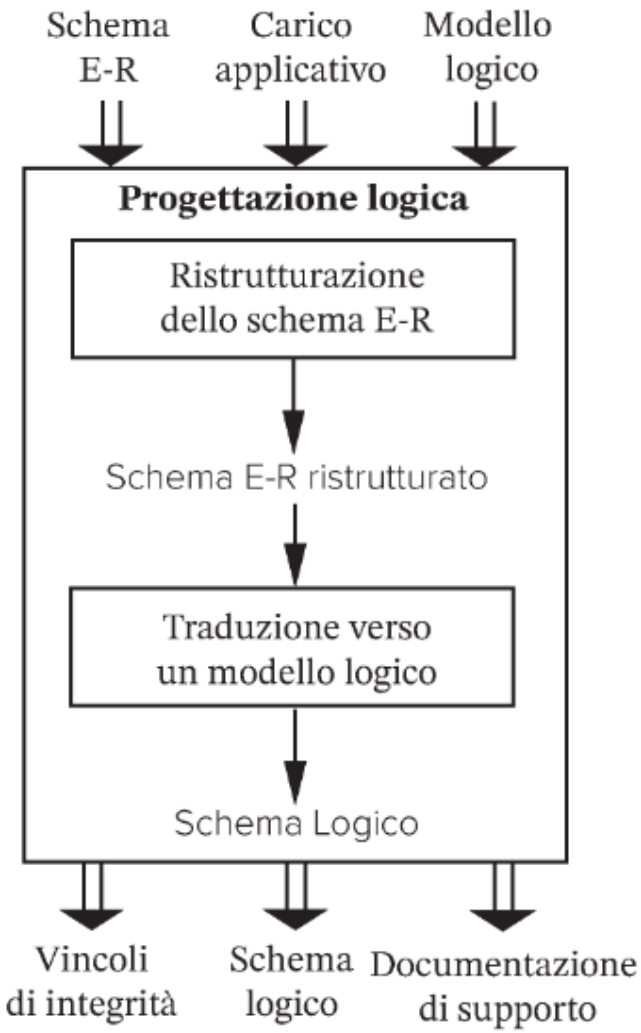


Figura 8.1: Progettazione logica di basi di dati

8.2 Analisi della prestazioni su schemi E-R

Uno schema E-R può essere modificato per ottimizzare alcuni *indici di prestazione* del progetto. È bene notare, però, che le prestazioni di una base di dati non sono valutabili durante la fase di progettazione logica, indipendenti da dettagli fisici, ed è questo il motivo per il quale si parla di indici di prestazione.
Tuttavia, ci sono due parametri che regolano le prestazioni dei sistemi software:

1. **costo di un'operazione**: viene valutato in termini di numero di occorrenze di entità e associazioni che vanno visitate per rispondere a un'operazione sulla base di dati.

2. **occupazione di memoria**: viene valutato in termini dello spazio di memoria necessario per memorizzare i dati descritti dallo schema.

Per lo studio di questi parametri, è necessario conoscere:

- **Volume dei dati**. Ovvero il numero di occorrenze di ogni entità e associazione nello schema e le dimensioni di ciascun attributo.
- **Caratteristiche delle operazioni**. Ovvero:
  - tipo dell'operazione, interattiva o *batch*;
  - frequenza, numero medio di esecuzioni in un certo intervallo di tempo;
  - dati coinvolti, entità e/o associazioni.

Consideriamo il seguente schema E-R:

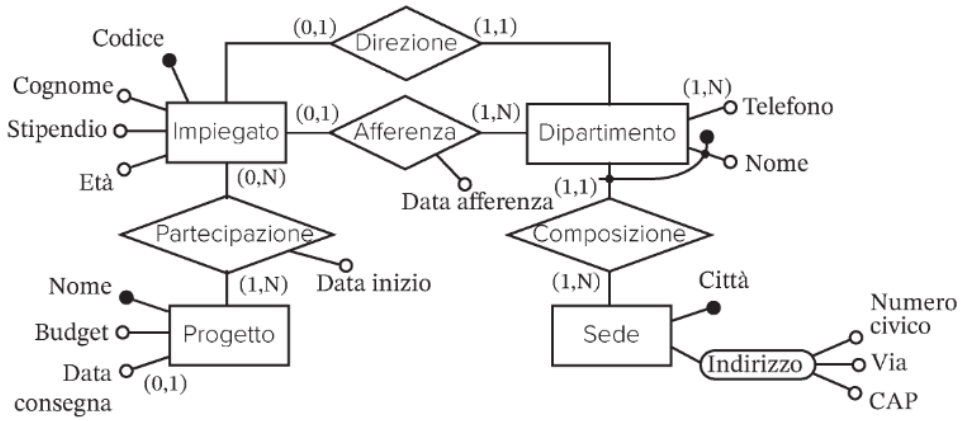


Figura 8.2: Uno schema E-R sul personale di un'azienda

Le operazioni possibili su questa base di dati potrebbero essere:

- Assegna un impiegato a un progetto;

- Trova i dati di un impiegati, del dipartimento nel quale lavora e dei progetti ai quali partecipa;
- trova i dati di tutti gli impiegati di un certo dipartimento;
- per ogni sede, trova i suoi dipartimenti con il cognome del direttore e l'elenco degli impiegati del dipartimento.

Notiamo che le operazioni sulla base di dati seguono la regola "80-20", ovvero l'80% del carico è generato dal 20% delle operazioni e questo ci permette di valutare il carico concentrandoci solo sulle operazioni principali previste.
Il volume dei dati e le caratteristiche generali delle operazioni possono essere descritte facendo uso della *tavola dei volumi* e della *tavola delle operazioni* mostrate in figura:

Tavola dei volumi

Concetto	Tipo	Volume
Sede	E	10
Dipartimento	E	80
Impiegato	E	2000
Progetto	E	500
Composizione	R	80
Afferenza	R	1900
Direzione	R	80
Partecipazione	R	6000

Tavola delle operazioni

Operazione	Tipo	Frequenza
Op. 1	I	50 al giorno
Op. 2	I	100 al giorno
Op. 3	I	10 al giorno
Op. 4	B	2 a settimana

Nella tavola dei volumi vengono riportati tutti i concetti dello schema, le entità e assocazioni, con il volume previsto. Nella tavole delle operazioni vengono riportati, per ogni operazione, la frequenza prevista e un simbolo che indica se l'operazione è interattiva (I) o batch (B).
Nella tavola dei volumi, il numero di occorrenze delle associazioni dipende da due parametri: il numero di occorrenze delle entità coinvolte nelle assocazioni e il numero medio di partecipazioni di un'occorrenza di entità alle occorrenze di associazioni. Ad esempio, il numero di occorrenze dell'associazione `Composizione` è pari al numero dei dipartimenti, perchè le cardinalità dicono che un dipartimento appartiene a una sola sede. Il numero di occorrenze dell'associazione `Afferenza` è poco meno del numero degli impiegati, perchè le cardinalità dicono che ci sono impiegati che non afferiscono a nessun dipartimento.

Infine, se assumiamo che ciascun impiegati partecipa in media a 3 progetti, abbiamo $2000 \times 3 = 6000$ occorrenze per l'associazione `Partecipazione`.
Inoltre, per ogni operazione possiamo descrivere graficamente i dati coinvolti con uno *schema di operaazione*, che consiste nel frammento dello schema E-R interessato dall'operazione sul quale viene disegnato il "cammino logico" da percorrere per accedere alle informazioni di interesse.

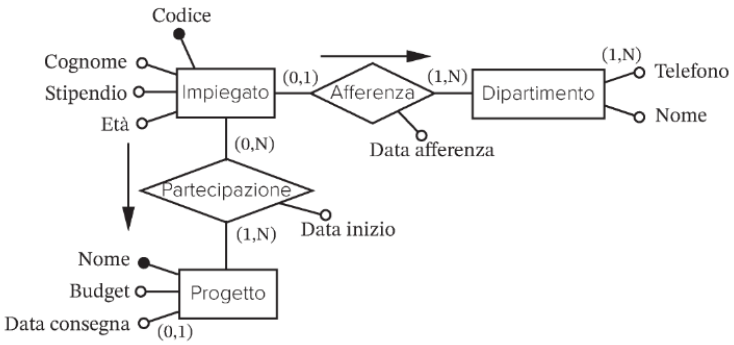


Figura 8.3: Uno schema di operazione

Per ottenere le informazioni di interesse su un impiegato si parte dall'entità `Impiegato` per accedere, attraverso l'associazione `Afferenza`, al suo dipartimento e, attraverso l'associazione `Partecipazione`, ai progetti ai quali partecipa.
Con queste informazioni è possibile fare una stima del costo di un'operazione contando il numero di accessi alle occorrenze di entità e associazioni necessario per eseguire l'operazione. Il meccanismo è basato sulle seguenti regole:

- si contano il numeri di accessi alle occorrenze del primo concetto presente nel cammino logico dello schema di operazione, assumendo che il recupero di un'occorrenza richieda un solo accesso;
- si prosegue nel cammino assumendo che, avendo recuperato nei passi precedenti un'occorrenza, possiamo accedere tramite essa a tutte le occorrenze di altri concetti che coinvolgono questa occorrenza.

Tornando all'operazione precedente in figura, dobbiamo accedere a un'occorrenza dell'entità `Impiegato` per poi accedere a un'occorrenza dell'entità `Afferenza` e, attraverso questa, a un'occorrenza dell'entità `Dipartimento`. Dopodichè, per

Concetto	Costrutto	Accessi	Tipo
Impiegato	Entità	1	L
Afferenza	Relazione	1	L
Dipartimento	Entità	1	L
Partecipazione	Relazione	3	L
Progetto	Entità	3	L

conoscere i dati dei progetti ai quali lavora, dobbiamo accedere a tre occorrenze dell'assocazione `Partecipazione` e, attraverso queste, a tre occorrenza dell'entità `Partecipazione`. Possiamo riasssumere questo in una *tavola degli accessi* riportata in figura: In questa tavola, viene anche riportato il tipo di accesso, se è in lettura L o in scrittura S. Questa distinzione è importante perchè, generalmente, le operazioni di scrittura sono più onerose di quelle di lettera.

8.3 Ristrutturazione di schemi E-R

La fase di ristrutturazione di uno schema E-R si può suddividere in una serie di passi sequenziali:

1. **Analisi delle ridondanze**. Si decide se eliminare o mantenere eventuali rindondanze presenti nello schema;

2. **Eliminazioni delle generalizzazioni**. Tutte le generalizzazioni presenti nello schema vengono analizzate e sostituite da altri costrutti;

3. **Partizionamento/accorpamento di entità e associazioni**. Si decide se è opportuno partizionare concetti dello schema in più concetti, o, viceversa, accorpare concetti separati in un unico concetto;

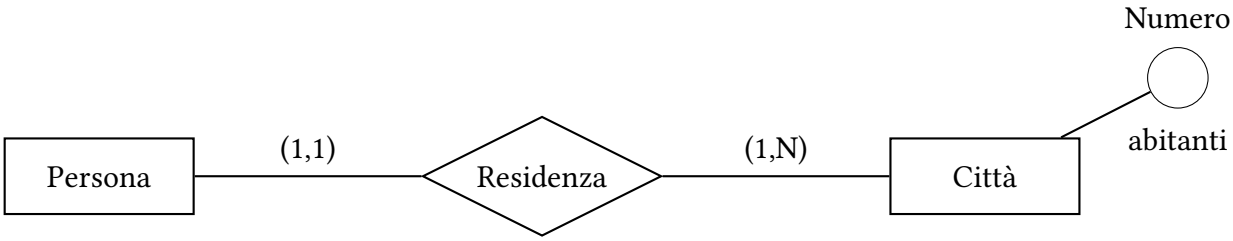
4. **Scelta degli identificatori principali**. Si seleziona un identificatore per quelle entità che ne hanno più di uno.

8.3.1 Analisi delle ridondanze

Una ridondanza in uno schema concettuale corrisponde alla presenza di un dato che può essere derivato, cioè ottenuto attraverso una serie di operazioni, da altri dati. Ci possono essere diversi casi di ridondanza in uno schema E-R:

- Attributi derivabili, occorrenza per occorenza, da altri attributi della stessa entità.
- Attributi derivabili da attributi di altre entità o associazioni, di solito attraverso funzioni aggregative.
- Attributi derivabili da operazioni di conteggio di occorrenze.
- Assocazioni derivabili dalla composizione di altre associazioni in presenza di cicli.

La presenza di un dato derivato presenta un vantaggio e alcuni svantaggi. Il vantaggio è una riduzione degli accessi necessari per calcolare il dato dderivato, gli svantaggi sono una maggiore occupazione di memoria (che è uno svantaggio trascurabile) e la necessità di effettuare operazioni aggiuntive per mantenere il dato derivato aggiornato.
La decisione di mantenere o eliminare una ridondanza va presa confrontando costo di esecuzione delle operaioni che coinvolgono il dato ridondante e relativa occupazione di memoria, nei casi di presenza e assenza della ridondanza.
Consideriamo il seguente schema:



Supponiamo che per questo schema vengono effettuate due operazioni principali:

1. Memorizza una nuova persona con la relativa città di residenza;
2. Stampa tutti i dati di una città, incluso il numero di abitanti.

Proviamo a valutare gli indici di prestazione in caso di presenza del dato ridondante *Numero abitante*.
La tavola dei volumi e delle operazioni per lo schema rappresentato sono le seguenti:

Tavola dei volumi

Concetto	Tipo	Volume
Città	E	200
Persona	E	1 000 000
Residenza	R	1 000 000

Tavola delle operazioni

Operazione	Tipo	Frequenza
Op. 1	I	500 al giorno
Op. 2	I	2 al giorno

Assumendo che il numero di abitanti di una città richieda 4 byte, abbiamo che il dato ridondante richiede $4 \times 200 = 800$ byte, ovvero meno di 1 kilobyte di memoria aggiuntiva.
L'operazione 1 richiede un accesso in scrittura all'entità `Persona`, un accesso in scrittura all'entità `Residenza` e infine un accesso in lettura, per cercare la città di interesse, e uno in scrittura, per incrementare il numero di abitanti di quell'occorrenza all'entità `Città`, il tutto ripetuto per 500 volte al giorno, per un totale di 1500 accessi in scrittura e 500 accessi in lettura. Il costo dell'operazione 2 è ttrascurabile perchè richiede un solo accesso in lettura all'entità `Città` da ripetere due volte al giorno. Quindi, contanto che gli accessi in scrittura abbiano un costo doppio rispetto a quelli in lettura, abbiamo un totale di 3500 accessi al girono in caso di presenza di dato ridondante.
Se il dato ridondante è assente, per l'operazione 1 abbiamo bisogno di un accesso in scrittura all'entità `Persona` e un accesso in scrittura all'associazione `Residenza`, per un totale di 1000 accessi in scrittura al girono. Per l'operazione 2, invece, abbiamo bisogno di un accesso in lettura all'entità `Città`, che possiamo trascurare, e di 5000 accessi in lettura all'assocazione `Residenza` in media per calcolare il numero di abitanti di questa città, per un totale di 10000 accessi in lettura al giorno. Contando doppi gli accessi in scrittura abbiamo un totale di 12000 accessi al giorno in assenza del dato ridondante. Quindi abbiamo circa 8500 accessi in più rispetto al caso di dato ridondante contro un risparmi di un solo kilobyte. Possiamo concludere, quindi, che conviene in questo caso mantenere il dato ridondante.

8.3.2 Eliminazione delle generalizzazioni

I sistemi tradizionali per la gestione delle basi di dati non consentono di rappresentare direttamente una generalizzazione, risulta necessario trasformare questo costrutto in altri costrutti del modello E-R per i quali esiste un'implementazione naturale: le entità e le associazioni.
Per rappresentare una generalizzazione mediante entità e associazioni abbiamo tre alternative disponibili.
Supponiamo di avere un'entità E_0 che ha due entità figlie E_1 ed E_2.

1. **Accorpamento delle figlie della generalizzazione nel genitore**. Le entità E_1 ed E_2 vengono eliminate e le loro proprietà, attributi ed eventuali partecipazioni ad associazioni e generalizzazioni, genono aggiunte all'entità genitore E_0. A tale entità viene aggiunto un ulteriore attributo che serve a distiguere il "tipo" di un'occorrenza di E_0, cioè se apparteneva a E_1, a E_2 o nel caso di generalizzazione non totale, a nessuna di esse.

2. **Accorpamento del genitore della generalizzazione nelle figlie**. L'entità genitore E_0 viene eliminata, e, per la proprietà dell'ereditarietà, i suoi attributi, il suo identificatore e le relazioni a cui tale entità partecipava, vengono aggiunti alle entità figlie E_1 ed E_2.

3. **Sostituzione della generalizzazione con assocazioni**. La generalizzazione si traforma in due associazioni uno a uno che legano rispettivamente l'entità genitore con le entità figlie E_1 ed E_2. Non ci sono trasferimenti di attributi o associazioni e le entità E_1 ed E_2 sono identificate esternamente dall'entità E_0. Nello schema che si ottiene, però, vanno aggiunti dei vincoli: ogni occorrenza di E_0 non può partecipare contemporaneamente a R_{G1} ed R_{G2}: inoltre, se la generalizzazione è totale, ogni occorrenza di E_0 deve partecipare o a un'occorrenza di R_{G1} o ad una di R_{G2}.

La scelta tra le varie alternative può essere fatta considerando vantaggi e svantaggi di ognuna delle scelte possibili relativamente all'occupazione di memoria e al costo delle operazioni coinvolte. Ci sono comunque delle regole di carattere generale.

- La prima alternativa è conveniente quando le operazioni non fanno molta distinzione tra le occorrenze e tra gli attributi di E_0, E_1 ed E_2. In questo caso, anche se abbiamo uno spreco di memoria per la presenza di valori nulli, la scelta ci assicura in minore numero di accesso rispetto alle altre dove le occorrenze e gli attributi sono distribuiti tra le varie entità.

- La seconda alternativa è possibile solo se la generalizzazione è totale, altrimenti le occorrenze di E_0 che non sono occorrenze né di E_1 né di E_2 non sarebbero rappresentate. È conveniente quando ci sono operazioni che si riferiscono solo a occorrenze di E_1 oppure di E_2 e dunque fanno delle distinzioni tra tali entità. In questo caso abbiamo un risparmio di memoria rispetto alla prima alternativa in quanto gli attributi non assumono mai valori nulli e c'è una riduzione degli accessi rispetto alla terza alternativa, perchè non si deve visitare E_0 per accedere ad alcuni attributi di E_1 o E_2.

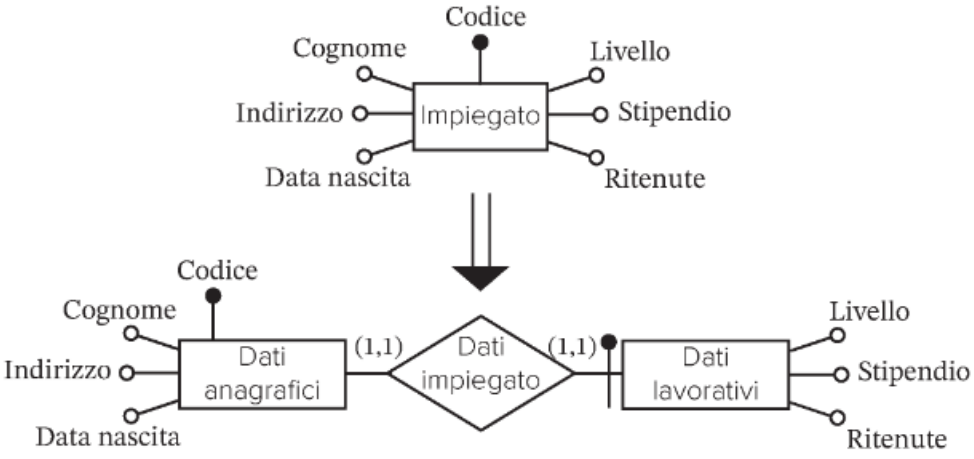
- La terza alternativa è conveniente quando la generalizzazione non è totale, ma non è un vincolo necessario, e ci sono operazioni che si riferiscono solo a occorrenze di E_1 $\left(E_2\right)$ oppure di E_0, e dunque fanno distinzioni tra entità figlia ed entità genitore. In questo caso c'è un risparmio di memoria rispetto alla prima alternativa per l'assenza di valori nulli, ma c'è un incremento degli accessi per mantenere la consistenza delle occorrenze rispetto ai vincoli introdotti.

La ristrutturazione delle generalizzazioni è un tipico caso per il quale il conteggio delle istanze e degli accessi non è sempre sufficiente per scegliere la migliore alternativa possibile. Infatti da quanto detto, la terza alternativa sembra non convenga quasi mai perchè richiede un numero di accessi superiore ma quest'alternativa ha il grosso vantaggio di generare entità con pochi attributi e questo si traduce, a livello pratico, in strutture logiche di poccole dimensioni per le quali un accesso fisico permette di recuparare molti dati in una volta sola. Quindi, in alcuni casi va effettuata un'analisi più accurata che tiene conto di altri fattori come le dimensioni dei domini degli attributi e la quantità di dati che è possibile recuperare con una sola operazione di accesso a memoria secondaria.
Le alternative non sono le uniche ammesse, ma è possibile effettuare ristrutturazioni che sono combinazioni delle tre trasformazioni presentate.

8.3.3 Partizionamento/accorpamento di concetti

Entità e associazioni in uno schema E-R possono essere partizionati o accorpati per garantire una maggior efficienza delle operazioni in base al seguente principio: gli accessi si riducono separando attributi di uno stesso concetto che vengono acceduti da operazioni diverse e raggruppando attributi di concetti diverse che vengono acceduti dalle medesime operazioni.

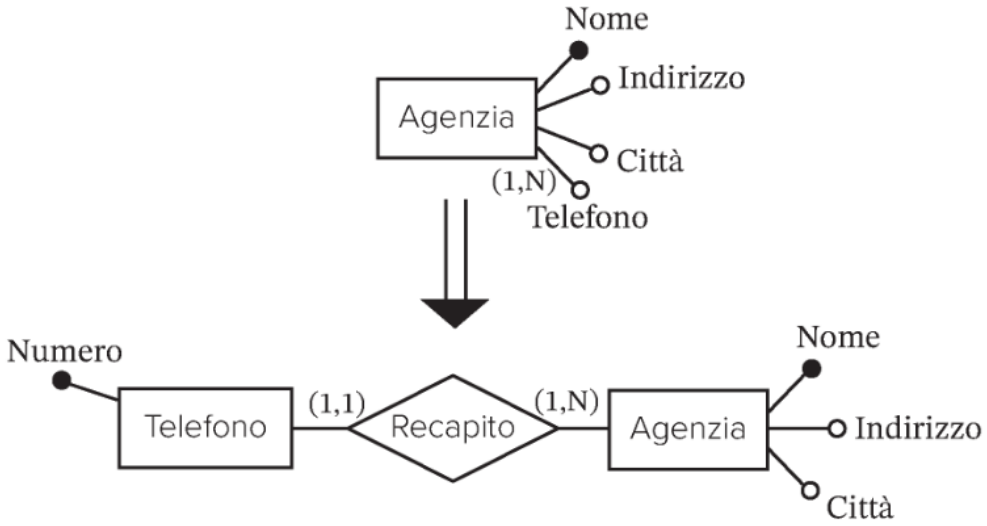
Partizionamenti di entità Consideriamo il seguente esempio:



Notiamo che l'entità `Impiegato` viene sostituita da due entità, collegate da un'associazione uno a uno, che descrivono i dati anagrafici degli impiegati e i dati relativi alla loro retribuzione. Questa ristrutturazione è conveniente se le operazioni riguardanti un impiegato richiedono solo informazioni di carattere anagrafico o solo di carattere lavorativo.
Un partizionamento di questo genere prende il nome di *decomposizione verticale* di un'entità, nel senso che si suddivide il concetto operando sui suoi attributi. È possibile effettuare anche delle *decomposizioni orizzontali* nelle quali la suddivisione avviene sulle occorrenze dell'entità. Ad esempio, per l'entità `Impiegato` ci potrebbero essere alcune operazioni che riguardano soltanto gli analisti e altre che rigurdano solo il venditore.
I partizionamenti orizzontali hanno un effetto collaterale: quello di dover duplicare tutte le associazioni a cui l'entità originaria partecipa. Questo può avere ripercussioni sulle prestazioni del sistema. I partizionamenti verticali generano entità con pochi attributi che possono essere tradotte in strutture logiche sulle quali, con un accesso, è possibile recuperare molti dati. Anche in questo caso, il semplice conteggio degli accessi e delle occorrenze non è sempre sufficiente per scegliere la miglior alternativa possibile.

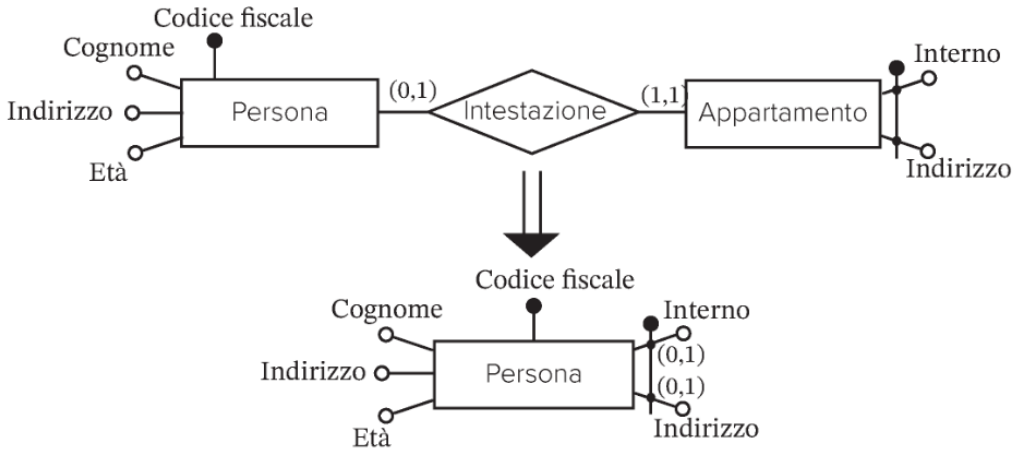
Eliminazione di attributi multivalore Un particolare tipo di partizionamento è quello che riguarda l'eliminazione di attributi multivalore. Questa ristrutturazione è necessaria perchè il modello relazionale non permette di rappresentare

in maniera diretta questo tipo di attributo.
Consideriamo l'esempio in figura:



L'attributo *Telefono* dell'entità `Agenzia` viene partizionata in due entità: una con lo stesso nome e gli stessi attributi dell'entità originale escluso l'attributo multivalore, e l'entità `Telefono`, con il solo attributo *Numero*, legata mediante un'associazione uno a molti con l'entità `Agenzia`.

Accorpamento di entità L'accorpamento è l'operazione inversa del partizionamento. Consideriamo l'esempio in figura:



L'entità `Persona` e l'entità `Appartamento`, legate dall'associazione uno a uno `Intestazione`, vengono accorpate in un'unica entità contenente gli attributi di entrambi. Questa ristrutturazione è utile se le operazioni relative alla persona coinvolgono sempre i dati relativi all'appartamento che occupa, e vogliamo risparmiare gli accessi necessari per risalire a questi dati attraverso l'associazione che li lega. Un difetto di questa ristrutturazione è la possibile presenza di valori nulli dovuta al fatto che le cardinalità ci dicono che ci sono persone che non sono intestatarie di nessun appartamento, e quindi non esistono valori per attributi *Indirizzo* e *Interno*.
Gli accorpamenti, in genere, si effettuano su associazioni uno a uno, raramente su associazioni uno a molti e quasi mai su associazioni molti a molti in quanto si potrebbero generare delle ridondanze.

Altri tipi di partizionamento/accorpamento Le tecniche viste per il partizionamento e accorpamento di entità può essere esteso alle associazioni. In alcuni casi, conviene decomporre un'associazione tra due entità in due o più associazioni tra le stesse entità, per separare occorrenze dell'associazione originale accedute sempre separatamente e, viceversa, accorpare due o più associazioni tra le stesse entità in un'unica associazione, quando le relative occorrenze vengono sempre accedute contemporaneamente.

8.3.4 Scelta degli identificatori principali

La scelta degli identificatori principali è essenziale nelle traduzioni verso il modello relazionale perchè in questo modello le chiavi vengono usate per stabilire legami tra dati in relazioni diverse. Inoltre, i DBMS richiedono di specificare una *chiave primaria* sulla quale vengono costruite automaticamente delle strutture ausiliarie, dette *indici*, per il reperimento efficiente di dati. Quindi, se esistono entità per le quali sono stati specificati più identificatori, bisogna decidere quali di questi identificatori verrà utilizzato come chiave primaria.
Ci sono diversi criteri per la scelta della chiave primaria e sono elencati di seguito.

- Gli attributi con valori nulli non possono costituire identificatori principali. Infatti, quest'ultimi non garantiscono l'accesso a tutte le occorrenze dell'entità corrispondente.

- Un identificatore composto da uno o da pochi attributi è da preferire a identificatori costituiti da molti attributi. Questo garantisce che le strutture ausiliarie per accedere ai dati, gli indici, siano di dimensioni ridotte, permette un risparmio di memoria nella realizzazione dei legami logici tra le varie relazioni e facilita le operazioni di join.

- Un identificatore interno con pochi attributi è da preferire a un identificatore esterno, che magari coinvolge diverse entità. I motivi sono gli stessi del punto precedente.

- Un identificatore che viene utilizzato da molte operazioni per accedere alle occorrenze di un'entità è da preferire rispetto agli altri. In questo modo, infatti, tali operazioni possono essere eseguite efficientemente poichè possono trarre vantaggio dagli indici creati automaticamente dai DBMS.

A questo punto, se nessuno degli identificatori candidati soddisfa tali requisiti, è possibile pensare di introdurre un ulteriore attributo all'entità: questo attributo conterrà valori speciali, detti *codici*, generati appositamente per identificare le occorrenze delle entità. È consigliabile tenere traccia degli identificatori non selezionati come principali ma che vengono utilizzati da qualche operazione per accedere ai dati. Per questi identificatori è possibile definire, in fase di progettazione fisica, degli *indici secondari* che consentono l'accesso efficiente ai dati e possono essere usati in alternativa agli indici definiti automaticamente sugli identificatori principali.

8.4 Traduzione verso il modello relazionale

La seconda fase della progettazione logica corrisponde a una traduzione tra modelli di dati diversi: a partire da uno schema E-R ristrutturato si costruisce uno schema logico *equivalente*, in grado cioè di rappresentare le medesime informazioni. Faremo riferimento a una versione semplificata del modello E-R, senza generalizzazioni e attributi multivalore, e nella quale ogni entità ha un solo identificatore.

8.4.1 Entità e associazioni molti a molti

Consideriamo lo schema in figura:



La traduzione nel modello relazionale prevede:

- per ogni entità, una relazione con lo stesso nome avente per attributi i medesimi attributi dell'entità e per chiave il suo identificatore;
- per l'associazione, una relazione con lo stesso nome avente per attributi gli attributi dell'associazione e gli identificatori delle entità coinvolte; tali identificatori formano la chiave della relazione.

Lo schema relazione che si ottiene è quindi il seguente:

`Impiegato`(Matricola, Cognome, Stipendio)
`Progetto`(Codice, Nome, Budget)
`Partecipazione`(Matricola, Codice, DataInizio)

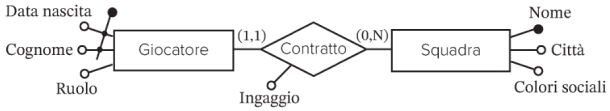
Per rendere più comprensibile il significato dello schema è conveniente effettuare alcune ridenominazioni. Per esempio, si può chiarire il contenuto della relazione `Partecipazione` definendola come segue:

`Partecipazione`(Impiegato, Progetto, DataInizio)

La ridenominazione è essenziale in alcuni casi, come nel caso di associazioni ricorsive.
Per quanto riguarda associazioni con più di due entità partecipanti, queste si traducono in maniera analoga a quanto detto per le associazioni binarie. In quest'ultimo caso bisogna, però, prestare attenzione ad alcuni casi particolari, nei quali l'insieme delle chaivi delle relazioni che rapprsentano le entità coinvolte costituisce in realtà una *superchiave* ridondante della relazione che rappresenta l'associazione dello schema E-R.

8.4.2 Associazioni uno a molti

Consideriamo lo schema in figura:



Secondo la regola vista per le associazioni molti a molti, la traduzione di questo schema dovrebbe essere la seguente:

`Giocatore`(Cognome, DataNascita, Ruolo)
`Squadra`(Nome, Città, ColoriSociali)
`Contratto`(Giocatore, DataNascitaGiocatore, NomeSquadra, Ingaggio)

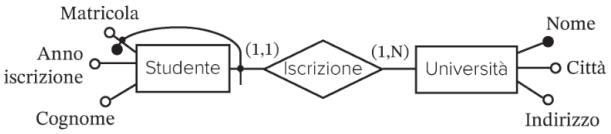
Va notato, che nella relazione `Contratto`, la chiave è costituita solo dall'identificatore di `Giocatore` perchè le cardinalità ci dicono che ogni giocatore ha un contratto con una sola squadra. A questo punto le relazioni `Contratto` e `Giocatore` hanno stessa chiave ed è possibile fonderle in un'unica relazione.

`Giocatore`(Cognome, DataNascita, Ruolo, NomeSquadra, Ingaggio)
`Squadra`(Nome, Città, ColoriSociali)

In questo schema, esiste il vincolo di integrità referenziale tra l'attributo *NomeSquadra* di `Giocatore` e l'attributo *Nome* di `Squadra`. Quindi, l'entità che partecipa all'associazione con cardinalità massima pari a 1, viene tradotta in una relazione che contiene anche gli identificatori delle altre entità coinvolte nell'associazione ed eventualmente gli attributi dell'associazione stessa e non c'è più bisogno di rappresentare l'associazione di partenza.

8.4.3 Entità con identificatore esterno

Le entità con identificatori esterni danno luogo a relazioni con chiavi che includono gli identificatori delle entità "identificanti". Consideriamo lo schema in figura:



Lo schema relazione a quest schema è il seguente:

`Studente`(Matricola, NomeUniversità, Cognome, AnnoIscrizione)
`Università`(Nome, Città, Indirizzo)

nel quale esiste un vincolo di integrità referenziale tra l'attributo *NomeUniversità* della relazione `Studente` e l'attributo *Nome* della relazione `Università`.
Rappresentando l'identificatore esterno si rappresenta direttamente anche l'associazione tra le due entità. Ricordiamo che le entità identificate esternamente partecipano all'associazione sempre con cardinalità massima pari a 1.
Questo tipo di traduzione è valido indipendentemente dalla cardinalità con cui l'altra entità partecipa all'associazione.

8.4.4 Associazioni uno a uno

Per le associazioni uno a uno ci sono, in genere, diverse possibilità di traduzione. Iniziamo con le associazioni uno a uno con partecipazioni obbligatorie per entrambe le entità. Consideriamo lo schema in figura:

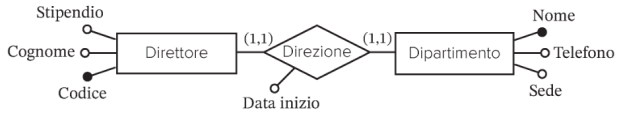


Figura 8.4: Associazione uno a uno con partecipazione obbligatoria delle due entità

Per questo tipo di associazione abbiamo due possibilità ugualmente valide:

`Direttore`(Codice, Cognome, Stipendio, DipartimentoDiretto, InzioDirezione)
`Dipartimento`(Nome, Telefono, Sede)

con vincolo di integrità referenziale tra *DipartimentoDiretto* di `Direttore` e *Nome* di `Dipartimento`. Oppure:

`Direttore`(Codice, Cognome, Stipendio)
`Dipartimento`(Nome, Telefono, Sede, Direttore, InizioDirezione)

con vincolo di integrità referenziale tra *Direttore* di `Dipartimento` e *Codice* di `Direttore`.
È possibile anche un'ulteriore alternativa nella quale si rappresentano tutti i concetti in un'unica relazione contenente tutti gli attributi in gioco. Questa alternativa è, però, da escludere, perchè non dobbiamo dimenticarci che lo schema che stiamo traducendo è il risultato di una fase di ristrutturazione nella quale sono state effettuate precise scelte riguardanti l'accorpamento e/o partizionamento di entità.
Consideriamo il caso di associazione uno a uno con partecipazione opzionale per una sola entità. Consideriamo lo schema in figura:



Figura 8.5: Associazione uno a uno con partecipazione opzionale di una sola entità

In questo caso abbiamo una soluzione preferibile rispetto alle altre:

`Impiegato`(Codice, Cognome, Stipendio)
`Dipartimento`(Nome, Telefono, Sede, Direttore, InizioDirezione)

per la quale esiste il vincolo di integrità referenziale tra l'attributo *Direttore* della relazione `Dipartimento` e l'attributo *Codice* della relazione `Impiegato`. Questa alternativa è preferibile rispetto a quella in cui l'associazione viene rappresentata nella relazione `Impiegato` mediante il nome del dipartimento diretto perchè avremmo, per questo attributo, possibili valori nulli.
Consideriamo il caso in cui entrambe le entità hanno partecipazione opzionale. Consideriamo lo schema in figura e supponiamo che possano esistere dipartimenti senza direttori.



Figura 8.6: Associazione uno a uno con partecipazione opzionale per entrambe le entità

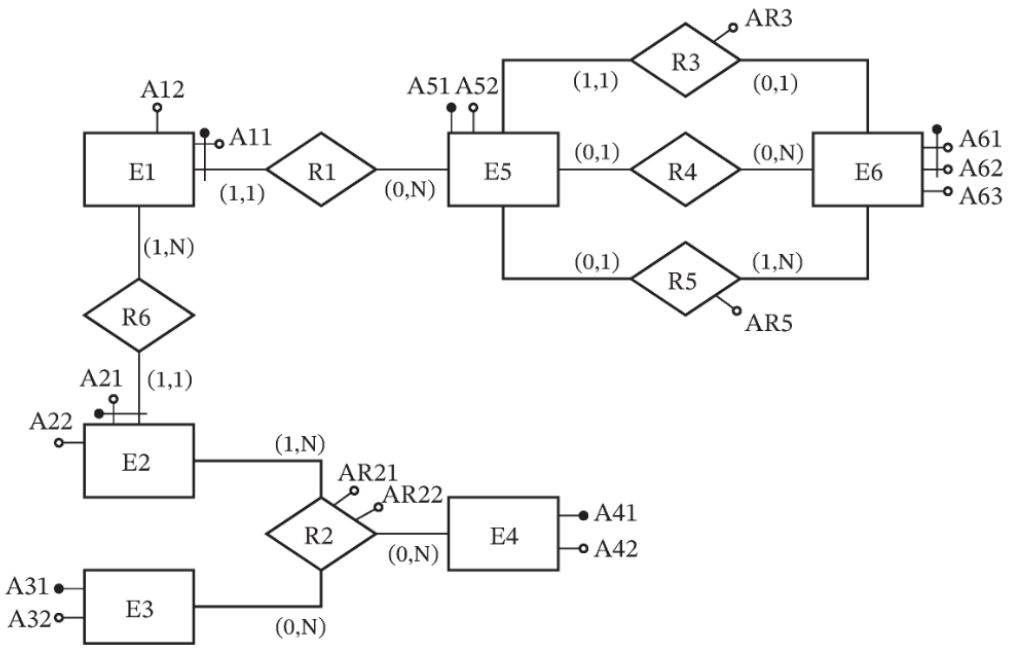
In questo caso esiste un'ulteriore possibilità che prevede tre relazioni separate:

`Impiegato`(Codice, Cognome, Stipendio)
`Dipartimento`(Nome, Telefono, Sede)
`Direzione`(Direttore, Dipartimento, DataInzioDirezione)

Su questo schema abbiamo due vincoli di integrità referenziale: uno tra l'attributo *Direttore* della relazione `Direzione` e l'attributo *Codice* della relazione `Impiegato` e l'altro tra l'attributo *Dipartimento* della relazione `Direzione` e l'attributo *Nome* della relazione `Dipartimento`.
Questa soluzione ha il vantaggio di non presentare mai valori nulli sugli attributi che rappresentano l'associazione. Per contro, abbiamo bisogno di una relazione inn più con un conseguente aumento della complessità della base di dati. La soluzione con tre relazioni è da prendere in considerazione solo se il numero di occorrenze dell'associazione è molto basso ripsetto alle occorrenze delle entità che partecipano all'associazione. In questo caso, si evitano la presenza di molti valori nulli.

8.4.5 Traduzione di schemi complessi

Consideriamo lo schema in figura:



Traduciamo ciascuna entità con una relazione. La traduzione delle entità dotate di identificatore interno è immediata:

$$\text{E3}(\underline{\text{A31}}, \text{A32}) \quad \text{E4}(\underline{\text{A41}}, \text{A42}) \quad \text{E5}(\underline{\text{A51}}, \text{A52})$$

$$\text{E6}(\underline{\text{A61}}, \underline{\text{A62}}, \text{A63})$$

Traduciamo ora le entità con identificazioni esterne. Otteniamo le seguenti relazioni:

$$\text{E1}(\underline{\text{A11}}, \underline{\text{A51}}, \text{A12}) \quad \text{E2}(\underline{\text{A21}}, \underline{\text{A11}}, \underline{\text{A51}}, \text{A22})$$

Passiamo ora alla traduzione delle associazioni. Le traduzioni di R1e R6 sono già state tradotte come conseguenza dell'identificazione esterna di E1 ed E2 rispettivamente. Assumiamo di aver deciso di ottenere un numero minimo di relazioni nello schema finale e cerchiamo di accopare quando possibile. Questo comporta una possibile presenza di valori nulli nelle istanze relazionali per tutti gli attributi introdotti traducendo relazioni dal lato di cardinalità (0, 1). Otteniamo le seguenti modifiche da effettuare allo schema iniziale:

- per tradurre R3 introduciamo con opportune ridenominazioni gli attributi che identificano E6 tra quelli di E5, nonchè l'attributo *AR3* proprio di R3; introduciamo *A61R3*, *A62R3* e *AR3* in E5;

- analogamente per R4, introduciamo *A61R4* e *A62R4* in E5;
- analogamente per R5, introduciamo *A61R5*, *A62R5* e *AR5* in E5.

Infine traduciamo l'unica associazione molti a molti:

R2(<u>A21</u>, <u>A11</u>, <u>A51</u>, <u>A31</u>, <u>A41</u>, AR21, AR22)

Lo schema relazionale ottenuto è il seguente:

E1(<u>A11</u>, <u>A51</u>, A12) E2(<u>A21</u>, <u>A11</u>, <u>A51</u>, A22)

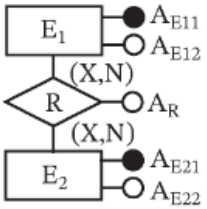
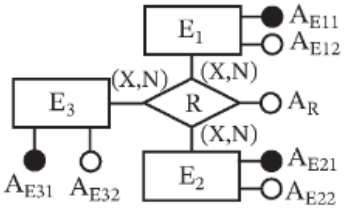
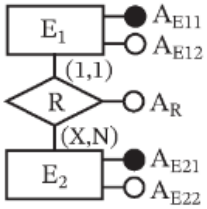
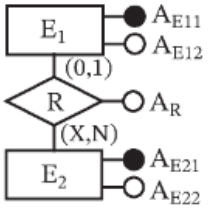
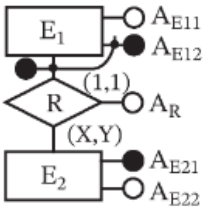
E3(<u>A31</u>, A32) E4(<u>A41</u>, A42)

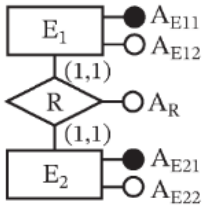
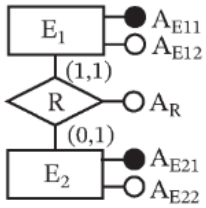
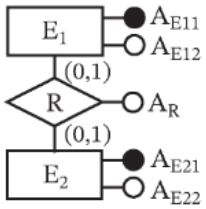
E5(<u>A51</u>, A52, A61R3, A62R3, AR3, A61R4, A62R4, A61R5, A62R5, AR5)

E6(<u>A61</u>, <u>A62</u>, A63) R2(<u>A21</u>, <u>A11</u>, <u>A51</u>, <u>A31</u>, <u>A41</u>, AR21, AR22)

8.4.6 Tabelle riassuntive

Le traduzioni viste vengono riassunte nella seguente tabella. I simboli X e Y indicano una qualunque tra le cardinalità ammesse, gli asterischi indicano la possibilità di avere valori nulli sugli attributi relativi e la sottolineatura tratteggiata indica una chiave alternativa a quella indicata da una sottolineatura piena.

Tipologia	Concetto iniziale	Risultati possibili
Associazione binaria molti a molti		$E_1(\underline{A_{E11}},A_{E12})$ $E_2(\underline{A_{E21}},A_{E22})$ $R(\underline{A_{E11}},\underline{A_{E21}},A_R)$
Associazione ternaria molti a molti		$E_1(\underline{A_{E11}},A_{E12})$ $E_2(\underline{A_{E21}},A_{E22})$ $E_3(\underline{A_{E31}},A_{E32})$ $R(\underline{A_{E11}},\underline{A_{E21}},\underline{A_{E31}},A_R)$
Associazione uno a molti con partecipazione obbligatoria		$E_1(\underline{A_{E11}},A_{E12},\underline{A_{E21}},A_R)$ $E_2(\underline{A_{E21}},A_{E22})$
Associazione uno a molti con partecipazione opzionale		$E_1(\underline{A_{E11}},A_{E12})$ $E_2(\underline{A_{E21}},A_{E22})$ $R(\underline{A_{E11}},\underline{A_{E21}},A_R)$ Oppure: $E_1(\underline{A_{E11}},A_{E12},\underline{A^*_{E21}},A^*_R)$ $E_2(\underline{A_{E21}},A_{E22})$
Associazione con identificatore esterno		$E_1(\underline{A_{E12}},\underline{A_{E21}},A_{E11},A_R)$ $E_2(\underline{A_{E21}},A_{E22})$

Tipologia	Concetto iniziale	Risultati possibili
Associazione uno a uno con partecipazione obbligatoria per entrambe le entità		$E_1(\underline{A_{E11}},A_{E12},\underline{\underline{A_{E21}}},A_R)$ $E_2(\underline{A_{E21}},A_{E22})$ Oppure: $E_2(\underline{A_{E21}},A_{E22},\underline{\underline{A_{E11}}},A_R)$ $E_1(\underline{A_{E11}},A_{E12})$
Associazione uno a uno con partecipazione opzionale per un'entità		$E1(\underline{A_{E11}},A_{E12},\underline{\underline{A_{E21}}},A_R)$ $E_2(\underline{A_{E21}},A_{E22})$
Associazione uno a uno con partecipazione opzionale per entrambe le entità		$E1(\underline{A_{E11}},A_{E12})$ $E_2(\underline{A_{E21}},A_{E22},A^*_{E11},A^*_R)$ Oppure: $E_1(\underline{A_{E11}},A_{E12},A^*_{E21},A^*_R)$ $E_2(\underline{A_{E21}},A_{E22})$ Oppure: $E_1(\underline{A_{E11}},A_{E12})$ $E_2(\underline{A_{E21}},A_{E22})$ $R(\underline{A_{E11}},\underline{\underline{A_{E21}}},A_R)$

Capitolo 9

Normalizzazione

9.1 Ridondanza e anomalie

Consideriamo la seguente relazione:

Impiegato	Stipendio	Progetto	Bilancio	Funzione
Rossi	20 000	Marte	2000	Tecnico
Verdi	35 000	Giove	15 000	Progettista
Verdi	35 000	Venere	15 000	Progettista
Neri	55 000	Venere	15 000	Direttore
Neri	55 000	Giove	15 000	Consulente
Neri	55 000	Marte	2000	Consulente
Mori	48 000	Marte	2000	Direttore
Mori	48 000	Venere	15 000	Progettista
Bianchi	48 000	Venere	15 000	Progettista
Bianchi	48 000	Giove	15 000	Direttore

Questa relazione ha come chiave la coppia di attributi (`Impiegato, Progetto`). Si può inoltre verificare che le tuple della relazione soddisfano le seguenti proprietà:

- lo stipendio di ciascun impiegato è unico ed è funzione del solo impiegato, indipendentemente dei progetti cui partecipa;
- il bilancio di ciascun progetto è unico e dipende dal solo progetto, indipendentemente dagli impiegati che vi partecipano.

Questi fatti hanno conseguenze sul contenuto della relazione e sulle operazioni che si possono effettuare su di essa. Limitiamo le osservazioni alla prima proprietà:

- il valore dello stipendio di ciascun impiegato è ripetuto in tutte le tuple relative a esso: si ha quindi una *ridondanza*;
- se lo stipendio di un impiegato varia, è necessario andarne a modificare il valore in tutte le tuple corrispondenti affinchè la dipendenza continui a valere, questo comporta di effettuare più modifiche contemporaneamente e prende il nome di *anomalia di aggiornamento*;
- se un impiegato interrompe la partecipazione a tutti i progetti senza lasciare l'azienda, e quindi tutte le corrispondenti tuple vegnono eliminate, non è possibile conservare traccia del suo nome e del suo stipendio, che potrebbero rimanere di interesse; questo problema viene indicato come *anomalia di cancellazione*;
- se si hanno informazioni su un nuovo impiegato, non è possibile inserirle finchè questi non viene assegnato a un progetto; in questo caso parliamo di *anomalia di inserimento*.

La presenza di questi inconvenienti è dovuta all'utilizzo di un'unica relazione per rappresentare informazioni eterogenee.
Generalizzando, possiamo arrivare alla seguenti conclusioni, che evidenziano i difetti presentati da relazioni che riuniscono concetti fra loro disomogenei.

- È possibile che alcuni dati debbano essere ripetuti in diverse tuple, senza aggiungere informazioni significative.
- Se alcune informazioni sono ripetute in modo ridondante, il relativo aggiornamento deve essere ripetuto per ciascuna occorrenza dei relativi dati.
- La cancellazione di una tupla può comportare l'eliminazione di tutti i concetti in questione, cioè anche di quelli che conservano la loro validità.
- L'inserimento di informazioni relative a uno solo dei concetti di pertinenza per una relazione non è possibile se non esiste un intero insieme di concetti in grado di costituire una tupla completa.

9.2 Dipendenze funzionali

Per analizzare i concetti introdotti nel paragrafo precedente, è necessario far uso di uno specifico strumento di lavoro: la *dipendenza funzionale*. La dipendenza

funzioanle è un particolare vincolo di integrità per il modello relazionale che descrive legami di tipo funzionale tra gli attributi di una relazione.
Consideriamo la relazione nella figura 9.1 e abbiamo detto che lo stipendio di ciascun impiegato è unico e quindi, ogni volta che in tupla compare un certo impiegato, il valore del suo stipendio rimane sempre lo stesso. Possiamo dire che il valore dell'attributo `Impiegato` determina il valore dell'attributo `Stipendio`, o più precisamente, una funzione che associa a ogni elemento del dominio dell'attributo `Impiegato` un solo elemento dell'attributo `Stipendio`.
Lo stesso discorso si può fare per gli attributi `Progetto` e `Bilancio`, perchè il valore del progetto determina il valore del bilancio del progetto stesso.
Formalizziamo questo concetto. Data una relazione r su uno schema $R\left(X\right)$ e due sottoinsieme di attributi non vuoti Y e Z di X, diremo che esiste su r una dipendenza funzionale tra Y e Z, se per ogni coppia di tuple t_1 e t_2 di r aventi gli stessi valori sugli attributi Y, risulta che t_1 e t_2 hanno gli stessi valori anche sugli attributi Z.
Una dipendenza funzionale tra gli attributi Y e Z viene indicata con $Y \rightarrow Z$ e, come gli altri vincoli di integrità, viene associata a uno schema: una relazione su quello schema verrà considerata corretta se soddisfa tale dipendenza funzionale.
Se l'insieme Z è composto dagli attributi $A_1, A_2, \ldots, A_k$, allora una relazione soddisfa $Y \rightarrow Z$ se e solo se essa soddisfa tutte le k dipendenze $Y \rightarrow A_1, \ldots, Y \rightarrow A_k$.
Una seconda osservazione è la seguente: in base alla definizione data, possiamo notare che, nella nostra relazione, è verificata anche la dipendenza funzionale:

$$\texttt{Impiegato Progetto} \rightarrow \texttt{Progetto}$$

in quanto due tuple con gli stessi valori sulla coppia di attributi `Impiegato` e `Progetto` hanno lo stesso valore sull'attributo `Progetto`. Questa è una dipendenza funzionale banale perchè asserisce una proprietàovvia di una relazione.
Le dipendenze funzionali dovrebbero servire a descrivere proprietà significative dell'applicazione che stiamo rappresentando. Diremo che una dipendenza funzionale $Y \rightarrow A$ è *non banale* se A non compare tra gli attributi di Y.
Un'ultima osservazione sulle dipendenze funzionali riguarda il loro legame con il vincolo di chiave. Se prendiamo una chiave K di una relazione r, si può facilmente verificare che esiste una dipendenza funzionale tra K e ogni altro attributo dello schema di r. Con riferimento al nostro esempio, abbiamo detto che gli attributi `Impiegato` e `Progetto` formano una chiave. Possiamo affermare che vale la dipendenza funzioanale `Impiegato Progetto` $\rightarrow$ `Funzione`. Possiamo concludere dicendo che il vincolo di dipendenza funzionale *generelizza* il

vincolo di chiave. Più precisamente, possiamo dire che una dipendenza funzionale $Y \rightarrow Z$ su uno schema $R\left(X\right)$ degenera nel vincolo di chiave se l'unione di Y e Z è pari a X.

9.3 Forma normale di Boyce e Codd

9.3.1 Definizione di forma normale di Boyce e Codd

In riferimento alla dipendenze funzionali, si possono introdurre delle proprietà dette *forme normali* che sono soddisfatte quando non ci sono anomalie.
Nel nostro esempio, le due proprietà casusa di anomalie corrispondono esattamente ad attributi coinvolti in dipendenze funzionali.

- La proprietà "lo stipendio di ciascun impiegato è funzione del solo impiegato, indipendentemente dai progetti cui partecipa" implica il soddisfacimento della dipendenza funzionale `Impiegato` → `Stipendio`.
- La proprietà "il bilancio di ciascun progetto dipende solo dal progetto, indipendentemente dagli impiegati che vi partecipano" corrisponde alla dipendenza funzionale `Progetto` → `Bilancio`.

Inoltre, l'attributo `Funzione` indica, per ciascun tupla, il ruolo svolto dall'impiegato nel progetto. Questo ruolo è unico per ciascuna coppia impiegato-progetto. Anche questa proprietà può essere modellata per mezzo di una dipendenza funzionale.

- La proprietà "in ciascun progetto, ciascuno degli impiegati coinvolti puo svolgere una e una sola funzione" implica il soddisfacimento della dipendenza funzionale `Impiegato Progetto` → `Funzione`

Abbiamo visto che la prime due proprietà generano ridondanze e anomalie indesiderate. La terza dipendenza non genera ridondanze perchè, dato che `Impiegato Progetto` è la chiave, la relazione non può contenere due tuple uguali su questi attributi. Per quanto riguarda le anomalie, da un punto di vista concettuale essa non ne può generare, in quanto ogni impiegato ha uno stipendio e ogni progetto ha un bilancio, e quindi per ogni coppia impiegato-progetto è possibile avere valori univoci per tutti gli altri attributi della relazione.
Possiamo quindi concludere che le ridondanze e le anomalie sono causate dalle dipendenze funzionali $X \rightarrow A$ che permettono la presenza di più tuple fra loro

uguali sugli attributi in X, cioè X non contiene una chiave.
Precisiamo queste idee per mezzo della più importante delle forme normali, detta di Boyce e Codd, dal nome dei suoi ideatori. Una relazione r è in *forma normale di Boyce e Codd* se per ogni dipendenza funzionale $X \to A$ definita su di essa, X contiene una chiave K di r, cioè X è superchiave per r.

9.3.2 Decomposizione in forma normale di Boyce e Codd

Data una relazione che non soddisfa la forma normale di Boyce e Codd è possibile, in molti casi, sostituirla con due o più relazioni normalizzate attraverso un processo di *normalizzazione*. Questo processo si basa su un semplice criterio: se una relazione rappresenta più concetti indipendenti, allora va decomposta in relazioni più piccole, una per ogni concetto.
Se alla relazione in figura 9.1 sostituiamo le seguenti tre relazioni:

Impiegato	Stipendio
Rossi	20 000
Verdi	35 000
Neri	55 000
Mori	48 000
Bianchi	48 000

Progetto	Bilancio
Marte	2000
Giove	15 000
Venere	15 000

Impiegato	Progetto	Funzione
Rossi	Marte	Tecnico
Verdi	Giove	Progettista
Verdi	Venere	Progettista
Neri	Venere	Direttore
Neri	Giove	Consulente
Neri	Marte	Consulente
Mori	Marte	Direttore
Mori	Venere	Progettista
Bianchi	Venere	Progettista
Bianchi	Giove	Direttore

ottenute per mezzo di proiezioni sugli insiemi di attributi rispettivamente corrispondenti ai tre concetti prima menzionati, eliminiamo anomalie e ridondanze: le tre relazioni sono infatti in forma normale Boyce e Codd.
Nell'esempio, la separazione delle dipendenze è stata facilitata dalla struttura delle dipendenze stesse, "naturalmente" separate e indipendenti l'una dall'altra. In molti casi pratici, la decomposizione può essere effettuata producendo tante relazioni quante sono le dipendenze funzionali definite. In generale, le dipendenze possono avere una struttura complessa: può non essere necessario basare la de-

composizione su tutte le dipendenze e può essere difficile individuare quelle su cui si deve basare la decomposizione.

9.4 Proprietà delle decomposizioni

Analizziamo come non tutte le decomposizioni siano desiderabili e individuando alcune proprietà essenziali che devono essere soddisfatte da una "buona" decomposizione.

9.4.1 Decomposizione senza perdita

Per discutere la prima proprietà, consideriamo la seguente relazione:

Impiegato	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Essa soddisfa le dipendenze funzionali:

$$\texttt{Impiegato} \rightarrow \texttt{Sede}$$

$$\texttt{Progetto} \rightarrow \texttt{Sede}$$

che specificano il fatto che ciascun impiegato opera presso un'unica sede e che ciascun progetto è sviluppato presso un'unica sede. Ciascun impiegato può partecipare a più progetti anche se, sulla base delle dipendenze funzioanli, devono essere tutti progetti assegnati alla sede cui afferisce.
Operando come nei casi precedenti, ovvero separando sulla base delle dipendenze, saremmo portati a decomporre la relazione in due parti:

- una relazione sugli attributi `Impiegato` e `Sede`, in corrispondenza alla dipendenza $\texttt{Impiegato} \rightarrow \texttt{Sede}$;
- l'altra sugli attributi `Progetto` e `Sede`, in corrispondenza alla dipendenza funzionale $\texttt{Progetto} \rightarrow \texttt{Sede}$

Otterremmo quindi:

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

Progetto	Sede
Marte	Roma
Giove	Milano
Saturno	Milano
Venere	Milano

Esaminiamo le due relazioni. In particolare, consideriamo come sia possibile ricostruire informazioni sulla partecipazione degli impiegati ai progetti. L'unica possibilità che abbiamo è di utilizzare l'attributo `Sede` , che è l'unico attributo comune alle due relazioni: possiamo cioè correlare un impiegato a un progetto se il progetto viene svolto nella sede presso cui l'impiegato opera. Purtroppo, in questo caso non riusciamo a ricostruire tutte e sole le informazioni nella relazione originaria: per esempio l'impiegato Verdi lavora a Milano e il progetto Saturno viene svolto presso la sede di Milano, ma Verdi non lavora a tale progetto.
La ricostruzione della relazione originaria a partire dalle sue proiezioni viene effettuata per mezzo di un'operazione di join naturale delle due proiezioni ma il join contiene tutte le tuple della relazione originaria più altre tuple.
La situazione nell'esempio corrisponde al caso generale: data una relazione r su un insieme di attributi X, se X_1 e X_2 sono due sottoinsiemi di X la cui unione sia pari a X stesso, allora il join delle deu relazioni ottenuteper proiezione da r su X_1 e X_2, è una relazione che contiene tutte le tuple di r, più eventualmente altre che possiamo chiamare "spurie". Diciamo che r si *decompone senza perdita* su X_1 e X_2 se il join delle due proiezioni è uguale a r stessa.
È possibile individuare una condizione che garantisce la decomposizione senza perdita di una relazione. Sia r una relazione su X e siano X_1 e X_2 sottoinsiemi di X tali che $X_1 \cup X_2 = X$; inoltre sia $X_0 = X_1 \cap X_2$; allora: r si decompone senza perdita su X_1 e X_2 se soddisfa la dipendenza funzionale $X_0 \rightarrow X_1$ oppure $X_0 \rightarrow X_2$.
Possiamo dire che r si decompone su due relazioni se l'insieme degli attributi comuni alle due relazioni è chiave per almeno una delle relazioni decomposte. Nell'esempio, possiamo vedere che l'intersezione degli insiemi di attributi su cui abbiamo effettuato le due proiezioni è costituita dall'attributo `Sede`, che non è il primo membro di alcuna dipendenza funzionale.

9.4.2 Conservazione delle dipendenze

Per introdurre la seconda proprietà esaminiamo la relazione nella figura 9.4.1. Volendo ancora rimuovere le anomalie, potremmo pensare di sfruttare solo la dipendenza `Impiegato` → `Sede` per ottenere una decomposizione senza perdita. Otteniamo in questa maniera due relazioni: una sugli attributi `Impiegato` e `Sede` e l'altra sugli attributi `Impiegato` e `Progetto`. Otteniamo, quindi:

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

Il join delle due relazioni produce la relazione di partenza, per cui possiamo dire che la relazione nella figura 9.4.1 si decompone senza perditua su `Impiegato`, `Sede` e `Impiegato`, `Progetto`. In effetti, `Impiegato` è chiave per la prima relazione, per cui la decomposizione senza perdita è garantita. La decomposizione, però, presenta un altro inconveniente. Supponiamo di voler inserire una nuova tupla che specifica la partecipazione dell'impiegato Neri, che opera a Milano, al progetto Marte. Sulla relazione originaria, un tale aggiornamento verrebbe individuato come illecito, perchè porterebbe a una violazione della dipendenza `Progetto` → `Sede`. Sulle relazioni decomposte invece, non è possibile rilevare alcuna violazione di dipendenze: sulla relazione avente per attributi `Impiegato` e `Progetto` non è possibile definire alcuna dipendenza funzionale e quindi non ci possono essere violazioni da rilevare, mentre nella relazione su `Impiegato` e `Sede` la tupla con valori Neri e Milano soddisfa la dipendenza funzionale `Impiegato` → `Sede`. Non è possibile, quindi, effettuare alcuna verifica sulla dipendenza `Progetto` → `Sede` perchè i due attributi `Progetto` e `Sede` sono stati separati.
Possiamo quindi concludere che, in ogni decomposizione, ciascuna delle dipendenze funzionali dello schema originario dovrebbe coinvolgere attributi che compaiono tutti insieme in uno degli schemi composti. In questo modo è possibile garantire, sullo schema decomposto, il soddisfacimento degli stessi vincoli il cui soddisfacimento è garantito dallo schema originario. Diremo che una decomposizione che soddisfa tale proprietà *conserva le dipendenze* dello schema originario.

9.4.3 Qualità delle decomposizioni

Le decomposizioni dovrebbero sempre soddisfare le proprietà di *decomposizione senza perdita* e *conservazione delle dipendenze*.

- La decomposizione senza perdita garantisce che le informazioni nella relazione originaria siano ricostruibili con precisione, senza informazioni spurie, a partire da quelle rappresentate nelle relazioni decomposte.
- La conservazione delle dipendenze garantisce che le relazioni decomposte hanno la stessa capacità della relazione originaria di rappresentare i vincoli di integrità e quindi rilevare aggiornamenti illeciti: a ogni aggiormento lecito sulla relazione originaria corrisponde un aggiornamento lecito sulle relazioni decomposte. Sulle relazioni decomposte, sono possimi ulteriori aggiornamenti, legati ai singoli concetti rappresentati in ciascuna di esse, che non hanno un corrispettivo sulla relazione originaria, senza però corrispondere a violazione dei vincoli.

Quindi, consderiamo accettabili solo le decomposizioni che soddisfano queste due proprietà.

9.5 Terza forma normale

9.5.1 Limitazioni della forma normale di Boyce e Codd

Consideriamo la seguente relazione:

Dirigente	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Marte	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Su di essa, possiamo supporre che siano definite le seguenti dipendenze:

- `Dirigente` → `Sede`: ogni dirigente opera presso una sede;
- `Progetto Sede` → `Dirigente`: ogni progetto ha più dirigenti che ne sono responsabili, ma in sedi diverse, e ogni dirigente può essere responsabile di più progetti; però, per ogni sede, un progetto ha un solo responsabile.

La relazione non è in forma normale di Boyce e Codd perchè il primo membro della dipendenza `Dirigente` → `Sede` non è superchiave. Inoltre, non è possibile effettuare alcuna buona decomposizione di questa relazione: infatti la dipendenza `Progetto Sede` → `Dirigente` coinvolge tutti gli attributi e quindi nessuna decomposizione è in grado di conservarla. Quindi, questo esempio mostra l'esistenza di schemi che violano la forma normale di Boyce e Codd per i quali non esiste alcuna decomposizione che conserva le dipendenze. Quindi, "la forma normale di Boyce e Codd non è raggiungibile".

9.5.2 Definizione di terza forma normale

Per trattare casi come quello dell'esempio precedente si ricorre a una forma normale meno restrittiva di quella di Boyce e Codd che non ammette ulteriori fonti di ridondanza e anomalia.
Diciamo che una relazione r è in *terza forma normale* se, per ogni dipendenza funzionale $X \rightarrow A$ definita su di essa, almeno una delle seguenti condizioni è verificata:

- X contiene una chiave K di r;
- A appartiene ad almeno una chiave di r.

Possiamo notare che lo schema precedente soddisfa la terza forma normale. Infatti, la dipendenza `Progetto Sede` → `Dirigente` ha come primo membro una chiave della relazione, mentre `Dirigente` → `Sede`, pur non contenendo una chiave al primo membro, ha un unico attributo al secondo membro che fa parte della chiave `Progetto Sede`.
La relazone presenta una forma di ridondanza in quanto ogni volta che un dirigente compare in una tupla, viene ripetuta per esso la sede in cui opera. Questa ridondanza, però, viene "tollerata" dalla terza forma normale perchè non sarebbe possibile una decomposizione che elimini tale ridondanza e al tempo stesso conservi tutte le dipendenze.
La terza forma normale è meno forte della forma normale di Boyce e Codd e quindi non offre le stesse qualità per una relazione; però rispetto alla forma normale di Boyce e Codd è sempre ottenibile.

9.5.3 Decomposizione in terza forma normale

Per ottenere una decomposizione in terza forma normale si può procedere come suggerito nel caso della forma normale di Boyce e Codd: una relazione che non

soddisfa la terza forma normale si decompone in relazioni ottenute per proiezione sugli attributi corrispondenti alla dipendenze funzionali, con l'unica accortezza di mantenere sempre una relazione che contiene una chiave della relazione originaria.
Consideriamo la seguente figura:

Impiegato	Progetto	Stipendio
Rossi	Marte	30000
Verdi	Giove	30000
Verdi	Giove	30000
Neri	Saturno	40000
Neri	Venere	40000

Per essa vale la sola dipendenza funzionale `Impiegato` → `Stipendio`. Una decomposizione in una relazione sugli attributi `Impiegato` e `Stipendio` e in un'altra sul solo attributo `Progetto` violerebbe la proprietà di decomposizione senza perdita, proprio perchè nessuna delle due relazioni contiene una chiave. Per garantire tale proprietà dobbiamo definire la seconda relazione sugli attributi `Impiegato` e `Progetto`, che formano una chiave della relazione originaria. La decomposizione è la seguente:

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

Impiegato	Stipendio
Rossi	30000
Verdi	30000
Neri	40000

Sulla prima relazione non sono definite dipendenze e la sua chiave è costituita da entrambi gli attributi.
Una decomposizione tesa in terza forma normale produce nalla maggior parte dei casi schemi in forma normale di Boyce e Codd. In particolare, se una relazione ha solo una chiave, allora le due forme normali coincidono, cioè una relazione in terza forma normale è anche in forma normale di Boyce e Codd.

9.5.4 Altre forme normali

Esistono altre forme normali a quelle di Boyce e Codd e alla terza forma normale. La prima forma normale stabilisce semplicemente una condizione che sta alla

base del modello relazionale stesso: gli attributi delle relazioni sono definite su valori atomici e non su valori complessi come insiemi o relazioni.
La seconda forma normale è una variante debole della terza e la introduciamo con un sempio. Consideriamo la seguente relazione:

Impiegato	Categoria	Stipendio
Neri	3	30000
Verdi	3	30000
Rossi	4	50000
Mori	4	50000
Bianchi	5	72000

Essa soddisfa le dipendenze `Impiegato` $\rightarrow$ `Categoria` e `Categoria` $\rightarrow$ `Stipendio` e quindi viola la terza forma normale, perchè `Categoria` non è chiave. La seconda forma normale tollera la dipendenza tra `Categoria` e `Stipendio`, perchè `Stipendio` dipende comunque dall'intera chiave `Impiegato`.
Riportiamo le definizioni originarie di seconda e terza forma normale.

- Una relazione è in seconda forma normale se su d essa non sono definite *dipendenze parziali*, cioè dipendenze fra un sottoinsieme proprio della chaive e altri attributi.

- Una relazione è in terza forma normale se su di essa non sono definite *dipendenze transitive*, cioè dipendenze della forma $K \rightarrow A$, dove K è la chiave ed esiste un altro sottoinsieme di attributi X, non chiave, con le dipendenze $K \rightarrow X$ e $X \rightarrow A$.

Le dipendenze parziali e quelle transitive violano la terza forma normale così come noi l'abbiamo definita perchè coinvolgono una dipendenza funzionale il cui primo membro non è superchiave e il cui secondo membro non fa parte della chaive.
Inoltre, esistono altre forme normali che fanno riferimento a vincoli di integrità diversi dalle dipendenze funzionali. Tutte queste forme normali vengono poco usate nelle applicazioni odierne in quanto è stato rilevato che la terza forma normale e la forma nornmale di Boyce e Codd forniscono il giusto compromesso tra semplicità e qualità dei risultato.

Quarta forma normale Prima di introdurre la definizione di quarta forma normale, introduciamo il concetto di dipendenza multivalente. Una dipendenza

multivalente, MVD, si ha quando un attributo determina in modo indipendente più attributi, ad esempio: `Prodotto` $\twoheadrightarrow$ `Taglia` e `Prodotto` $\twoheadrightarrow$ `Colore`. Dati U e V sottinsiemi di r, se W è l'insieme di attributi di r non compresi in U e in V, la MVD $U \twoheadrightarrow V$ esiste in r se e solo se:

$$r = r\left(UV\right) \text{ JOIN } r\left(W\right)$$

La MVD è una generalizzazione della dipendenza funzionale in quanto tutte le dipendenze funzionali sono dipendenze multivalente e non tutte le dipendenze multivalente sono dipendenze funzionali.
Se la dipendenza multivalente è una dipendenza funzionale non causa ridondanze.
Una relazione r è in quarta forma normale se e solo se, se compare una dipendenza multivalente $A \twoheadrightarrow B$, tutti gli attributi di r dipendono anche funzionalmente da A.

Quinta forma normale Prima di introdurre la quinta forma normale, introduciamo il concetto di dipendenza da join, JD. La dipendenza da join non è ne una dipendenza funzionale nè una dipendenza multivalente. Una relazione $r\nabla X, Y, Z\partial$ soddisfa la JD se e solo se:

$$R = \pi\left(X\right) \text{ JOIN } \pi\left(Y\right) \text{ JOIN } \pi\left(Z\right)$$

La dipendenza da join è la dipendenza più generale in quanto la dipendenza funzionale è un caso particolare di una dipendenza multivalente e la dipendenza multivalente è un caso particolare di dipendenza da join.
Una relazione r è in quinta forma normale se e solo se ogni dipendenza di join presente in essa è conseguenza delle sue chiavi candidate.
Se la dipendenza di join è conseguenza delle chiavi candidate, la relazione è già in quinta forma normale, ad esempio data la relazione `Dipendenti(matricola, CF, nome)` matricola e CF sono chiavi candidate e si può proiettare (matricola, CF), (CF, nome) e (matricola, nome) e ricostruire con join su matricola e CF. La relazione `Dipendenti` contiene una dipendenza da join, ma non contiene anomalie quindi è gia in quinta forma normale.

9.5.5 Normalizzazione e scelta degli attributi

Consideriamo la seguente relazione:

Dirigente	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Marte	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Esaminando meglio le specifiche, avremmo potuto descrivere l'applicazione di interesse in maniera più appropriata introducendo un ulteriore attributo `Reparto`, che partiziona le singole sedi, ottenendo la seguente relazione:

Dirigente	Progetto	Sede	Reparto
Rossi	Marte	Roma	1
Verdi	Giove	Milano	1
Verdi	Marte	Milano	1
Neri	Saturno	Milano	2
Neri	Venere	Milano	2

Le dipendenze, in questo caso, possono essere così definite:

- `Dirigente` → `Sede Reparto`: ogni dirigente opera presso una sede e dirige un reparto;
- `Sede Reparto` → `Dirigente`: per ogni sede e reparto c'è un solo dirigente;
- `Progetto Sede` → `Reparto`: per ogni sede, un progetto è assegnato a un solo reparto; la dipendenza funzionale `Progetto Sede` → `Dirigente` è quindi ricostruibile.

Per questo schema esiste una buona decomposizione. Infatti:

- la decomposizione è senza perdita, perchè gli attributi `Sede` e `Reparto` forma una chiave per la prima relazione;
- le dipendenze sono conservate, perchè per ciascuna dipendenza esiste una relazione decomposta che ne contiene tutti gli attributi;
- entrambe le relazioni sono in forma normale di Boyce e Codd, perchè tutte le dipendenza hanno il primo membro costituito da una chiave.

La decomposizione è la seguente:

Dirigente	Sede	Reparto
Rossi	Roma	1
Verdi	Milano	1
Neri	Milano	2

Progetto	Sede	Reparto
Marte	Roma	1
Giove	Milano	1
Marte	Milano	1
Saturno	Milano	2
Venere	Milano	2

Quindi, possiamo concludere che la non raggiungibilità della forma normale di Boyce e Codd può essere dovuta a un'analisi non sufficientemente accurata dell'applicazione.

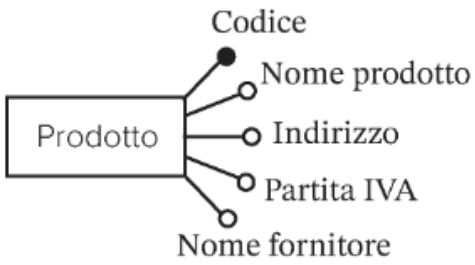
9.6 Progettazione di basi di dati e normalizzazione

La teoria della normalizzazione può essere utilizzata come base per operazioni di verifica di qualità di schemi, sia nella fase di progettazione concettuale sia in quella di progettazione logica.
Le attività di progettazione sono sepre soggette a errori e incompletezze, e quindi una revisione delle relazioni ottenuti in sede di progettazzione logica pùo portare all'individuazione di imprecisioni nella formulazione dello schema concettuale: la verifica spesso risulta semplice, perchè l'individuazione delle dipendenze funzionali e delle chiavi deve essere svolta nell'ambito di una singola relazione, che corrisponde a un'entità o un'associazione già analizzata nella progettazione concettuale. In questi contesti, la struttura delle dipendenze risulta relativamente semplice ed è quindi possibile individuare direttamente la decomposizione necessaria per ottenere la terza forma normale.

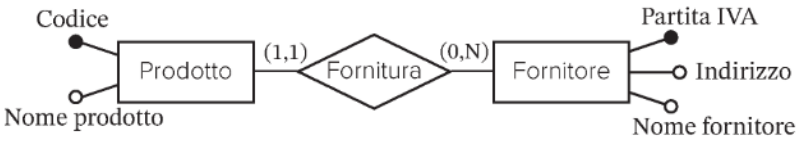
9.6.1 Verifiche di normalizzazione su entità

Le idee di base della normalizzazione possono essere utilizzate anche durante la fase di progettazione concettuale. È possibile considerare ciascuna entità e ciascuna associazione come una relazione. In particolare, la relazione che corrisponde a un'entità ha attributi che corrispondono esattamente agli attributi dell'entità. Per le entità con identificatore esterno sono necessari ulteriori attributi in corrispondenza alle entità che partecipano all'identificazione.
La verifica di normalizzazione può procedere come stato visto finora. È sufficiente considerare le dipendenze funzionali che sussistono fra gli attributi dell'entità

e verficare che ciascuna di esse abbia come primo membro l'identificatore. Per esempio, consideriamo la seguente entità:

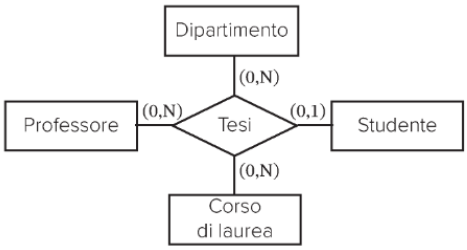


Nell'individuare le dipendenze relative a tale entità, possiamo notare che possono esistere fornitori diversi con lo stesso nome o stesso indirizzo, mentre tutte le proprietà di ogni fornitore sono identificate dalla partita IVA: quindi esiste la dipendenza funzionale `PartitaIVA` → `NomeFornitore, Indirizzo`. Inoltre, tutti gli attributi dipendono dall'attributo `Codice`. Poichè l'unico identificatore dell'entità è l'attributo `Codice`, possiamo concludere che l'entità viola la terza forma normale, in quanto la dipendenza `PartitaIva` → `NomeFornitore, Indirizzo` ha un primo membro che non contiene l'identificatore e un secondo membro composto da attributi che nno fanno parte della chiave. In questi casi, la verifica di normalizzazione ci permette di segnalare il fatto che lo schema concettuale non è accurato e ci suggerisce di decomporre l'unità stessa.
La decomposizione può avvenire con diretto riferimento alle dipendenze oppure, più semplicemente, ragionando sui concentti rappresentati dall'entità insieme a quelli che derivano dalle dipendenze funzionali.
In questo caso, il concetto di fornitore è indipendente da quello di prodotto e ha proprietà associate, quindi è opportuno modellare il concetto di fornitore per mezzo di un'entità, con identificatore costituito dall'attributo `PartitaIVA` e ulteriori attributi `NomeFornitore` e `Indirizzo`.
Se separiamo i concetti di `Prodotto` e `Fornitore` in due entità è opportuno che queste siano correlate, ossia che sussista un'associazione tra esse.
La decomposizione ottenuta soddisfa la due proprietà fondamentali: è una decomposizione senza perdita, perchè sulla base dei fornitori è possibile ricostruire i valori degli attributi dell'entità originaria, e conserva le dipendenze, perchè ciascuna delle dipendenze è contenuta in una delle entità o è ricostruibile da esse.
La decomposizione ottenuta è la seguente:



9.6.2 Verifiche di normalizzazione su associazioni

Per quanto riguarda le associazioni, il ragionamento è per certi aspetti più semplice, perchè l'insieme delle occorrenze di ciascuna associazione è una relazione e quindi è possibile applicare direttamente i concetti connessi con le forme normali, ma per altri più complesso, perchè i domini su cui tale relazione è definita sono gli insiemi delle occorrenze delle entità coinvolte. Per verificare il soddisfacimento della terza forma normale, è necessario individuare le dipendenze funzionali che sussistono fra le entità coinvolte. Dato che ogni relazione binaria è in terza forma normale e anche in forma di Boyce e Codd, la verifica di normalizzazione va effettuata solo sulle associazioni non binarie, cioè su quelle che coinvolgono almeno tre entità.
Consideriamo la seguente associazione:



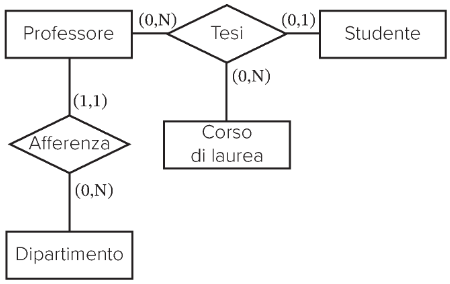
Lo scopo di quest'associazione è descrivere il fatto che gli studenti, iscritti ai corsi di laurea, svolgono testi presso i dipartimenti sotto la guida dei professori. Esaminando l'associazione, possiamo arrivare alle seguenti conclusioni:

- ogni studente è iscritto a un solo corso di laurea;
- ogni studente svolge una tesi sotto la suervisione di un solo professore;
- ogni professore afferisce a un solo dipartimento e gli studenti sotto la sua supervisione svolgono la tesi presso tale dipartimento.

Le proprietà dell'applicazione di interesse sono descritte in modo esauriente da tre dipendenze funzionali:

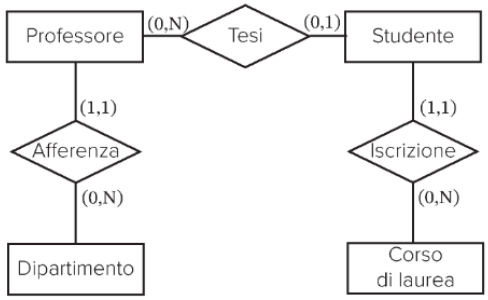
$$\texttt{Studente} \rightarrow \texttt{CorsoDiLaurea}$$
$$\texttt{Studente} \rightarrow \texttt{Professore}$$
$$\texttt{Professore} \rightarrow \texttt{Dipartimento}$$

La chiave della relazione risulta essere costituita da `Studente`: dato uno studente sono univocamente individuati il corso di laurea, il professore e il dipartimento. Di conseguenza, la terza dipendenza funzionale, $\texttt{Professore} \rightarrow \texttt{Dipartimento}$, causa una violazione della terza forma normale. In effetti, l'afferenza di un professore a un dipartimento è un concetto indipendente dall'esistenza di studneti che svolgono la tesi con il professore stesso. Quindi, l'assocazione presenta aspetti indesiderabili e che va quindi decomposta, separando le dipendenze funzionali con primi membri diversi. La decomposizione che si ottiene è la seguente:



9.6.3 Ulteriori decomposizioni di associazioni

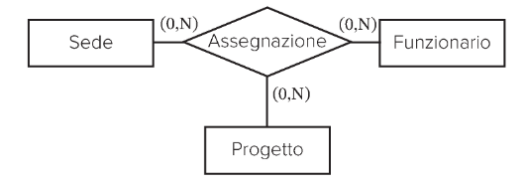
Considerando l'associazione nella figura 9.6.2 possiamo fare alcune considerazioni aggiuntive. L'associazione `Tesi` è in terza forma normale, perchè la sua chiave è costituita dall'entità `Studente` e le uniche dipendenze sono quelle che hanno la chiave `Studente` come primo membro, e cioè $\texttt{Studente} \rightarrow \texttt{Professore}$ e $\texttt{Studente} \rightarrow \texttt{CorsoDiLaurea}$. Le proprietà descritte dalle due dipendenze sono fra loro indipendenti: non tutti gli studenti stanno svolgendo una testi e quindi non tutti hanno un relatore. Dal punto di vista della normalizzazione, questa situazione non presenta problemi, perchè si assume che le relazioni possano contenere valori nulli, purchè non nella chiave. D'altra parte, a livello di modellazione concettuale è necessario distinguere i vari concetti. Attraverso le dipendenze possiamo notare che sarebbe opportuno decomporre ulteriorormente l'associazione, ottenendo il seguente schema:



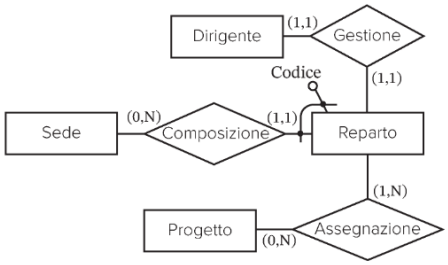
La decomposizione è accettabile perchè conserva le dipendenze ed è senza perdita.
È opportuno, quindi, decomporre le associazioni non binarie sulle quali sia definita qualche dipendenza il cui secondo membro contiene più di un'entità, ovvero è opportuno decomporre un'associazione ternaria se su di essa è definita una dipendenza funzionale il cui primo membro è costituito da un'entità e il secondo membro dalle altre due.

9.6.4 Ulteriori decomposizioni di schemi concettuali

Anche il caso in cui non esiste, per una relazione, una buona decomposizione in forma normale di Boyce e Codd, può essere esaminato nell'ambito della progettazione concettuale. Supponiamo di avere il seguente schema:



In questo schema valgono le seguenti dipendenze fra entità: `Dirigente` → `Sede` e `Progetto Sede` → `Dirigente`. L'associazione non è in forma normale di Boyce e Codd e non può essere utilmente decomposta. Possiamo, però, individuare la possibilità di introdurre il concetto di reparto, per mezzo di una nuova entità, come mostrato nel seguente schema:



Quest'entità partiziona le singole sedi, come indicato dal suo identificatore esterno. Inoltre, i vincoli di cardinalità ci dicono che ogni reparto di una sede ha associato un direttore e diversi progetti.